

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 12.09.2026, 19:01:48
Файлов исходного кода: 83
Суммарный объём кода: 1.17 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/83: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow dispatch:

```
 path: ./dist
```

```
 deploy:
```

```
 name: Publish to GitHub Pages
```

```
 needs: build
```

```
 runs-on: ubuntu-latest
```

```
 environment:
```

```
 name: github-pages
```

```
 url: ${ steps.deployment.outputs.page_url }
```

```
 steps:
```

```
 - name: Deploy
```

```
 id: deployment
```

```
 uses: actions/deploy-pages@v5
```

```
....
```

```
Patch относительно `main`
```

```
````diff
```

```
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
```

```
new file mode 100644
```

```
index 00000000..17418e3
```

```
--- /dev/null
```

```
+++ b/.github/workflows/deploy-pages.yml
```

```
@@ -0,0 +1,60 @@
```

```
+name: Deploy app to GitHub Pages
```

```
+
```

```
+on:
```

```
+  push:
```

```
+    branches:
```

```
+      - main
```

```
+      # Позволяет проверить Pages прямо из рабочей ветки
```

```
+      - "arena/**"
```

```
+  workflow_dispatch:
```

```
+
```

```
+permissions:
```

```
+  contents: read
```

```
+  pages: write
```

```
+  id-token: write
```

Универсальное пособие • полный исходный код

```
-----
+   runs-on: ubuntu-latest
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+   ```
```

`github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
|
|→ page-0001.png ... page-NNNN.png (
|
|→ full_code_all_page
#
```

# Готовые TXT и PDF коммитятся обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

- ```
    for policy in /etc/ImageMagick-6/policy.xml /
      if [ -f "$policy" ]; then
        sudo sed -i 's/rights="none" pattern="\{P
      fi
    done
    convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full\_code\_all\_pages`"
 echo "| \`public/export/full\_code\_all\_pages`"
 echo "| PNG-страниц | \$(ls tmp/export-pages) |"
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)

```
    steps:
      - name: Checkout
      - uses: actions/checkout@v4
      + uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
      - uses: actions/setup-node@v4
      + uses: actions/setup-node@v7
        with:
      -   node-version: "22"
      +   node-version: "24"
          cache: npm

      - name: Install ImageMagick + DeJaVu fonts
@@ -97,7 +97,7 @@ jobs:
        } >> "$GITHUB_STEP_SUMMARY"

      - name: Upload artifacts (PDF + TXT)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-export
          path: |
@@ -107,7 +107,7 @@ jobs:
          retention-days: 30

      - name: Upload artifacts (PNG-страницы)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-pages-png
          path: tmp/export-pages/*.png
    ....
```

Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

### Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```

`src/App.tsx`

Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
import { chapters } from '../data/content';
import { Navbar } from '../components/Navbar';
import { Sidebar } from '../components/Sidebar';
import { MobileToc } from '../components/MobileToc';
import { ChapterView } from '../components/ChapterView';
```



```

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" style={{ gap: 10px }}>
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" style={{ padding: 10px }}>
 <Sidebar selectedChapterId={activeChapterId} chapters={chapters} />
 </div>

 <main id="main-content" className="flex-1 min-w-0" style={{ padding: 10px }}>
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" style={{ padding: 10px 0 0 0 }}>
 <div className="min-w-0">
 <div className="text-[10px] uppercase" style={{ padding: 0 10px 0 0 }}>
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" style={{ padding: 0 10px 0 0 }}>
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded" style={{ padding: 10px 0 0 0 }}>
 <div
 className="h-full bg-gradient-to-r from-slate-800 to-slate-700"
 style={{ width: `${progress}%` }}
 />
 </div>
 </div>
 </div>
 </>
) : null}
 </Navbar>
 </div>
)

```

-----  
````  

Patch относительно `main`

````diff

diff --git a/src/App.tsx b/src/App.tsx

index 2091f33..4583756 100644

--- a/src/App.tsx

+++ b/src/App.tsx

@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/SimulatorModal";

import { SimulatorHub } from "../components/SimulatorHub";

+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {

- const [activeTab, setActiveTab] = useState<"guide" | "simulator">();

+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">();

const [activeChapterId, setActiveChapterId] = useState<string>();

const [modalVizId, setModalVizId] = useState<string>();

@@ -95,7 +96,7 @@ export default function App() {

</main>

</div>

</>

- ) : (

+ ) : activeTab === "simulator" ? (

<main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">

<SimulatorHub

onOpen={setModalVizId}

@@ -105,6 +106,8 @@ export default function App() {

}}

/>

</main>

+ ) : (

+ <PythonCompiler />

}}

```

 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-mono">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
 </button>
 <button
 id="tab-projects-btn"
 onClick={() => setActiveTab('projects')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'projects'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Code className="w-4 h-4" /> Проекты
 </button>
 </div>
</div>

```

```

 disabled={busy}
 className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
 </header>
);
};
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components.
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

  export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
  >

```

Универсальное пособие • полный исходный код

import { CheckCircle2, ExternalLink, Info, Loader2, Play

```
const PYODIDE_VERSION = "0.27.7";  
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo  
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
```

```
const STARTER_CODE = `# Первый запуск загрузит Python в  
name = "алгоритмы"
```

```
for number in range(1, 4):  
    print(f"{number}. Учим {name}!")  
`;  
;
```

```
type PyodideRuntime = {  
  runPythonAsync: (source: string) => Promise<unknown>;  
  setStdout: (options: { batched: (message: string) => ;  
  setStderr: (options: { batched: (message: string) => ;  
  setStdin: (options: { stdin: () => string | number | ;  
};
```

```
type PyodideModule = {  
  loadPyodide: (options: { indexURL: string }) => Promi  
};
```

```
let runtimePromise: Promise<PyodideRuntime> | null = nu
```

```
function getPythonRuntime() {  
  if (!runtimePromise) {  
    runtimePromise = import(/* @vite-ignore */ PYODIDE_I  
      (module as PyodideModule).loadPyodide({ indexURL:  
    });  
  }  
  return runtimePromise;  
}
```

```
function errorText(error: unknown) {  
  if (error instanceof Error) return error.message;  
  return String(error);  
}
```

```

        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:p
    <div className="max-w-6xl mx-auto">
        <div className="flex flex-col sm:flex-row sm:it
            <div>
                <div className="inline-flex items-center gap
                    <Terminal className="w-4 h-4" /> Боковая
                </div>
                <h2 className="text-2xl sm:text-3xl font-ex
                <p className="text-slate-400 mt-2 max-w-2xl
                    Пишите небольшой код и запускайте его пря
                    который переводит CPython в WebAssembly.
                </p>
            </div>
            <a
                href="https://pyodide.org/"
                target="_blank"
                rel="noreferrer"
                className="inline-flex items-center gap-1.5
            >
                Как это работает <ExternalLink className="w
            </a>
        </div>

        <div className="grid xl:grid-cols-[minmax(0,1.3
            <section className="rounded-2xl border border
                <div className="flex flex-wrap items-center
                    <div className="flex items-center gap-2">

```

```

        -none focus:ring-2 focus:ring-inset focus:ring-slate-400
        placeholder="Напишите Python-код..."
    />

    <div className="border-t border-slate-800 border-b border-slate-800 border-l border-slate-800 border-r border-slate-800" >
        <label className="block px-4 pt-3 text-slate-400" >
            Ввод для input() • по одной строке на клавишу
        </label>
        <textarea
            id="python-stdin"
            value={stdin}
            onChange={(event) => setStdin(event.target.value)}
            spellCheck={false}
            rows={2}
            placeholder={'Например:\nАлиса'}
            className="block w-full resize-y bg-transparent border border-slate-800"
        />
    </div>
</section>

<section className="rounded-2xl border border-slate-800" >
    <div className="flex items-center justify-between" >
        <div className="flex items-center gap-2" >
            <Terminal className="w-4 h-4 text-emerald-500" />
        </div>
        <span className={`inline-flex items-center gap-2`} >
            {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
            {runtimeReady ? "готов" : "не загружен"}
        </span>
    </div>
    <pre className="min-h-[360px] max-h-[560px] px-4 py-4 leading-6 text-slate-300" >
        {output}
    </pre>
    <div className="border-t border-slate-800 border-b border-slate-800 border-l border-slate-800 border-r border-slate-800" >
    </div>
</section>
</div>

```

```
+name = "алгоритмы"
+
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>
+  setStdout: (options: { batched: (message: string) =>
+  setStderr: (options: { batched: (message: string) =>
+  setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Prom
+});
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE
+      (module as PyodideModule).loadPyodide({ indexURL
+    });
+  }
+  return runtimePromise;
+}
+
+function errorText(error: unknown) {
+  if (error instanceof Error) return error.message;
+  return String(error);
+}
+
+export function PythonCompiler() {
+  const [code, setCode] = useState(STARTER_CODE);
+  const [stdin, setStdin] = useState("");
+  const [output, setOutput] = useState("Нажмите «Запус
+  const [running, setRunning] = useState(false);
```



```

+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+   };
+
+   return (
+     <main className="max-w-screen-2xl mx-auto px-4 sm:p
+     <div className="max-w-6xl mx-auto">
+       <div className="flex flex-col sm:flex-row sm:i
+       <div>
+         <div className="inline-flex items-center g
+           <Terminal className="w-4 h-4" /> Боковая
+         </div>
+         <h2 className="text-2xl sm:text-3xl font-e
+         <p className="text-slate-400 mt-2 max-w-2x
+           Пишите небольшой код и запускайте его пр
+           который переводит CPython в WebAssembly.
+         </p>
+       </div>
+       <a
+         href="https://pyodide.org/"
+         target="_blank"
+         rel="noreferrer"
+         className="inline-flex items-center gap-1.5
+       >
+         Как это работает <ExternalLink className="
+       </a>
+     </div>
+
+     <div className="grid xl:grid-cols-[minmax(0,1.
+     <section className="rounded-2xl border borde
+     <div className="flex flex-wrap items-cente
+       <div className="flex items-center gap-2">
+         <span className="w-2.5 h-2.5 rounded-fr
+         <span className="text-sm font-bold tex
+         <span className="text-[11px] text-slate
+       </div>
+       <div className="flex items-center gap-2">
+         <button
+           type="button"

```

```

+         </label>
+         <textarea
+             id="python-stdin"
+             value={stdin}
+             onChange={(event) => setStdin(event.target.value)}
+             spellCheck={false}
+             rows={2}
+             placeholder={'Например:\nАлиса'}
+             className="block w-full resize-y bg-tran
ceholder:text-slate-700"
+         />
+     </div>
+ </section>
+
+     <section className="rounded-2xl border border-slate-200" >
+         <div className="flex items-center justify-between" >
+             <div className="flex items-center gap-2" >
+                 <Terminal className="w-4 h-4 text-emerald-500" />
+             </div>
+             <span className={`inline-flex items-center gap-2`} >
+                 {runtimeReady && <CheckCircle2 className="text-emerald-500" />}
+                 {runtimeReady ? "готов" : "не загружен"}
+             </span>
+         </div>
+         <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+             {output}
+         </pre>
+         <div className="border-t border-slate-800 p-4" >
+         </div>
+     </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3" >
+     <div className="flex gap-2 rounded-xl border border-slate-200 p-4" >
+         <Info className="w-4 h-4 shrink-0 text-indigo-500" />
+         <span>Ctrl или Cmd + Enter запускает код. l
+     </div>
+     <div className="flex gap-2 rounded-xl border

```

```

    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});

```

Patch относительно `main`

```

````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {

```

## Универсальное пособие • полный исходный код

- Определение/правило в центре (``border-blue-500`` или ``border-blue-500`` или ``border-blue-500``)
  - Обязательная сетка аналогий (2 колонки: неформальный текст, код)
  - "Душные" детали (код, формулы, сложные доказательства)
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы. Используй только классы в файлах ``src/data/content.ts``. Не используй чистый `markdown`.

ФАЙЛ 4/83: `index.html`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512 байт

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
 <title> Дискретка для тупых – Интерактивный гид по дискретке
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen">
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

ФАЙЛ 5/83: `metadata.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312 байт

```
{
 "name": "Remix Remix: ExamPrep Reader",
 "description": "Удобная web-читалка для подготовки к экзаменам",
 "requestFramePermissions": [],
 "majorCapabilities": [
 "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
]
}
```

## Универсальное пособие • полный исходный код

```

 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
 },
 "description": "Интерактивное пособие по алгоритмам и
}
```

-----  
ФАЙЛ 7/83: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.5 КБ

-----  
# algv0 – Универсальное пособие по алгоритмам и структурам

Интерактивная web-читалка (React + Vite + Tailwind) с синтаксисом  
и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash  
npm install  
npm run dev          # предпросмотр на http://0.0.0.0:5174  
npm run build        # сборка в dist/ (single-file)  
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в один файл

| Команда              | Шаг       | Результат                     |
|----------------------|-----------|-------------------------------|
| ---                  | ---       | ---                           |
| `npm run export:txt` | код → txt | `public/export/full-code.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/...         |

## Универсальное пособие • полный исходный код

- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы а известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается кар мини-анимацией и кнопкой «Показать симулятор» (симуля Словарь – `src/data/glossary.ts`, мини-анимации – `src
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

### ## Структура

...

|                    |                                  |
|--------------------|----------------------------------|
| src/               |                                  |
| App.tsx            | – оболочка, привязка глав к визу |
| components/        | – интерактивные симуляторы (Дейк |
| data/              | – контент пособия (главы/билеты  |
| scripts/export/    | – пайплайн код → txt → png → pdf |
| public/export/     | – сгенерированные TXT и PDF (обн |
| .github/workflows/ | – автоматизация                  |

...

### # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструмен азуемые теги и классы.

### ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и
- \* Подзаголовки: Используют тег `

### `.

### ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика раграфы `

`.

### ## Математические формулы

В приложении используется MathJax.

Универсальное пособие • полный исходный код

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

|                    |           |                      |
|--------------------|-----------|----------------------|
| Глава              | id        | Что делать           |
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

|               |                                |                                        |
|---------------|--------------------------------|----------------------------------------|
| Билет         | Тема                           | Статус                                 |
| ---           | ---                            | ---                                    |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. Контент вхолостую.          |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной главы нет; аналогии**.        |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть без аналогии. |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; есть без аналогии. |

Дополнительно — «осиротевшие» визуализаторы без глав (контент есть)

- \* `stack-dfs` → `StackViz.tsx` — **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` — **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` — **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` — **\*\*Динамическое программирование\*\***

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухая математика»)

|       |     |             |
|-------|-----|-------------|
| Глава | id  | Комментарий |
| ---   | --- | ---         |

## Универсальное пособие • полный исходный код

|    |                                      |   |   |   |
|----|--------------------------------------|---|---|---|
| 16 | 16. Графы. Кратчайший путь. Флойд    | - | - | - |
| 17 | 17. Графы. Алгоритм Джонсона         | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал  | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима    | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка  | 1 | - | - |
| 21 | 21. Префикс-функция (п), КМП         | 4 | - | - |
| 22 | 22. Z-функция                        | 4 | - | - |
| 23 | 23. Алгоритм Ахо-Корасик             | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - |
| -  | Обзор: Кратчайшие пути и Графы       | - | - | - |
| -  | Алгоритмы на карте                   | - | - | - |
| -  | Эйлер: Путь ≠ Цикл                   | - | - | - |
| -  | Мосты в графах: код + визуализация   | 1 | - | - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be``.  
это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» – минимум `inli`
4. **\*\*Решить судьбу `alg-map`\*\*** – раскрыть или удалить.

ФАЙЛ 9/83: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
```



```

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

=====

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

-----

ФАЙЛ 11/83: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 KB

-----

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
```

```

const progress = useMemo(() => ((index + 1) / chapter

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 m-0 p-4" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0">
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded-t" />
 </main>
 </div>
 </>
) : null}
 </Navbar>
 </div>
)

```

}

ФАЙЛ 12/83: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРӨК: 124 | РАЗМЕР: 5.0 КБ

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

const escapeRe = (s: string) => s.replace(/[\.*+?^${}()|]/g, "\\$&");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
 const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l));
 // границы «не буква/цифра» – \b не работает с кириллицей
 return new RegExp(`(?<![\\p{L}\\p{N}])(?=${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * , чтобы на
```

```

 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("div"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("div"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
 const run = () => {
 const el = ref.current;
 if (!el) return;
 try {

```

```
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
 from {
 transform: translateX(-100%);
 }
 to {
 transform: translateX(0);
 }
}

@keyframes hintIn {
 from {
```

```
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}

.chapter-body :where(pre, code) {
 white-space: pre-wrap;
 word-break: break-word;
}

.chapter-body pre {
 overflow-x: auto;
 max-width: 100%;
 font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
 display: block;
 width: 100%;
 overflow-x: auto;
 border-collapse: collapse;
 font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
 cursor: pointer;
 padding: 0.35rem 0;
}

@media (max-width: 767px) {
 .chapter-body :where(.grid) {
 grid-template-columns: minmax(0, 1fr) !important;
 }
 .chapter-body :where(h2) {
```

```
@keyframes demoProgress {
 from {
 width: 0%;
 }
 to {
 width: 100%;
 }
}
```

---

ФАЙЛ 14/83: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/83: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
```

```

import { Chapter } from "../types";

export const additionalTickets: Chapter[] = [
 {
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные заготовки
 ь отрезок длины 7 (перекрывая друг друга по 1 см)
 ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
]

```



```
<p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 двоичная куча. Пара – точка на декартовой
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Постр
 платит $\approx n \cdot \log n$ сравнений, counting по ма
 точка вставляется спуском от корня: $x \leq \text{уз}$
 единственно. Весь процесс – в симуляторе под
 «соединить сам» – кликаешь две точки,
 ь родитель», «получился бы цикл», «равные »,
 «Миф $2k/2k+1$ » и «Поиск $\neq$ сортировка»
 ждый запрещённый ход.</p>
```

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">

 ext-2xl text-slate-300">Й
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">Sp
 х»». Спуск от корня: узел, который вместе
 скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
```

```

 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}
}

```

</div>  
</details>

<details class="bg-slate-900/40 rounded-lg l  
<summary class="p-4 font-bold text-slate  
-b border-slate-700/50">

Код: вставка, merge, erase (Скрыт

</summary>

<div class="p-5 text-sm text-slate-300  
<pre class="bg-slate-950 p-4 rounded

```

def insert(root, key, pri):
 if root is None:
 return Node(key, pri)
 if key <= root.key: # равные x — левый
 root.left = insert(root.left, key, pri)
 else:
 root.right = insert(root.right, key, pri)
 return root

```

```

for key, pri in pairs_sorted_by_pri_desc: # порядок э
 root = insert(root, key, pri)

```

<pre class="bg-slate-950 p-4 rounded

```

def merge(a, b): # все x в a < все x в b
 if not a or not b: return a or b
 if a.pri > b.pri:
 a.right = merge(a.right, b); return a
 b.left = merge(a, b.left); return b

```

```

def erase(root, key): # удалить произвольн

```

```
 <p class="text-xs uppercase tracking-wider">О(1)</p>
 <p class="text-emerald-300 font-bold">О(1)</p>
 <p class="text-slate-400 text-xs mt-1">О(1)</p>
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">О(1)</p>
 <p class="text-white font-bold">О(1)</p>
 <p class="text-slate-400 text-xs mt-1">О(1)</p>
 </div>
</div>
```

<p class="text-slate-300 text-sm mb-4">От к  
ет  $O(n)$ . AVL и красно-чёрные выбирают <b>пр  
по факту</b>: ничего не хранит, дереву разр

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
 </div>
 <p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га

```
 </div>
 <p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
 </div>
```

```
</div>
```

```
<h4 class="text-lg font-bold text-white mt-1">
<p class="text-slate-300 text-sm mb-3">Пово
```

```

 <td class="py-3 pr-3 font-b
 <td class="py-3 pr-3">x и p
 <td class="py-3 pr-3"><span
 <td class="py-3">p и g стал

 </tr>
 </tbody>
</table>
</div>
<p class="text-slate-300 text-sm mt-3">И по
у. Zig случается только последним.</p>

<div class="bg-rose-950/30 rounded-xl borde
 <p class="text-rose-300 font-bold text-
 <pre class="bg-slate-950 p-4 rounded-lg
аивно (x, потом x): 60→40→20 стало 20→40→
Zig-Zig (p, потом x): 60→40→20 стало кустом 20(40(60),.
 <p class="text-slate-300 text-sm mt-3">И
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mo
изации не будет.</p>
</div>

<p class="text-slate-300 text-sm mt-6">Полн
– и дерево из линии высоты 4 стало кустом в
ли ключ, но и починили дерево по пути
: дорогой запрос нельзя пройти много раз по
ров поддеревьев падает ровно на столько, ск
хуже идеального статического дерева под тв

<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
 <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>

```

```
def splay(x): # поднять x в корень
 while x.parent:
 p, g = x.parent, x.parent.parent
 if g is None: # ZIG
 rotate_up(x)
 elif (g.left is p) == (p.left is x): # ZIG-ZIG
 rotate_up(p)
 rotate_up(x)
 else: # ZIG-ZAG: НИЖНЯЯ
 rotate_up(x)
 rotate_up(x)
 return x
```

```
def find(root, key): # поиск: спуск + splay
 cur, last = root, None
 while cur:
 last = cur
 if key == cur.key: break
 cur = cur.left if key < cur.key else cur.right
 return splay(last) if last else None
```

<p class="text-xs text-slate-400">В

Одна строчка – и вся амортизация.</p>

</div>

</details>

</div>

</div>

</section>`,

},

{

id: "prefix-sums-2d",

title: "5. Двумерная матрица префиксных сумм",

type: "html",

description: "Сжатие суммы подматрицы за  $O(1)$ ",

category: "Алгоритмы матрицы",

content: `

<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">

<div class="flex items-center mb-6 flex-wrap gap-3">

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </div>
 </div>
 </div>
 </div>
 </div>
 </section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">

```

```

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 нвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением.
 </div>
 <p class="text-slate-300 text-sm mb-4">
 БЫХ направлениях по односторонним улицам. Как это выглядит
 у на карте.</p>
 </div>
 </div>
 </div>
 </div>
 </div>
 </section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
 </div>
 <div class="space-y-8">

```

```

 <p class="text-xl font-mono text-white">
тей > 1).</p>
</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb
правильный роутер - и два сегмента офиса б
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400
 <div class="bg-slate-900 p-4 rounded-lg mb-4
 <p class="text-lg text-indigo-300 italic
 <p class="text-xl font-mono text-white"
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg

```



```

 </div>
 <p class="text-slate-300 text-sm mb-4">
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площадь
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-4">
с самых дешевых дорог в стране". Он строит

```

```

 </div>
 </div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 rder border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру пл
 ства шлют гонцов к ближайшему чужому царств
 </div>
 </div>
 </div>
 </div>
 </div>
</div>
</section>`,
 },

```

```

 </summary>
 <div class="p-5 text-sm text-slate-300" style="border: 1px solid #444; border-radius: 10px; padding: 10px; margin-top: 10px;">
 <pre class="bg-slate-950 p-4 rounded" style="margin: 0;">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}
</pre>
 </div>
 </details>
 </div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Z-функция
 <h2 class="text-2xl sm:text-3xl font-bold text-white">22. Z-функция</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">Определение</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Z-функция строки s — это массив z из целых чисел, где $z[i]$ — это длина наибольшего общего префикса строки s и ее сдвигаемого в i символа подстроки $s[i:]$.</p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg">
 <pre>
// Пример для строки "abacab"
// z[0] = 0 (по определению)
// z[1] = 0 (не совпадает с началом)
// z[2] = 1 (a == a)
// z[3] = 0 (b != a)
// z[4] = 2 (ab == ab)
// z[5] = 0 (a != a)
// z[6] = 3 (abc == abc)
// z[7] = 0 (a != a)
// z[8] = 1 (b == b)
// z[9] = 2 (ab == ab)
// z[10] = 0 (a != a)
// z[11] = 1 (c == c)
// z[12] = 2 (ca == ca)
// z[13] = 0 (a != a)
// z[14] = 1 (b == b)
// z[15] = 2 (ab == ab)
// z[16] = 0 (a != a)
// z[17] = 1 (c == c)
// z[18] = 2 (ca == ca)
// z[19] = 0 (a != a)
// z[20] = 1 (b == b)
// z[21] = 2 (ab == ab)
// z[22] = 0 (a != a)
// z[23] = 1 (c == c)
// z[24] = 2 (ca == ca)
// z[25] = 0 (a != a)
// z[26] = 1 (b == b)
// z[27] = 2 (ab == ab)
// z[28] = 0 (a != a)
// z[29] = 1 (c == c)
// z[30] = 2 (ca == ca)
// z[31] = 0 (a != a)
// z[32] = 1 (b == b)
// z[33] = 2 (ab == ab)
// z[34] = 0 (a != a)
// z[35] = 1 (c == c)
// z[36] = 2 (ca == ca)
// z[37] = 0 (a != a)
// z[38] = 1 (b == b)
// z[39] = 2 (ab == ab)
// z[40] = 0 (a != a)
// z[41] = 1 (c == c)
// z[42] = 2 (ca == ca)
// z[43] = 0 (a != a)
// z[44] = 1 (b == b)
// z[45] = 2 (ab == ab)
// z[46] = 0 (a != a)
// z[47] = 1 (c == c)
// z[48] = 2 (ca == ca)
// z[49] = 0 (a != a)
// z[50] = 1 (b == b)
// z[51] = 2 (ab == ab)
// z[52] = 0 (a != a)
// z[53] = 1 (c == c)
// z[54] = 2 (ca == ca)
// z[55] = 0 (a != a)
// z[56] = 1 (b == b)
// z[57] = 2 (ab == ab)
// z[58] = 0 (a != a)
// z[59] = 1 (c == c)
// z[60] = 2 (ca == ca)
// z[61] = 0 (a != a)
// z[62] = 1 (b == b)
// z[63] = 2 (ab == ab)
// z[64] = 0 (a != a)
// z[65] = 1 (c == c)
// z[66] = 2 (ca == ca)
// z[67] = 0 (a != a)
// z[68] = 1 (b == b)
// z[69] = 2 (ab == ab)
// z[70] = 0 (a != a)
// z[71] = 1 (c == c)
// z[72] = 2 (ca == ca)
// z[73] = 0 (a != a)
// z[74] = 1 (b == b)
// z[75] = 2 (ab == ab)
// z[76] = 0 (a != a)
// z[77] = 1 (c == c)
// z[78] = 2 (ca == ca)
// z[79] = 0 (a != a)
// z[80] = 1 (b == b)
// z[81] = 2 (ab == ab)
// z[82] = 0 (a != a)
// z[83] = 1 (c == c)
// z[84] = 2 (ca == ca)
// z[85] = 0 (a != a)
// z[86] = 1 (b == b)
// z[87] = 2 (ab == ab)
// z[88] = 0 (a != a)
// z[89] = 1 (c == c)
// z[90] = 2 (ca == ca)
// z[91] = 0 (a != a)
// z[92] = 1 (b == b)
// z[93] = 2 (ab == ab)
// z[94] = 0 (a != a)
// z[95] = 1 (c == c)
// z[96] = 2 (ca == ca)
// z[97] = 0 (a != a)
// z[98] = 1 (b == b)
// z[99] = 2 (ab == ab)
 </pre>
 </div>
 </div>
 </div>
</section>
`
 }
}

```

```

 <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
 Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 пример сортировка чисел). Класс NP –
 енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ь отличный переводчик на английский (Сведен
 аче B, то B – это NP-Полная (сосредо
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
];

```

```

 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

* <span class="text-green-400 font-
шел-вошёл-вышел).

* <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-2 border-rose-500">
 <div class="absolute -top-3 left-4 bg-slate-800 border-2 border-green-500 shadow-md">
 Путь: Нормальная прогулка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная).
 Талый пришёл в бар (вторая нечётная).
 Домой вернуться не смог, потому что
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2 border-rose-500">
 <div class="absolute -top-3 left-4 bg-slate-900 border-2 border-rose-500 shadow-md bg-slate-950">
 Цикл: Гамильтонов синдром (болезнь)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Гамильтон – это болезнь.
 ng> ровно раз и вернуться домой.
 Ты обходишь все бары (вершины) ровно
 Главное – зайти и выйти, не заходя
 Никто не знает, как решать быстро.


```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-emerald-400">Пример 1: Единственная верёвка

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Пример 1: Единственная верёвка
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-400
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков
 Единственная ниточка. Порвётся — граф p
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
 Мост: Единственная верёвка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спустился в пещеру.
 Из пещеры и нет других выходов — только одна верёвка.
 Если верёвка (ребро v-u) оборвётся, ты застрянешь.
 Если бы из пещеры и был чёрный ход, ты бы вышел.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2 border-emerald-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
 Мост: Единственная верёвка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спустился в пещеру.
 Из пещеры и нет других выходов — только одна верёвка.
 Если верёвка (ребро v-u) оборвётся, ты застрянешь.
 Если бы из пещеры и был чёрный ход, ты бы вышел.
 </p>
 </div>
</div>

```

```

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

```

```

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
}

```

</div>

<p class="text-xs text-slate-400">Занес

</div>

</details>

</div>

</section>`,

},

{

id: "planarity-euler-formula",

Все 5 вершин соединены со всеми. На Теорема Куратовского:  $K_5$  — эталон не Если внутри графа спрятан  $K_5$  — забу

K3,3: 3 дома и 3 колодца

Три дома хотят соединиться с тремя Нарисовать без пересечений невозможно У него  $V=6$   $9 \leq 12$  — проходит, но он в Почему? Потому что у двудольного гра Отсюда более строгое ограничение:  $9 > 8$

Доказательство непланарности  $K_5$  (Скры

Доказательство

Пусть  $K_5$  планарен.  $V = 5$ ,  $E = 10$ .  
 $F = E - V + 2 = 10 - 5 + 2 = 7$   
Каждая грань ограничена минимум  
Сумма длин граней  $= 2E = 20$ .  
Отсюда:  $2E \geq 3F \Rightarrow 20 \geq 21$  — сп  
Значит,  $K_5$  не может быть планар



```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района соединены, а
 её перекроют (удалят вершину) – районы будут
 гхрупкость системы.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
 <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной
 абелем (мостом). Сами свитчи – это точки со
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
 <p class="text-blue-400 font-bold">Особ
 </p>
 Для стартовой вершины обхода правил
 то выше неё прыгать некуда). Поэтому для не
 у него больше 1 независимого ребенка
 </p>
 </div>
</details>

```

```
 (сделаем неориентированным), то любая
 о и есть критическая SCC! Если удалить эту
 . Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];
```

---

ФАЙЛ 19/83: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

---

```
import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph-bridges
const toRemove = ["graph-planar-colors", "graph-bridges"];

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
```

```
* Глоссарий «википедийных» подсказок.
*
* Любое слово из `labels`, встреченное в тексте главы,
* в подчёркнутый термин: при наведении (или тапе на мо
* с объяснением на пальцах и мини-визуализацией.
*/
export interface GlossaryEntry {
 id: string;
 /** Варианты написания в тексте (регистр не важен). */
 labels: string[];
 title: string;
 /** Объяснение «на пальцах» – 1–2 фразы. */
 short: string;
 /** Строгая формулировка / что важно не забыть. */
 formal?: string;
 complexity?: string;
 /** Какая мини-анимация показывается в карточке. */
 demo?: DemoKind;
 /** id полного симулятора (см. vizRegistry) – кнопка
 simulator?: string;
}

export const GLOSSARY: GlossaryEntry[] = [
 {
 id: "bfs",
 labels: ["BFS", "обход в ширину", "поиск в ширину",
 title: "BFS – обход в ширину",
 short:
 "Волна от источника: сначала все соседи, потом со
 formal:
 "Очередь (FIFO). Кладём старт, достаём вершину –
 у рёбер.",
 complexity: "O(V + E)",
 demo: "bfs",
 simulator: "queue-bfs",
 },
 {
 id: "dfs",
```

```

 title: "Стек (LIFO)",
 short: "Стопка тарелок: кладёшь сверху и берёшь сверху",
 formal: "push / pop / top за $O(1)$. На стеке живёт рекурсия",
 demo: "stack",
 simulator: "stack-dfs",
},
{
 id: "queue",
 labels: ["очередь", "очереди", "очередью", "FIFO"],
 title: "Очередь (FIFO)",
 short: "Очередь в столовой: кто первым встал, тот первым уходит",
 formal: "push_back / pop_front за $O(1)$. Основа BFS",
 demo: "queue",
 simulator: "queue-bfs",
},
{
 id: "dsu",
 labels: ["DSU", "СНМ", "система непересекающихся множеств"],
 title: "DSU — система непересекающихся множеств",
 short:
 "«Кто твой староста?» Каждый элемент знает своего старосту",
 formal:
 "find со сжатием путей + union по рангу/размеру. Косвенно доказывает существование",
 complexity: "почти $O(1)$ — обратная функция Аккермана",
 demo: "dsu",
 simulator: "mst-kruskal",
},
{
 id: "trie",
 labels: ["бор", "бора", "боре", "бору", "бором", "борт"],
 title: "Бор (trie, префиксное дерево)",
 short:
 "Дерево, где путь от корня по буквам и есть слово",
 formal: "Из корня по символам; в вершине хранится фактическое слово",
 complexity: "поиск слова — $O(|\text{слова}|)$ ",
 demo: "trie",
 simulator: "aho-corasick",
},

```

```

 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "пре
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от н
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] =
 complexity: "препроцессинг O(n), запрос O(1)",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "sparse-table",
 labels: [
 "разреженная таблица", "разреженная таблица", "sp
 "разреженная", "двоичные подъёмы", "двоичный подъ
],

```

```

 formal: "Для не-корня: v — точка сочленения Э ребро",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная
 formal: " $V-1$ ребро, никаких циклов. Краскал (жадно и
 но).",
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизи
 "амортизированное", "амортизированного", "амортизи
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер
 formal: "Метод потенциалов: дорогая операция оплачи
 ассив.",
 demo: "amortized",
},
{
 id: "np",
 labels: [
 "NP-полная", "NP-полнота", "NP-трудная", "NP-полн
 "класс P", "полином", "полиномиальное",
],
 title: "Класс NP и NP-полнота",

```

```

 от кусок.",
 formal:
 "π[i] — длина наибольшего собственного префикса s
 возвращая указатель по тексту.",
 complexity: "O(n + m)",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начи
 аждой букве.",
 formal:
 "z[i] = длина наибольшего общего префикса s и s[i]
 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "дин
 "мемоизация", "мемоизацию", "подзадач", "подзадач
],
 title: "Динамическое программирование",
 short:
 "Решаем задачу через ответы на её меньшие копии и
 formal:
 "Нужны: оптимальная подструктура + перекрывающиеся
 ция).",
 demo: "dp",
 simulator: "dynamic-programming",
},
{
 id: "shortest-path",

```

```

 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрываясь",
 formal:
 "Связный граф: эйлеров цикл все степени чётные;
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного",
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе бумаги",
 formal:
 " $V - E + F = 2$ для связного планарного графа. Курсив",
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "всестороннее",
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягивают»",
 formal:
 "Три случая: zig (родитель – корень), zig-zig (узлы в одной ветви),
 zig-zag (узлы в разных ветвях).",
 demo: "amortized",
 simulator: "splay-tree",
},
{
 id: "adjacency",
 labels: ["матрица смежности", "список смежности", "матрица смежности",
 title: "Способы хранить граф",
 short:

```



```

 short:
 "Внешний цикл — это не «шаг», а «какие пересадки

 шить пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Поряд

 ами.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
},
{
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на кажд
 formal:
 "Множество посещённых + куча рёбер на границе. До
 усок.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-prim",
},
{
 id: "boruvka",
 labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
 title: "Алгоритм Борувки",
 short:
 "Все компоненты одновременно шлют «гонца» к ближай
 formal:
 "Фаза: для каждой компоненты ищем минимальное исх
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-boruvka",
},
{
 id: "kruskal",
 labels: ["Краскал", "Краскала", "Краскалу", "Kruska

```

```

 },
 {
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "с"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам ну

 рядковым номером.",
 formal:
 "sort + unique + binary search: значение → индекс",
 complexity: "O(n log n)",
 demo: "coord-compress",
 },
 {
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова де"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу

 ча. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge

 я через них. Ожидаемая высота O(log n).",
 complexity: "ожидаемо O(log n)",
 demo: "heap",
 },
 {
 id: "bst",
 labels: ["бинарное дерево поиска", "дерево поиска", "д"],
 title: "Бинарное дерево поиска",
 short:
 "Слева всё меньше корня, справа — всё больше. Пои",
 formal:
 "Без балансировки дерево может вырождаться в списо",
 complexity: "O(высоты): от O(log n) до O(n)",
 demo: "segment-tree",
 },
],

```

```

 },
 {
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "с"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где (Z-функция) и где встречаются слова словаря (бор),
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция.
 demo: "prefix-function",
 simulator: "string-kmp",
 },
 {
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворо"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встаёт на место своего
 дившемуся месту. Порядок ключей при этом не ломается.
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x$.
 Левый поворот вокруг p: $x = p.right; p.right = x$.
 g и Zig-Zag.",
 complexity: "O(1)",
 demo: "zig",
 simulator: "splay-rotations",
 },
 {
 id: "zig",
 labels: ["Zig", "Zag", "зиг"],
 title: "Zig — одиночный поворот",
 short:
 "Самый простой случай: родитель x уже сидит в корневом узле.
 канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию",
 demo: "zig",
 simulator: "splay-rotations",
 },
],

```

```
import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Обзор: Кратчайшие пути и Графы",
 type: "html",
 category: "Графы",
 content: `<section id="intro-content" class="mb-12">
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Обзор
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <p class="text-slate-300 text-sm mb-4">Пред
 оезда). Алгоритмы поиска кратчайшего пути п
 <div class="mt-8 mb-4">
 <div id="slot-mnemonic-cards"></div>
 </div>
 </div>
</div>
</section>`
 },
 {
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
 category: "Графы",
 content: `<section id="dijkstra-content" class="mb-12">
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Дейкс
</div>

<div class="space-y-8">
```

```

g V). Гарантирует корректность только для н
 </div>
 </details>

 <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class=
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Форд-Б
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-rose-400">Аналогия 1: Параноик

 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-rose-300 italic">Параноик
 <p class="text-xl font-mono text-white">Параноик
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-6">Параноик
 берет ВСЕ возможные дороги V-1 раз подряд.

```



```

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50"
 <summary class="p-4 font-bold text-slate-400 outline
 pen:border-slate-700/50">
 Внутренняя структура автомата и код BFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Каждая вершина имеет таблицу переходов <code>go</code> по
 ссылке <code>term_link</code> (ближайшая терминальная вершина)
 <div class="bg-black/50 p-4 rounded-lg font-mono">
struct Node {
 map<char, int> go; // Предвычисленные переходы
 int link = 0; // Суффиксная ссылка: куда перейти по
 int term_link = 0; // Терминальная ссылка: мат. слово
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные link
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка – это переход по
 nodes[child].link = nodes[nodes[v].link].go[ch];

 // Если суффиксная ссылка сама является словом
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {

```

ФАЙЛ 22/83: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 106 | РАЗМ:

```
export interface QuizQuestion {
 question: string;
 options: string[];
 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечётными степенями",
 options: [
 "Есть эйлеров цикл (вернись домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)",
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных больше, то эйлеров цикла не будет. Если нечётных 0, то эйлеров цикл будет."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полная задача?",
 options: [
 "Потому что Гамильтон — болезнь, и лечить её тяжело",
 "Потому что для эйлерова цикла есть простой критерий",
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий нечётности степеней. Для гамильтонова цикла нет простого критерия. Это NP-полная задача, которую не известно, можно ли решить за полиномиальное время."
 }
]
```



```

 "O(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
 question: "Планарный граф имеет V=6, E=12. Возможн
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ – на грани пр
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с V=
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не
 },
 {
 question: "Почему K5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ – нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин – несчастливое число"
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, $3V - 6 = 9$. $10 > 9$
 }
}

```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Корпоративная пре

```
</div>
```

```
<p class="text-slate-300 text-sm mb
ы рассылать 10 000 писем, он пишет одно пис
счетах сотрудников только тогда, когда сотр
женное обновление).</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Матрешки-коробки

```
</div>
```

```
<p class="text-slate-300 text-sm mb
ше, и так далее. Если нам нужно покрасить п
Внутри всё синее!" на большую коробку. Опус
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

Строгое доказательство / Код (Скр

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounde
```

```
function push(node) {
 if (lazy[node] !== 0) {
 lazy[2 * node] += lazy[node];
 tree[2 * node] += lazy[node];
 lazy[2 * node + 1] += lazy[node];
 tree[2 * node + 1] += lazy[node];
 lazy[node] = 0;
```





```

<h4 class="text-lg font-bold text-indigo">
 <div class="space-y-4 text-sm text-slate">
 <div>
 <strong class="text-white block">
 корень?
 <p>Оно поднимет узел, на котором
 ска. За счет сдвига этого листа в корень, д
 /p>
 </div>

 <div>
 <strong class="text-white block">
 <p>Если агент постоянно переключ
 находящимися на разных концах дерева, его
 ащая дерево в вытянутый бамбук для другого.
 . Постоянное свинг-переключение превратит п
 </div>

 <div>
 <strong class="text-white block">
 <p>Хотя один поиск может стоить
 емах обладают прекрасным свойством: они не
 по пути. "Палочное" дерево прессуется в сб
 осов.</p>
 </div>
 </div>
</div>
</section>`
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 }

```

```
export const tickets14to20: Chapter[] = [
 {
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 14
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Графы
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Дейкстра
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Дейкстра — алгоритм поиска кратчайшего пути в графе.
 <p class="text-xl font-mono text-white">Графы — это набор вершин, соединенных ребрами.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в пути.
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Он не может пересматривать уже "зафиксированные"
 </div>
 </div>
 </div>
 </div>
`
 }
]
```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
опали во временную петлю.</p>
 </div>
 </div>
 <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-slate-900
border-emerald-500 shadow-md">
 Суть фокуса (По шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
 share <code class="bg-slate-900 text-pink-400">
 шу себе проходить через вершину k?></i>

```

```

 </div>
 <p class="text-slate-300 text-sm mb-6">
 Дейкстра (Билет 14) быстрая, но работает
 ает очень медленно. Мы хотим сделать Джонсона
 е пути остались теми же.
 </p>
 <ol class="text-sm text-slate-300 space-y-4">
 Добавляем фиктивную вершину
 Запускаем от неё медленный поиск
 «потенциалы» <code class="text-pink-400">
 Меняем старые веса: новое р
 Магия! Все веса тепер
 ренные потенциалы сокращаются).
 Теперь смело запускаем быстрый поиск

 </div>
</div>
<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</div>
</section>`
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">

```



```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">
 <div class="absolute -top-3 left-4 right-4 bottom-4" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
</div>
</div>
</div>
</section>`
},
{
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">
 <div class="absolute -top-3 left-4 right-4 bottom-4" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">
 Аналогия: Племена
 </div>
 </div>
 </div>
 </div>
</section>`
}
]
}

```

```
сом, оканчивающимся в позиции i.</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb
 Мы идём по строке слева направо. К
 строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слова
x</code>. Тупой алгоритм начнет искать заголо
</code>, а это начало нашего слова! Не надо
 </p>
 </div>
```

```
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb
 Для LLM, которая стримит токены
 каждом шаге. Если мы частично совпали с зап
 акого места этого слова мы можем продолжить
 </p>
 </div>
 </div>
```

```
<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Пример вычисления (Скрыто)
 </summary>
```

```

{
 id: "string-z-func",
 title: "22. Z-функция — Сдвиг и наложение",
 type: "html",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 са 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Тут мы берём всю строку и «приклеива
 я начну читать строку отсюда, насколько длинна
 Это самый быстрый способ найти искомый символ
 >, считаешь Z-функцию, и там, где <code>Z[i]</code>
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4

```

```

for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}

```

```

 </div>

```

```

 </details>

```

```

 </div>

```

```

</div>

```

```

</section>`

```

```

},

```

```

{

```

```

 id: "aho-corasick",

```

```

 title: "23. Алгоритм Ахо-Корасик",

```

```

 type: "html",

```

```

 category: "Строки",

```

```

 content: `

```

```

<section id="aho-corasick" class="mb-12 scroll-mt-10">

```

```

 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```



```

```

 <h2 class="text-2xl sm:text-3xl font-bold text-white">

```

```

 </div>

```

```

 <div class="space-y-8">

```

```

 <div class="bg-slate-700/50 p-6 rounded-xl border-l">

```

```

 <h3 class="text-xl font-bold text-blue-400 mb-4">

```

```

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text">

```

```

 <p class="text-sm text-blue-300 italic mb-2">Осн

```

```

 <p class="text-lg font-mono text-white">Бор (Тр

```

```

 </div>

```

```

 <p class="text-slate-300 text-sm">Позволяет найти

```

```

 го один проход.</p>

```

```

 </div>

```

```

 <!-- Аналогии -->

```

```

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">

```

```

 <div class="bg-slate-800 p-6 rounded-lg border bo

```

```

 return t[v].go[c];
 }</pre>
 </div>
</details>
</div>
</section>`
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-purple-400
 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-purple-300 italic
 <p class="text-xl font-mono text-white"
Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс NP –
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->

```

```

 { /* База представлений */ }
 <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-slate-300">База представлений
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">Вася Петя знает ли Вася Петю?
 <p class="text-xl font-mono text-white">(V + E).</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb-4">Матрица глупо, но проверка "знает ли Вася Петю" матрица
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb-4">Списки т. Оптимально для почти всех задач.</p>
 </div>
 </div>
 </div>
 </div>

```

```

 { /* DFS vs BFS */ }
 <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-indigo-500 shadow-md">
 <h3 class="text-xl font-bold text-indigo-400">DFS vs BFS
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = Ж

```

## Код обходов C++ (Основа)

</summary>

<div class="grid grid-cols-1 md:grid-co

<pre class="bg-slate-950 p-3 rounded

// DFS (Рекурсия)

vector<bool> vis;

vector<vector<int>> adj;

void dfs(int v) {

vis[v] = true;

for (int u : adj[v]) {

if (!vis[u]) dfs(u);

}

</pre>

<pre class="bg-slate-950 p-3 rounded

// BFS (Очередь)

queue<int> q;

q.push(start\_node);

vis[start\_node] = true;

while(!q.empty()) {

int v = q.front(); q.pop();

for (int u : adj[v]) {

if (!vis[u]) {

vis[u] = true;

q.push(u);

}

}

</pre>

</div>

</details>

</div>

</div>

</section>`,

},

{

id: "graph-planar-colors",

title: "7. Графы. Планарные. Покраска",

```

 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Графы</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Графы</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Графы – это абстрактные модели, состоящие из вершин (узлов) и ребер (связей).</p>
 <p class="text-xl font-mono text-white">Графы используются для моделирования структур, таких как социальные сети, маршруты, базы данных и т.д.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm mb-4">Представьте, что вершины – это острова, а ребра – это мосты, соединяющие их.</p>
 <p class="text-slate-300 text-sm mb-4">Задача: найти кратчайший путь от одного острова до другого.</p>
 <p class="text-slate-300 text-sm mb-4">Карта – остались ли серые (неисследованные) острова?</p>
 <p class="text-slate-300 text-sm mb-4">Путь через океан – столько и компонент.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```



```

<div class="bg-slate-950 p-4 rounded
 <pre class="text-xs font-mono t
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
 visited[v] = true;
 for (int to : adj[v]) {
 if (!visited[to]) {
 dfs(to);
 }
 }
 ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

void topological_sort(int n) {
 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
 reverse(ans.begin(), ans.end()); // Разворачиваем с
}</pre>
 </div>
 </div>
 </details>
 </div>
 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Связь с Билетом 12
 </div>
 <p class="text-slate-300 text-sm mb"
 SCC (Билет 10) склеивает
 </br>

 Точки сочленения (Билет 12)
 ость монолита. Вырвешь точку сочле
 </p>
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Душный мод: Код Алгоритм Косарайю
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <p class="text-emerald-400 font-bol"
 <ol class="list-decimal list-inside"
 Запускаем DFS на исходном графе

 Разворачиваем этот список (только вершины)
 Переворачиваем все ребра в обратном направлении
 Идем по <code>order</code>:
 !

 </div>
 </details>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",

```

```

content: `
<section id="graph-articulation" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3"
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-rose-300 italic m
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 <div class="p-3 bg-slate-950 rounde
 <p class="text-slate-300 text-sm">
 Важно: Это работает то
еются к точкам сочленения. То есть, <strong
ег - это тупик со степенью 1).
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
 order-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">

```



-----  
РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ  
=====

-----  
ФАЙЛ 27/83: src/components/ArticulationPointsViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1  
-----

```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-react";
```

```
interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface Edge {
 u: number;
 v: number;
}
```

```
const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];
```

```
const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
```

```
const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
};
```

recordStep("Начало DFS (Тарьян) для поиска точек со

```
const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`);

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], currentLow[to]);
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${v}]`,
 v,
);
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], currentLow[to]);

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}]`,
 v,
);
 }
 }
};
```

```

 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
 };

 useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
 }, [isPlaying, currentStepIndex, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 text-white"
 >

```

```

 strokeWidth={isBridge ? "4" : "2"}
 strokeDasharray={isBridge ? "8,4" : "1"}
 />
);
 }}}

```

```

{/* Nodes */}
{nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id;
 const isVisited = currentStep.visited.has(node.id);
 const isAp = currentStep.ap.has(node.id);

```

```

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
 const r = isAp ? 24 : 20;

```

```

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-100"

```



```

 : "-" }
 </text>
 </>
 })
</g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 fon
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Точка сочленения
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Посещена
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Текущая
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-8 h-1 bg-rose-500 bor

```

```
};
```

ФАЙЛ 28/83: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';
```

```
interface Node { id: string; x: number; y: number; name: string; }
```

```
interface Edge { u: string; v: string; w: number; }
```

```
interface Step {
```

```
 iteration: number;
```

```
 checkingEdge: { u: string; v: string; w: number } | null;
```

```
 distances: { [key: string]: number };
```

```
 previous: { [key: string]: string | null };
```

```
 log: string;
```

```
 hasNegativeCycle?: boolean;
```

```
 activeCodeLine: number;
```

```
}
```

```
export default function BellmanFordViz() {
```

```
 const [hasCycle, setHasCycle] = useState(false);
```

```
 const [steps, setSteps] = useState<Step[]>([]);
```

```
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
```

```
 const [isPlaying, setIsPlaying] = useState(false);
```

```
 const nodes: Node[] = [
```

```
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
```

```
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
```

```
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
```

```
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
```

```
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
```

```
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
```

```
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
```

```
];
```

```

 { line: 10, text: ' return dist' },
];

useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расстояние до А: 999',
 activeCodeLine: 2,
 });

 const vCount = nodes.length;
 for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}. ${
 updated ? `Релаксация успешна! dist[${edge.v}] = ${dist[edge.v]} `
 : `Релаксация не удалась. dist[${edge.v}] = ${dist[edge.v]} `
 }`,
 activeCodeLine: updated ? 6 : 5,
 });
 }
 }
}
```



```

 const isChecking = currentStep.checkingEdge;
 v;
 const isNeg = edge.w < 0;
 const isCyclePart = hasCycle && ((edge.u === 'D' ||
 && edge.v === 'D'));

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b';
 else if (currentStep.hasNegativeCycle &&

 return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : strokeColor} strokeDash={isChecking ? '4,4' : 'none'} />
 <circle cx={({uNode.x + vNode.x} / 2) cy={({uNode.y + vNode.y} / 2)} r={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : strokeColor} />
 <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)} fill={strokeColor} fontSize="11" fontWeight="bold" textAnchor="middle">{edge.w}</text>
 </g>
);
 });
}}}

{nodes.map((node) => {
 const dist = currentStep.distances[node.id];
 const isCheckingNode = currentStep.checkingNode === node.id);

 return (
 <g key={node.id} className="transition-...>
 <circle cx={node.x} cy={node.y} r={isCheckingNode ? '#7dd3fc' : '#64748b'} strokeWidth={2} />
 <text x={node.x} y={node.y + 5} fill={strokeColor} />
 <text x={node.x} y={node.y - 28} fill={strokeColor} fontW
 ight="bold" textAnchor="middle" className="label">{node.name}</text>
 <text x={node.x} y={node.y + 35} fill={strokeColor} fontW
 split(' ')[0]</text>
 </g>
);
})}

```



```
const steps: Step[] = [];
const queue: number[] = [];
const visited = new Set<number>();

// Init
steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0)."});

queue.push(0);
visited.add(0);
steps.push({
 activeLine: 2,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Добавляем стартовую вершину 0 в очередь и по"});

while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}`});

 const curr = queue.shift()!;
 steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
```

```

 steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
 });

 return steps;
 };

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const step = STEPS[stepIdx];

 const handleNext = useCallback(() => {
 setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
 }, []);

 // @ts-expect-error зарезервировано под кнопку «назад»
 const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
 }, []);

 useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
 }, [isPlaying, stepIdx, handleNext]);

```



```

<div className="bg-slate-800 border border-slate-600 border-radius-50px]">
 <svg className="w-full h-full min-h-[350px]" >
 {/* Рёбра */}
 {EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from)
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n1)
 return (
 <line
 key={idx}
 x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
 className={`transition-all duration-500 ${
 isVisited ? 'stroke-slate-600' : 'stroke-slate-400'
 }`}
 />
)
 })}

 {/* Вершины */}
 {NODES.map(n => {
 const isCurrent = step.currentNode === n.id
 const isInQueue = step.queue.includes(n.id)
 const isVisited = step.visited.includes(n.id)

 let fill = '#1e293b'; // slate-800
 let stroke = '#475569'; // slate-600
 let scale = 1;

 void scale;

 if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#1e293b'; // slate-800
 stroke = '#475569'; // slate-600
 scale = 1;
 }

 return (
 <circle
 key={n.id}
 cx={n.x} cy={n.y}
 fill={fill}
 stroke={stroke}
 r={n.radius * scale}
 className="transition-all duration-500"
 />
)
 })}


```

```

 Шаг {step}
 </div>
 <div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); vi' },
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmp' },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in grap' },
 { line: 7, code: ' if neighbor not in' },
 { line: 8, code: ' visited.add(ne' },
 { line: 9, code: ' queue.push(nei'
]}.map((cl) => {
 const isActive = step.activeLine === cl
 return (
 <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5` : 'te
 <span className="opacity-40 w-6 inl
 {c
 </div>
 });
])}
</div>
</div>

<div className="bg-slate-800/80 border border
 <div className="text-sm font-bold text-eme
 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500
 </div>
 <div className="flex items-center gap-2 mi
 {step.queue.length === 0 ? (
 <span className="text-slate-500 text-x
) : (
 step.queue.map((qId, i) => (
 <span key={`${qId}-${i}`} className=
 enter justify-center rounded-md shadow-lg sl

```

```
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко Всем Флойд',
 caption: 'Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
 };

export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;

 return (
 <div>
 <div className={`rounded-2xl overflow-hidden border-2 border-${info.borderColor} shadow-2xl`} >

 </div>
 <p className={`text-center text-xs mt-2 font-bold text-${info.captionColor}`} >{info.caption}</p>
 </div>
);
}
```

---

ФАЙЛ 31/83: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";

interface Props {
 prev?: Chapter;
```

```

 </div>
);
}

return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-white border border-gray-200 transition-all"
 >
 <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
 <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
 </div>
 <div className="text-sm font-bold text-slate-700">
 {prev.title}
 </div>
 </button>
) : (
 <div className="hidden sm:block" />
)}

 {next && (
 <button
 onClick={() => onGo(next.id)}
 className="group text-right p-4 rounded-xl bg-white border border-gray-200 transition-all sm:col-start-2"
 >
 <div className="flex items-center justify-end text-indigo-400 mb-1">
 Следующая тема <ChevronRight className="w-3.5 h-3.5" />
 </div>
 <div className="text-sm font-bold text-slate-700">
 {next.title}
 </div>
 </button>
)}
 </nav>
);
};

```

```

 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
}, []));

const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(
}, [cancelHide]);

useEffect(() => {
 setAnchor(null);
 setSaving("idle");
}, [chapter.id]);

const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
}, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect()
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;

```

```

 <Loader2 className="w-3.5 h-3.5 animate-s
) : saving === "done" ? {
 <Check className="w-3.5 h-3.5 text-emeral
) : {
 <Download className="w-3.5 h-3.5" />
 }}
 {saving === "done" ? "Файл сохранён" : "Ска
</button>
<button
 onClick={() => window.print()}
 title="Распечатать или сохранить в PDF сред
 className="flex items-center gap-1.5 text-x
 300 hover:text-white hover:border-indigo-50
>
 <Printer className="w-3.5 h-3.5" /> Печать
</button>

</div>

<div
 ref={contentRef}
 className="prose prose-invert max-w-none"
 onMouseOver={handlePointerOver}
 onMouseOut={handlePointerOut}
 onClick={handleClick}
 onFocus={handleFocus}
 dangerouslySetInnerHTML={{ __html: chapter.co
/>

<p className="mt-6 flex items-start gap-2 text-
ed-lg px-3 py-2">
 <MousePointerClick className="w-4 h-4 shrink-

 Подчёркнутые термины – интерактивные: навед
 объяснение на пальцах, мини-анимацию и кноп

</p>

```

ФАЙЛ 33/83: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useRef } from "react";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "lucide-react";
```

```
interface CodeLine {
 line: string;
 highlight: string;
}
```

```
const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },
 { line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
 { line: " } else {", highlight: "normal" },
 { line: " dfs(to, v);", highlight: "normal" },
 { line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " if (low[to] > tin[v]) {", highlight: "normal" },
 { line: " // ЭТО МОСТ!", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: "}", highlight: "normal" },
];
```

```
interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
}
```

```

 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridge: null,
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3 : ",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,6,2],
 activeEdge: null, backEdge: [2,0],
 log: "Видим соседа A (уже посещён)! Обратное ребро ",
 activeCodeLine: [7,8,9]
 },
 {
 currentNode: 3, visitedIndices: [0,1,6,2,3],

```



```

 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) — мост!
 activeCodeLine: [13,14,15,16,17]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) — мост!
 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D).
 activeCodeLine: [11,13,16,17,18]
 },
 {
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм завершён!
 activeCodeLine: []
 },
];

```

```

export function DfsBridgesSimulator() {
 const [dfsIdx, setDfsIdx] = useState(0);
 const [dfsAuto, setDfsAuto] = useState(false);
 const dfsTimer = useRef<NodeJS.Timeout | null>(null);

 useEffect(() => {

```

```
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-5 gap-6">
 { /* Graph visualization */ }
 <div className="md:col-span-3 bg-slate-950 rounded-lg p-6">
 <div className="absolute inset-0 bg-[radial-gradient(circle_1px_1px_1px)]">
 <svg className="w-full h-[280px] z-10 relative">
 {GRAPH_EDGES.map(e => {
 const u = GRAPH_NODES.find(n => n.id === e.u);
 const v = GRAPH_NODES.find(n => n.id === e.v);
 const bridge = step.bridges.includes(e.key);
 const activeTree = step.activeEdge[0] === e.u || step.activeEdge[0] === e.v;
 const activeBack = step.backEdge[0] === e.u || step.backEdge[0] === e.v;
 let sc = "#475569", sw = 2, dash = "none";
 if (bridge) { sc = "#eab308"; sw = 4; }
 else if (activeTree) { sc = "#10b981"; sw = 2; }
 else if (activeBack) { sc = "#f43f5e"; sw = 2; }
 return <line key={e.key} x1={u.x} y1={u.y} x2={v.x} y2={v.y} stroke={sc} strokeWidth={sw} strokeDash={dash} />;
 })}
 {step.currentNode !== null && step.currentNode.id === n.id ? (
 <circle cx={n.x} cy={n.y} r={18} fill="#fff" />
) : null}
 </svg>
 </div>
 </div>
 <div className="md:col-span-2">
 { /* ... */ }
 </div>

```

```

 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includes(cl)
 return <div key={i}
 className={`px-1 transition-all duration-200
 ${hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-600"}
 `}>
 {cl.line} |>
 {cl.line} || ' '
 </div>
 })}
 </pre>
 </div>
 </div>

 {/* Stack */}
 <div className="bg-slate-900/50 rounded-xl border border-slate-800 p-4">
 <div className="text-[10px] font-bold text-slate-400">
 Стек DFS
 {step.stack.length}
 </div>
 <div className="flex flex-col-reverse gap-1">
 {step.stack.length === 0
 ? <div className="text-[10px] text-slate-400">Стек пуст</div>
 : step.stack.map((id, i) => {
 <div key={id} className={`px-2 py-0.5 transition-colors
 ${i === step.stack.length - 1
 ? "bg-emerald-600/20 text-emerald-400"
 : "bg-slate-900 text-slate-400"}
 `}>Вершина {GRAPH_NODES.find(n=>n.id === id).name}</div>
 {step.parent[id]}
 </div>
)}
 </div>
 </div>
</div>

{/* Log */}
<div className="bg-indigo-950/10 border border-indigo-800 p-4">

```

```

 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];

```

```

const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];

```

```

const adjList: { [key: string]: { v: string; w: number }[] } = {};
nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
});

```

```

const codeLines = [
 { line: 1, text: 'function dijkstra(graph, start):' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' pq = PriorityQueue(); pq.push(start, 0)' },
 { line: 4, text: ' while not pq.empty():' },
 { line: 5, text: ' d, u = pq.pop() // Извлекаем элемент с минимальным расстоянием' },
 { line: 6, text: ' if d > dist[u]: continue' },
 { line: 7, text: ' for v, weight in graph.neighbors(u):' },
 { line: 8, text: ' if dist[u] + weight < dist[v]:' },
 { line: 9, text: ' dist[v] = dist[u] + weight' },
 { line: 10, text: ' pq.push(v, dist[v])' },
];

```

```

 });

 const neighbors = edges.filter(e => e.u === u);
 for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Смотрим соседа ${v}. Текущее расстояние
 ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

 if (dist[u] + edge.w < dist[v]) {
 dist[v] = dist[u] + edge.w;
 prev[v] = u;
 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Улучшение (Релаксация)! Новое кратчайшее
 расстояние до ${v} равно ${dist[v]}.`,
 activeCodeLine: 9,
 });
 }
 }
}

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Дейкстра всё доставил! Все возможные кратчайшие
 расстояния найдены!`,
 activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []);

```



```

 </text>
 </g>
);
 }}}

 { /* Вершины */
 {nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id
 const isVisited = currentStep.visited[node.id]
 const dist = currentStep.distances[node.id]

 return (
 <g key={node.id} className="transition-all duration-300">
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVisited ? '#a5b4fc' : '#f8d7da'}
 stroke={isCurrent ? '#a5b4fc' : isVisited ? '#a5b4fc' : '#f8d7da'}
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-300"
 />
 <text x={node.x} y={node.y + 5} fill="black">
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill="white">
 {node.name}
 </text>
 <text x={node.x} y={node.y - 28} fill="white">
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill="white">
 {node.name.split(' ')[0]}
 </text>
 </g>
);
 })}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">

```

```

 }}}
 </div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div>
 <h4 className="text-lg font-bold text-rose-950">
 Таблица расстояний
 </h4>
 <div className="overflow-x-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-rose-950">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 <th className="py-2 px-3">Из (пред. вершина)</th>
 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n.id]
 const prev = currentStep.previous[n.id]
 const vis = currentStep.visited[n.id]
 const isCurr = currentStep.currentNode === n.id

 return (
 <tr key={n.id} className={`border-b border-rose-950 ${
 isCurr ? 'bg-indigo-950/60 font-medium' : ''
 }`}>
 <td className="py-2.5 px-3 flex items-center">

 {n.name}
 </td>
 <td className="py-2.5 px-3 font-medium">
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text-sm">
 {prev ? nodes.find((p) => p.id === prev).name : ''}
 </td>
 </tr>
)
 })}
 </tbody>
 </table>
 </div>
 </div>
 </div>

```



ФАЙЛ 35/83: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
```

```
interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'return';
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}
```

```
const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];
```

```
export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const [isPlaying, setIsPlaying] = useState<boolean>(false);
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' | 'graph'>('table');
```

```
 useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};
```

```

 action: 'call',
 desc: `Вычисляем рекурсивно: fibMemo(${n - 1})`,
 codeLine: 4,
 memoState: { ...memoMap }
 });

 const res1 = simulateFib(n - 1);
 const res2 = simulateFib(n - 2);
 memoMap[n] = res1 + res2;

 generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: memo[${n}] = ${res1} + ${res2}`,
 codeLine: 5,
 memoState: { ...memoMap }
 });

 return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: `Расчет завершен! fibMemo(${targetN}) = ${finalRes}`,
 codeLine: 6,
 memoState: { ...memoMap }
});

setSteps(generatedSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [targetN]);
```

```

 className="bg-transparent text-white font
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
</div>

<div className="flex items-center bg-slate-950
 <button
 onClick={() => { setIsPlaying(false); set
 className="p-2 text-slate-400 hover:text-
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-
 isPlaying
 ? 'bg-amber-500 text-slate-950 hover:
 : 'bg-emerald-600 text-white hover:
 }`
 >
 {isPlaying ? <Pause className="w-4 h-4" />
 {isPlaying ? 'Пауза' : 'Пуск'}}
 </button>
 <button
 onClick={() => { setIsPlaying(false); set

```

```

 </div>

 {currentStep && (
 <div className="mt-6 pt-4 border-t border-slate-400">
 <div className="text-sm font-medium text-slate-400">

 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-400 p-2">
 Текущий N: <strong className="text-emerald-400">{currentStep.n}
 </div>
 </div>
)}
 </div>

 {/* Tabs */}
 <div className="flex items-center gap-2 border-b border-slate-400">
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3 border rounded-md
 ${activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg-white'
 : 'border-transparent text-slate-400 hover:text-slate-300'}
 `}
 >
 <Eye className="w-4 h-4" />
 Шаг-бай-шаг состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3 border rounded-md
 ${activeTab === 'code'
 ? 'border-emerald-500 text-emerald-400 bg-white'
 : 'border-transparent text-slate-400 hover:text-slate-300'}
 `}
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
 </div>

```

.....

```

const edge = C1_EDGES.find(e => (e.u === last && e.v === vId))
if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери другую вершину!")
 return
}
const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`
if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощает повторений!")
 setC1Done(true); return
}
const nextEdges = [...c1VisitedEdges, key];
const nextPath = [...c1Path, vId];
setC1VisitedEdges(nextEdges);
setC1Path(nextPath);

if (nextEdges.length === C1_EDGES.length) {
 const startDegree = C1_EDGES.filter(e => e.u === last).length
 const endDegree = C1_EDGES.filter(e => e.v === vId).length
 const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0
 if (isCycle && nextPath[0] === vId) {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, факт!")
 } else {
 setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! Сделай шаг назад!`)
 }
 setC1Done(true);
} else {
 setC1Msg(`Пройдено рёбер: ${nextEdges.length} / ${C1_EDGES.length}`)
}
};

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">Решение задачи
 <button onClick={resetC1} className="flex items-center justify-center
 old border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> Сброс
 </button>
 </div>
 </div>
)

```

```
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
 : c1Done
 ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
 : "bg-slate-900/50 border-slate-700 text-slate-50"
 }` }>
 {c1Msg}
 </div>
</div>
);
}
```

---

ФАЙЛ 37/83: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

interface Question {
 id: number;
 question: string;
 options: string[];
 correctAnswer: number;
 explanation: string;
}

const quizQuestions: Question[] = [
 {
 id: 1,
 question: 'Что мы делаем в алгоритме Краскала, если ребро соединяет два вершины, которые уже принадлежат одному компоненту связности (т.е. если ребро, красный) цвет?',
 options: [
 'Мы радуемся и добавляем его в минимальное остовное дерево',
 'Мы перекрашиваем их в синий цвет и делим граф по компонентам связности',
 'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
 'Мы вызываем алгоритм Дейкстры для поиска нового пути'
],
 correctAnswer: 2,
 explanation: 'В алгоритме Краскала ребра, соединяющие вершины в уже существующем компоненте связности, отбрасываются, так как они создают цикл. Правильный ответ: Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл.'
 },
 {
 id: 2,
 question: 'Какой из следующих вариантов НЕ является корректным шагом в алгоритме Краскала?',
 options: [
 'Сортировка всех ребер по весу',
 'Выбор ребра с наименьшим весом, которое не создает цикла',
 'Удаление ребра, которое создает цикл',
 'Добавление ребра, которое создает цикл'
],
 correctAnswer: 3,
 explanation: 'Шаг 4: Если ребро создает цикл, его отбрасывают. Добавление ребра, которое создает цикл, является неверным шагом.'
 },
 {
 id: 3,
 question: 'Какой из следующих вариантов НЕ является корректным шагом в алгоритме Краскала?',
 options: [
 'Сортировка всех ребер по весу',
 'Выбор ребра с наименьшим весом, которое не создает цикла',
 'Удаление ребра, которое создает цикл',
 'Добавление ребра, которое создает цикл'
],
 correctAnswer: 3,
 explanation: 'Шаг 4: Если ребро создает цикл, его отбрасывают. Добавление ребра, которое создает цикл, является неверным шагом.'
 },
 {
 id: 4,
 question: 'Какой из следующих вариантов НЕ является корректным шагом в алгоритме Краскала?',
 options: [
 'Сортировка всех ребер по весу',
 'Выбор ребра с наименьшим весом, которое не создает цикла',
 'Удаление ребра, которое создает цикл',
 'Добавление ребра, которое создает цикл'
],
 correctAnswer: 3,
 explanation: 'Шаг 4: Если ребро создает цикл, его отбрасывают. Добавление ребра, которое создает цикл, является неверным шагом.'
 },
 {
 id: 5,
 question: 'Какой из следующих вариантов НЕ является корректным шагом в алгоритме Краскала?',
 options: [
 'Сортировка всех ребер по весу',
 'Выбор ребра с наименьшим весом, которое не создает цикла',
 'Удаление ребра, которое создает цикл',
 'Добавление ребра, которое создает цикл'
],
 correctAnswer: 3,
 explanation: 'Шаг 4: Если ребро создает цикл, его отбрасывают. Добавление ребра, которое создает цикл, является неверным шагом.'
 },
 {
 id: 6,
 question: 'Какой из следующих вариантов НЕ является корректным шагом в алгоритме Краскала?',
 options: [
 'Сортировка всех ребер по весу',
 'Выбор ребра с наименьшим весом, которое не создает цикла',
 'Удаление ребра, которое создает цикл',
 'Добавление ребра, которое создает цикл'
],
 correctAnswer: 3,
 explanation: 'Шаг 4: Если ребро создает цикл, его отбрасывают. Добавление ребра, которое создает цикл, является неверным шагом.'
 },
 {
 id: 7,
 question: 'Какой из следующих вариантов НЕ является корректным шагом в алгоритме Краскала?',
 options: [
 'Сортировка всех ребер по весу',
 'Выбор ребра с наименьшим весом, которое не создает цикла',
 'Удаление ребра, которое создает цикл',
 'Добавление ребра, которое создает цикл'
],
 correctAnswer: 3,
 explanation: 'Шаг 4: Если ребро создает цикл, его отбрасывают. Добавление ребра, которое создает цикл, является неверным шагом.'
 },
 {
 id: 8,
 question: 'Какой из следующих вариантов НЕ является корректным шагом в алгоритме Краскала?',
 options: [
 'Сортировка всех ребер по весу',
 'Выбор ребра с наименьшим весом, которое не создает цикла',
 'Удаление ребра, которое создает цикл',
 'Добавление ребра, которое создает цикл'
],
 correctAnswer: 3,
 explanation: 'Шаг 4: Если ребро создает цикл, его отбрасывают. Добавление ребра, которое создает цикл, является неверным шагом.'
 },
 {
 id: 9,
 question: 'Какой из следующих вариантов НЕ является корректным шагом в алгоритме Краскала?',
 options: [
 'Сортировка всех ребер по весу',
 'Выбор ребра с наименьшим весом, которое не создает цикла',
 'Удаление ребра, которое создает цикл',
 'Добавление ребра, которое создает цикл'
],
 correctAnswer: 3,
 explanation: 'Шаг 4: Если ребро создает цикл, его отбрасывают. Добавление ребра, которое создает цикл, является неверным шагом.'
 },
 {
 id: 10,
 question: 'Какой из следующих вариантов НЕ является корректным шагом в алгоритме Краскала?',
 options: [
 'Сортировка всех ребер по весу',
 'Выбор ребра с наименьшим весом, которое не создает цикла',
 'Удаление ребра, которое создает цикл',
 'Добавление ребра, которое создает цикл'
],
 correctAnswer: 3,
 explanation: 'Шаг 4: Если ребро создает цикл, его отбрасывают. Добавление ребра, которое создает цикл, является неверным шагом.'
 }
];
```

```

 'Использовать динамическую типизацию как в Python
 'Постоянно менять требования к проекту в процессе
],
 correctAnswer: 1,
 explanation: 'Точно! Главный принцип ДП – мемоизаци
},
{
 id: 5,
 question: 'Какая структура данных идеально помогает
 options: [
 'СНМ (Система непересекающихся множеств / Disjoin
 'Стек (LIFO).',
 'Красно-черное дерево.',
 'Хэш-таблица со списками коллизий.'
],
 correctAnswer: 0,
 explanation: 'Блестяще! СНМ (DSU) позволяет за прак
 '
 }
];

```

```

export const ExpressQuiz: React.FC = () => {
 const [currentQIndex, setCurrentQIndex] = useState<num
 const [selectedOption, setSelectedOption] = useState<
 const [isAnswered, setIsAnswered] = useState<boolean>
 const [score, setScore] = useState<number>(0);
 const [showResults, setShowResults] = useState<boolean>

 const currentQ = quizQuestions[currentQIndex];

 const handleSelectOption = (index: number) => {
 if (isAnswered) return;
 setSelectedOption(index);
 setIsAnswered(true);
 if (index === currentQ.correctAnswer) {
 setScore(prev => prev + 1);
 }
 };
};

```



```

 : ' Стоит повторить главы пособия и еще
 </p>
 </div>

 <button
 onClick={handleRestart}
 className="inline-flex items-center gap-2 px-4 rounded-xl shadow-lg shadow-indigo-600/30"
 >
 <RotateCcw className="w-5 h-5" />
 Пройти тест заново
 </button>
 </div>
);
}

return (
 <div className="bg-slate-900/90 border border-slate-800" >
 <div className="flex items-center justify-between" >
 <div className="flex items-center gap-3" >

 Экспресс-тест

 <h3 className="text-xl font-bold text-white">
 </h3>
 </div>

 Вопрос {currentQIndex + 1} / {quizQuestions.length}

 </div>

 <div className="mb-8">
 <h4 className="text-xl md:text-2xl font-bold text-white">
 {currentQ.question}
 </h4>

 <div className="space-y-4">
 {currentQ.options.map((option, idx) => {

```

```

 </div>
 </div>

 {isAnswered && (
 <div className="bg-slate-950 border border-slate-800 p-4">
 <div className="flex items-center gap-2 mb-2">
 <AlertCircle className="w-4 h-4" />
 Объяснение:
 </div>
 <p className="text-slate-300 text-sm leading-relaxed">
 {currentQ.explanation}
 </p>
 </div>
)}

 <div className="flex justify-end pt-4 border-t border-slate-800">
 <button
 onClick={handleNext}
 disabled={!isAnswered}
 className={
 "px-8 py-3.5 font-bold rounded-xl transition-colors"
 (!isAnswered
 ? "bg-slate-800 text-slate-500 border border-slate-700"
 : "bg-indigo-600 hover:bg-indigo-500 active:bg-indigo-700")
 }
 >
 {currentQIndex + 1 < quizQuestions.length ? 'Следующий' : 'Завершить'}
 </button>
 </div>
 </div>
);
};

```

```

2: [{ v: 4, w: 2 }],
3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
4: [{ v: 6, w: 1 }],
5: [{ v: 6, w: 5 }],
6: [{ v: 1, w: 1 }],
};

const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] = ' },
 { line: 8, text: ' return dist' },
];

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist = initialMatrix.map(row => [...row]);

 simulationSteps.push({
 k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Инициализация матрицы прямых путей, ∞ (99'
 activeCodeLine: 2,
 });

 const N = 7;
 for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через про
 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {

```

```

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 <div className="flex flex-wrap items-center justify"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white"
 <p className="text-sm text-emerald-300">Полн
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button onClick={() => { setIsPlaying(false);
 text-sm"> C6poc</button>
 <button onClick={() => setIsPlaying(!isPlaying
 -amber-600' : 'bg-emerald-600')}`}>{isPlaying
 <button onClick={() => { setIsPlaying(false);
 ; }} className="px-3 py-2 bg-emerald-600 te
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6 m
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-indigo-950/2
 stify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indi
 ont-bold shadow-sm">
 {currentStep.k >= 0 && currentStep.k < 7 ?
 </div>

 <svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-fw-normal" markerWidth=

```

```

 <circle cx={node.x} cy={node.y} r={isI
s] ? 4 : 2} className="transition-colors du
 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y + 32} fill=
 </text>
 </g>
);
 }}}
</svg>

```

```

<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400 r
 <p className="min-h-[40px] flex items-cente
</div>
</div>

```

```

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (
 <div key={u} className={`flex flex-wrap
 isI ? 'bg-amber-900/40 border-ambe
 isK ? 'bg-indigo-900/40 border-ind
 }`>
 <span className="font-bold text-whi
e-400">→
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u
 const isVia = currentStep.k !== c
 const isDown = currentStep.j === edge.v);
 return (
 <span key={idx} className={`px-1
 dow' : isVia ? 'bg-indigo-500 text-white bo

```

```

 else if (i === j) bg = 'bg-slate-900/60';
 return (<td key={j} className={`p-1.5 ${bg} text-slate-600`}>
 {codeLines[i][j]}
 </td>);
 })}
 </tr>
 })}
</tbody>
</table>
</div>
</div>

 {/* Код алгоритма */}
 <div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-1.5">
 <div>
 <h4 className="text-lg font-bold text-slate-600">Код алгоритма
 <div className="bg-slate-950/80 rounded-lg p-1.5">
 {codeLines.map(item => {
 <div key={item.line} className={`py-1.5 px-1.5 ${
 item.col === item.line ? 'bg-blue-900/80 text-amber-400' :
 'text-slate-600 w-5/6'
 }`}>
 {codeLines[item.line][item.col]}
 </div>
 })}
 </div>
 </div>
 </div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 39/83: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import { useState, useEffect, useMemo } from 'react';
```

```
function generateDFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const stack: string[] = []; // for visualization, just for visualization

 function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Зависит от: ${Array.from(vis).join(', ')}` });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Процесс: ${v}, ${u} в стек`, visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Обратный шаг: ${v} из стека`, visitedNodes: Array.from(vis) });
 }

 dfs(start);
 return steps;
}
```

```
function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начало: ${start}`, visitedNodes: Array.from(vis) });
}
```





```

 <circle cx={n.x} cy={n.y}
 className={`stroke-${n.label}`}
 <text x={n.x} y={n.y}
 className={`text-${n.label}`}
 {n.label}
 </text>

 {
 /* Ripple for BFS mode */
 {current && mode === 'bfs' && n !== current} && {
 <circle cx={n.x} cy={n.y}
 className="stroke-emerald-500"
 >
 }}
 </g>
 </svg>
 </div>

 {
 /* Structure View (Stack / Queue) */
 <div className="bg-slate-900 border border-emerald-500 p-10"
 r">
 <div className={`absolute -top-3 left-3 right-3 bottom-3
 ${mode === 'dfs' ? 'bg-slate-900' : 'bg-slate-800'}
 border border-emerald-500`}
 {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
 </div>

 <div className="flex flex-col gap-2 mt-10"
 {step.queueOrStack && step.queueOrStack.length > 0}
 >
 {step.queueOrStack.map((item, idx) => {
 <div key={idx} className={`w-full p-2
 ${mode === 'dfs' ? 'bg-slate-800' : 'bg-slate-900'}
 text-emerald-200`}
 ${idx === (mode === 'dfs' ? step.current : step.top) ? 'font-weight: bold' : ''}
 >
 {item}
 </div>
 })}
 </div>
 </div>
 }
 </div>

```

```
 <div className="text-sm text-slate-500">
 {step.desc}
 </div>
 </div>
 </div>
 </div>
);
}
```

---

ФАЙЛ 40/83: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
}
```

```

 { name: 'Маша (Перелом)', symptom: ' Открытый перелом' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🩸' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];

```

```

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Болит сердце' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🩸' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
 });
 return arr;
})();

```

```

let uid = 300;

```

```

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const [customName, setCustomName] = useState('');
 const [customPriority, setCustomPriority] = useState(0);
 const [log, setLog] = useState<string[]>([]);
 const [shakeEmpty, setShake] = useState(false);
 const [busy, setBusy] = useState(false);
 const [healingPatient, setHealing] = useState<Patient>({});

```

```

 const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

```

```

 // --- INSERT: Новый пациент поступает в приемный покой

```

```

 const insertPatient = useCallback((name: string, priority: number) => {

```

```

 const randomShift = Math.floor(Math.random() * 10)
 const prio = Math.max(10, Math.min(99, preset.prior
 insertPatient(preset.name, prio);
 });

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);

 addLog(` СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

 // Step 1: Root lights up as winner
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, winner: true } : x));

 setTimeout(() => {
 // Step 2: Root exits the tree, tree reorganizes
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, winner: false } : x));

 setTimeout(() => {
 // Run binary heap extraction
 setHeap(prev => {
 if (prev.length === 0) return [];
 const a = prev.map(x => ({ ...x }));

```

```

 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < heap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 return res;
});

return (
 <div className="flex flex-col gap-6">
 { /* МНЕМОНИКА / ПРАВИЛО */ }
 <div className="bg-emerald-950/40 border border-emerald-500">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 text-center">
 Queue</h3>
 <p className="text-xs text-slate-300 mt-1 leading-relaxed">
 В реанимации не важно, кто пришел раньше, а кто
 остояние (максимальный приоритет).
 Куча (Heap) автоматически перестраивает пац
 оказывался на самом верху дерева (в корне).
 </p>
 </div>
 </div>

 { /* УПРАВЛЕНИЕ */ }
 <div className="flex flex-col lg:flex-row items-stretch">
 { /* Добавить случайного */ }
 </div>
 </div>
);

```

```

 Везти
 </button>
 </form>

 {/* Забрать самого тяжелого */}
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r
 40 disabled:cursor-not-allowed text-white from-slate-800
 transition-all cursor-pointer flex items-center justify-center"
 >
 В реанимацию (EXTRACT MAX)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white
 cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 {/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
 <div className="grid grid-cols-1 lg:grid-cols-12 gap-4">

 {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}
 <div className={`lg:col-span-8 bg-slate-900 rounded-2xl min-h-[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`}>
 <div className="absolute top-4 left-4 text-[14px] font-medium">
 Сортировка по тяжести (Двоичная куча / Max-Heap)
 </div>

 <div className="relative w-full h-full min-h-[200px]">
 <svg className="absolute inset-0 w-full h-full" viewBox="0 0 1000 1000">
 {edges}
 </svg>
 </div>
 </div>
 </div>

```

```
</div>
```

```
<div className="w-full h-3 bg-gradient-to-r from-rose-500 to-rose-300">
</div>
```

```
{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 rounded-2xl p-4">
 <div className="absolute top-4 left-4 text-[10px] font-mono">
 Операционная койка
 </div>
```

```
<div className="flex-1 flex flex-col justify-between p-4">
 {healingPatient ? (
 <div className="flex flex-col items-center justify-center flex-grow-1">
 <div className="relative w-full h-full">
 0

 Операционная койка

 Операционная койка
 </div>
 </div>
 <div>
 <h4 className="text-white font-black text-2xl mb-2">
 Пациент: {healingPatient.name}
 </h4>
 <p className="text-emerald-400 text-xl font-mono">
 Состояние: Стабилизация ({healingPatient.status})
 </p>
 <div className="flex justify-center gap-4">
 <div>

 <Heart className="w-4 h-4 text-rose-500"></Heart>
 </div>
 </div>
 </div>
 </div>
) : (
```

```
<div className="text-center text-slate-600">
 0
 <p className="text-xs font-bold font-mono">Операционная койка</p>
 <p className="text-[10px] mt-1">Ждём пациента</p>
</div>
```

```
* Общий множитель скорости. Кадры были слишком короткими
* заметить, что именно переехало, и анимация читалась
*/
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```
/** Наведение на карточку ставит анимацию на паузу — мож
```

```
export const DemoPauseContext = createContext(false);
```

```
export function useLoop(steps: number, ms = 900) {
```

```
 const [step, setStep] = useState(0);
```

```
 const paused = useContext(DemoPauseContext);
```

```
 const period = Math.round(ms * SPEED);
```

```
 useEffect(() => {
```

```
 if (steps <= 1 || paused) return;
```

```
 const t = setInterval(() => setStep((s) => (s + 1)),
```

```
 return () => clearInterval(t);
```

```
 }, [steps, period, paused]);
```

```
 return step;
```

```
}
```

```
export const C = {
```

```
 idle: "#334155",
```

```
 idleStroke: "#475569",
```

```
 text: "#e2e8f0",
```

```
 dim: "#94a3b8",
```

```
 active: "#6366f1",
```

```
 done: "#10b981",
```

```
 warn: "#f59e0b",
```

```
 bad: "#f43f5e",
```

```
 edge: "#475569",
```

```
};
```



```
export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);
```

---

ФАЙЛ 42/83: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
 const step = useLoop(3, 1200);
 // i -> j напрямую 9; через k=1 получаем 3+4=7
 const pos: Record<string, [number, number]> = { i: [4, 4], j: [9, 9], k: [3, 3] };
 const viaOpen = step >= 1;
 const relaxed = step >= 2;
 return (
 <Svg
 caption={
```

```

 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) => e === v);

 return (
 <Svg caption={step === 0 ? "Граф распался на острова" : ""}
 {comps.flatMap((edges, ci) =>
 edges.map(([u, v], i) => (
 <Edge key={`_${ci}-${i}`} a={pos[u]} b={pos[v]} />
))
)}
 {Object.entries(pos).map(([k, [x, y]]) => {
 const ci = compOf(k);
 return <Node key={k} x={x} y={y} r={12} label={k} color={ci} />
 })}
 </Svg>
);
};

```

```

/** Что такое граф: вершины, рёбра, направление и вес.
export const GraphBasicsDemo: React.FC = () => {
 const step = useLoop(3, 1300);
 const pos: Record<string, [number, number]> = { A: [50, 50], B: [150, 50], C: [250, 50], D: [350, 50], E: [450, 50], F: [550, 50], G: [650, 50], H: [750, 50], I: [850, 50], J: [950, 50] };
 const edges: [string, string, number][] = [
 ["A", "B", 1], ["B", "C", 1], ["C", "D", 1], ["D", "E", 1], ["E", "F", 1], ["F", "G", 1], ["G", "H", 1], ["H", "I", 1], ["I", "J", 1],
 ["A", "D", 2], ["B", "E", 2], ["C", "F", 2], ["D", "G", 2], ["E", "H", 2], ["F", "I", 2], ["G", "J", 2],
 ["A", "E", 3], ["B", "F", 3], ["C", "G", 3], ["D", "H", 3], ["E", "I", 3], ["F", "J", 3],
 ["A", "J", 4], ["B", "I", 4], ["C", "H", 4], ["D", "G", 4], ["E", "F", 4], ["F", "E", 4], ["G", "D", 4], ["H", "C", 4], ["I", "B", 4], ["J", "A", 4],
];
 return (
 <Svg
 caption={
 step === 0
 ? "Вершины — объекты (города, слова, состояния)"
 : step === 1
 ? "Рёбра — связи между ними"
 : "Вес ребра — цена перехода"
 }
 >
 {edges.map(([u, v, w], i) => {
 const a = pos[u];
 const b = pos[v];
 const weight = w;
 return <Edge key={`_${i}`} a={a} b={b} weight={weight} />
 })}
 </Svg>
);
};

```

```

 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотн
 }
>
{raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width
 <text x={startX + i * (boxW + 4) + boxW / 2}
 {v}
 </text>
 </g>
)}}

{step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} hei
 <text x={53 + i * 52} y={77} textAnchor="mi
 {v}
 </text>
 </g>
)}}

{step >= 2 &&
 raw.map((v, i) => {
 <g key={`c${i}`}>
 <rect x={startX + i * (boxW + 4)} y={98} wi
 <text x={startX + i * (boxW + 4) + boxW / 2
 {sorted.indexOf(v)}
 </text>
 </g>
)}}
</Svg>
);
};

```

```
};
```

```
/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => {
 <SplayFrames
 captions={["р – корень, х под ним", "Один поворот: "]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p":
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"p":
]}
 />
};
```

```
/** Zig-Zig – х и р с одной стороны, крутим сначала вер
export const ZigZigDemo: React.FC = () => {
 <SplayFrames
 captions={["Линия g → p → x («бамбук»)", "Поворот В
 frames={[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112]
]}
 />
};
```

```
/** Zig-Zag – х и р с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
 <SplayFrames
 captions={["Зигзаг: g влево, x вправо", "Поворот НИ
 frames={[
 { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
 { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
 { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
]}
 />
};
```

```

const head = visited[visited.length - 1];
return (
 <Svg caption={head ? `Идём вглубь: ${visited.join("
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(i
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
 }}}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={k} stroke={k}
))}
 </Svg>
);
};

export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["труссы", "штаны"],
 ["труссы", "носки"],
 ["штаны", "боты"],
 ["носки", "боты"]
];
 const order = ["труссы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);
 const pos = Object.fromEntries(nodes.map((n) => [n.id, n]));
 return (
 <Svg caption={`Порядок: ${placed.join(" → ") || "..."}
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed[i]}
))}
 {nodes.map((n) => (
 const idx = placed.indexOf(n.id);
 return (
 <g key={n.id}>
 <rect x={n.x - 34} y={n.y - 12} width={68} height={24}
 " }} />
 <text x={n.x} y={n.y} textAnchor="middle" d

```

```

 };
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился на 2" : "Убрали C → граф остался связным"}>
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
))}
 </Svg>
);
 };
};

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [["A", "B"], ["A", "C"], ["A", "D"], ["B", "C"], ["B", "D"], ["C", "D"]];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : "Убрали вершину D → граф остался связным"}>
 {edges.map(([u, v], i) => (cut && (u === "C" || v === "D") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color="black" />))}
 {Object.entries(pos).map(([k, [x, y]]) => (
 cut && k === "C" ? (
 <circle key={k} cx={x} cy={y} r={12} fill="none" />
) : (
 <Node key={k} x={x} y={y} label={k} fill={k === "C" ? "red" : "black"} />
)
))}
 </Svg>
);
};
};

```

```

 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
 }}}
 {Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={step
 }}}
 </Svg>
);
 };
};

```

ФАЙЛ 44/83: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

export const HeapDemo: React.FC = () => {
 const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [
 const step = useLoop(states.length, 1000);
 const heap = states[step];
 const pos: [number, number][] = [[130, 24], [80, 68],
 const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
 return (
 <Svg caption="Новый элемент всплывает наверх, пока
 <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b=
 <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b=
 {heap.map((v, i) => {
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label
 }}}
 </Svg>
);
};
};

```

```

export const StackDemo: React.FC = () => {
 const states = [["A"], ["A", "B"], ["A", "B", "C"], [

```

```

 <line x1={24} y1={71} x2={10} y2={71} stroke={C.d
 </Svg>
 });
};

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [155, 95], [205, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f59e0b"];
 const root = (i: number): number => (p[i] === i ? i : p[p[i]]);
 return (
 <Svg caption="find = «кто твой староста», union = 0"
 {p.map((par, i) =>
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}
 fill="none" stroke={rootColor[root(i)]} stroke-width={2}
 >
) : null
)}
 {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
 </Svg>
);
};

```

```

const TRIE_NODES = [
 { id: 0, x: 130, y: 22, ch: "*", parent: null as number },
 { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
 { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
 { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

```

```

export const TrieDemo: React.FC = () => {
 const step = useLoop(TRIE_NODES.length + 1, 700);
 return (
 <Svg caption="Путь от корня по буквам = слово; общий

```



```

 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];
 const b = TRIE_NODES[to];
 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${a.x + 1} ${a.y}, ${b.x} ${b.y}`}
 fill="none" stroke={C.warn} strokeWidth={1.5}
 >
);
 })}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} f={n.f} />
))}
 </Svg>
);
};

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
 return (
 <Svg caption="Запрос собирается из $O(\log n)$ готовых" />
 {nodes.map((n, i) => (
 <g key={i}>
 <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={20}
 fill={n.lvl === step ? C.active : C.idle} stroke={C.warn} />
 <text x={n.x} y={n.y} textAnchor="middle" dom
 fill={C.idle} />
 </g>
))}
 </Svg>
);
};
};

```

```

<Svg caption={`Уровень j=${step}: готовые ответы дл
 {Array.from({ length: 8 }).map((_, i) => {
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={18}>
 <text x={25 + i * 30} y={29} textAnchor="middle">
 </g>
 })}
 {Array.from({ length: count }).map((_, i) => {
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 18}
 fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.done}>
 })}
</Svg>
);
};

```

```

export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svg caption={`Сделано ${step} операций, суммарно $
 {costs.map((c, i) => {
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={18}>
 fill={i < step ? (c > 2 ? C.bad : C.done) : C.done}>
 })}
 <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} />
 <text x={130} y={20} textAnchor="middle" fontSize={14}>
 </Svg>
);
};

```

```

export const NpDemo: React.FC = () => {
 const step = useLoop(2, 1400);
 return (
 <Svg caption={step === 0 ? "Найти решение – долго (N^2)" : "Решение найдено"}>
 <ellipse cx={130} cy={64} rx={112} ry={48} fill={C.done} />
 <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333" />
 </Svg>
);
};

```

```

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} wid
 fill={idx === step - 1 ? C.active : fil
 {filled && (
 <text x={57 + c * 38} y={36 + r * 32} t
 fontSize={10} fontWeight="700" fill="
)}
 </g>
);
 })
)}
 <text x={130} y={122} textAnchor="middle" fontSize
 каждая клетка считается один раз и переиспользу
 </text>
 </Svg>
);
};

```

ФАЙЛ 45/83: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo
import {

```

```

 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
 };

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
};

```

---

ФАЙЛ 46/83: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```

import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";

```

```

interface Props {

```

```
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-contain">
 <Component />
 </div>
 </div>
</div>,
document.body
);
};
```

---

ФАЙЛ 47/83: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect, useLayoutEffect, useRef, useState } from "react";
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";
```

```
export interface HintAnchor {
 termId: string;
 rect: DOMRect;
}
```

```
interface Props {
 anchor: HintAnchor | null;
 onClose: () => void;
 onOpenSimulator: (vizId: string) => void;
 onCardEnter: () => void;
 onCardLeave: () => void;
}
```

```

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 style={{
 top: pos ? pos.top : -9999,
 left: pos ? pos.left : -9999,
 width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 9999),
 visibility: pos ? "visible" : "hidden",
 }}
 role="dialog"
 >
 <div className="bg-slate-900 border border-indigo-500" >
 <div className="flex items-start gap-2 px-3 pt-3" >
 <h4 className="text-sm font-bold text-indigo-400" >
 {entry.term}
 </h4>
 <button
 onClick={onClose}
 className="text-slate-500 hover:text-white"
 aria-label="Заккрыть подсказку"
 >
 <X className="w-4 h-4" />
 </button>
 </div>
 <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
 <p className="text-[13px] leading-relaxed text-slate-300" >
 {entry.description}
 </p>
 <p>
 {entry.demo && <MiniDemo kind={entry.demo} />
 }
 </p>
 </div>
 </div>
 </div>
);

```

```

const [step, setStep] = useState(0);
const MAX_STEP = 7;

const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];

const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
];

const isNodeVisible = (id: string) => {
 if (id === 'S' && (step === 0 || step === 7)) return true;
 return false;
};

const getPotential = (id: string) => {
 if (step >= 2 && step < 7) {
 if (id === 'S') return 0;
 if (id === 'A') return 0;
 if (id === 'B') return -2;
 if (id === 'C') return -1;
 }
 return null;
};

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-width-2px";
 }
 return "stroke-amber-500 stroke-width-2px";
};

```

```

const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0})`
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${-2})`
 return e.newWeight;
 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1})`
 return e.newWeight;
 }
 return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;
 if (e.id === 'BC' && step === 5) return true;
 if (e.id === 'CA' && step === 6) return true;
 return false;
};

const isEdgeTextEmerald = (e: any) => {
 if (e.id === 'AB' && step > 4) return true;
 if (e.id === 'BC' && step > 5) return true;
 if (e.id === 'CA' && step > 6) return true;
 return false;
};

const getDesc = () => {

```



```

 case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра B → C.
 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$
 </div>
);
 case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра C → A.
 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$
 </div>
);
 case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S
 Все новые веса ребер в графе стали
 При этом старые кратчайшие пути оста
 </div>
);
 default: return "";
 }
};

return (
 <div className="bg-slate-800 p-4 rounded-xl border">
 <h4 className="text-sm md:text-base font-bold">

 Визуализация: Перевзвешивание алгоритмом
 </h4>

 <div className="flex flex-col md:flex-row gap-4">
 <div className="bg-[#0f172a] flex-1 min-h-100 p-4">
 <svg width="400" height="300" class="w-full h-full">
 <defs>

```

```

 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 className={`stroke-${u.id}`}
 markerEnd={getMarkerEnd(u.id)}
 />

 <text
 x={midX} y={midY}
 textAnchor="middle"
 dominantBaseline="bottom"
 className={`text-${u.id}`}
 style={{
 textHeight: 1.2,
 textEmergence: 1,
 fill: isS ? 'fire' : 'white'
 }}
 >{getEdgeContent(u.id)}
 </text>
 </g>
)
 }
 }

 /** Nodes */
 {nodes.map(n => {
 if (!isNodeVisible(n.id)) return
 const pot = getPotential(n.id)

 return (
 <g key={n.id} className={`node-${n.id}`}>
 <circle
 cx={n.x} cy={n.y}
 className={`stroke-${n.id}`}
 style={{
 fill: n.id === 'S'
 ? 'fill-[#2e8b57]'
 : 'fill-slate-500'
 }}
 />
 <text x={n.x} y={n.y}
 style={{
 fill: 'white'
 }}
 >{n.label}
 </g>
)
 })}

```

```

 </button>
 <button onClick={() => setStep(
 className={`flex-1 font-bold
 ${step === MAX_STEP
 ? 'bg-emerald-600/50'
 : 'bg-indigo-600 hover:bg-indigo-500'}
 {step === MAX_STEP ? "Готово" : "Далее"}
 {step !== MAX_STEP && <Skip>
 </button>
 </div>

 {/* Progress Dots */}
 <div className="flex justify-center gap-2" >
 {Array.from({length: MAX_STEP + 1}, (_, i) => (
 <div key={i} className={`w-10 h-10 rounded-full
 ${i < step ? 'bg-indigo-900' : 'bg-slate-200'}
 >
 </div>
 </div>
 </div>
</div>
</div>
);
}

```

ФАЙЛ 49/83: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";
```

```

 activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 // Generate step-by-step Kosaraju steps
 const generatedSteps: AlgoStep[] = [];
 const visited = new Set<number>();
 const order: number[] = [];
 const sccMap: Record<number, number> = {};

 const recordStep = (
 phase: AlgoStep["phase"],
 desc: string,
 currentNode: number | null,
 transposed: boolean,
 currentSccId: number,
) => {
 generatedSteps.push({
 phase,
 desc,
 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u }))
 : [...initialEdges],
 });
 };

 recordStep(

```

```
// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
}

const topoOrder = [...order].reverse();
recordStep(
 "transpose",
 `Первый проход завершен! Стек выхода в топологическом порядке`,
 null,
 true,
 -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном графе`,
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
```

```

 setCurrentStepIdx(0);
 }, []));

const step = steps[currentStepIdx] || {
 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,
 order: [],
 transposed: false,
 sccMap: {},
 currentSccId: -1,
 activeEdges: initialEdges,
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1500);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getSccColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

```

```

<defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#666666" />
 </marker>
 <marker
 id="arrow-active"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#1f77b4" />
 </marker>
 <marker
 id="arrow-transposed"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#666666" />
 </marker>
</defs>

```

```

{/* Edges */}
{initialEdges.map((edge, idx) => {

```

```

 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 markerEnd={`url(#${markerId})`}
 className="transition-all duration-300"
 />
);
 }}}

{/* Nodes */}
{initialNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited =
 step.visited.has(node.id) ||
 (step.phase === "dfs1" && step.currentNode === node.id);
 const sccId = step.sccMap[node.id];

 let classes = "fill-slate-800 stroke-slate-800";
 if (sccId) {
 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600";
 } else if (isVisited && step.phase === "dfs2") {
 classes = "fill-emerald-700 stroke-emerald-700";
 }

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration-300`}
 strokeWidth="3"
 />

```



```

 { /* Logs & Stacks */ }
 <div className="lg:col-span-4 bg-slate-950 p-5
 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Порядок обхода и стек
 </h4>

 <div className="bg-slate-900 border border-slate-800 p-4">
 <p className="text-xs text-white leading-relaxed">
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto">
 { /* Top-Sort output / Order stack representation */ }
 <div>

 СТЕК ВЫХОДА (DFS 1 ORDER):

 <div className="flex flex-wrap gap-1 bg-slate-900 p-4">
 {step.order.length === 0 ? (

 Нет элементов в стеке

) : (
 [...step.order].reverse().map((val, idx) => {
 <div
 key={idx}
 className="bg-indigo-950 border border-indigo-800
 px-2 py-1 text-xs text-white"
 >
 {val}
 </div>
 })
)}
 </div>
 </div>

 { /* List of actions log */ }
 </div>
 </div>
)}
</div>

```

```
 </button>
 </div>
 </div>
 </div>
</div>
</div>
);
};
```

---

ФАЙЛ 50/83: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from '@ant-design/icons';
import './KruskalSimulator.css';

interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}

interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'rejected';
 color: string;
}

interface StepLog {
 title: string;
```

```

const [heapQueue, setHeapQueue] = useState<string[]>()
const [activeCheatTab, setActiveCheatTab] = useState<

const [logs, setLogs] = useState<StepLog[]>([
 {
 title: 'Введение в раскраску (Краскал)',
 description: '«Краскал – это раскраска?». Предста
 овы начинать!',
 type: 'info',
 },
]);

// --- KRUSKAL STEPS ---
const kruskalSteps = [
 // Step 1: Inspect A-B (2)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в
 description: 'Оно соединяет две бесцветные верши
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Include A-B
 () => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в осто́ве',
 description: 'Вершины A and B окрашены в красны
 type: 'success'
 }, ...prev]));
 },
 // Step 3: Inspect D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',

```

```

 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю — мы перекрашиваем всё в один цвет!',
 type: 'warning'
 }, ...prev]);
},
// Step 8: Include A-D (Merge to purple)
() => {
 setVertices(prev => prev.map(v => v.color === 'blue' ? 'purple' : v.color));
 setEdges(prev => prev.map(e => e.status === 'included' ? 'excluded' : e.status));
 setLogs(prev => [{
 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий кланы',
 type: 'success'
 }, ...prev]);
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро B-E (вес 6)',
 description: 'НО: это ребро соединяет вершины B и E, которые уже соединены!',
 type: 'warning'
 }, ...prev]);
},
// Step 10: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе появится цикл!',
 type: 'error'
 }, ...prev]);
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',

```

```

 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'
 }, ...prev]);
},
// Step 16: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро C-F.',
 type: 'error'
 }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветн
 type: 'info'
 }, ...prev]));
},
// Step 18: Include G-H
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в осто́ве',
 description: 'Вершина H перекрашена в фиолетовы
 type: 'success'
 }, ...prev]));
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. С

```

```

setVertices(prev => prev.map(v => v.id === 4 ? {
setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E
setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5
setLogs(prev => [{
 title: 'Шаг 1: Плесень зарождается в Токио (Вер
 description: 'Мы выбрали вершину E как стартовую
 ритетную очередь (Heap): D-E(3), B-E(6), E-F(
 type: 'info'
}, ...prev]));
},
// Step 2: Pop D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро
 description: 'Плесень тянется по ребру D-E к вер
 type: 'info'
 }, ...prev]));
},
// Step 3: Include D-E, add D's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D:
 type: 'success'
 }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вер
 type: 'info'

```

```

 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро В-С (вес 4)',
 description: 'Плесень тянется к вершине С.',
 type: 'info'
 }, ...prev]));
 },
 // Step 9: Include В-С, add С's edges
 () => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 eap' } : e));
 setHeapQueue(['В-Е (6 - цикл)', 'Е-Ф (7)', 'D-G (8)']);
 setLogs(prev => [{
 title: 'Успех: Вершина С захвачена',
 description: 'Ребро С-Ф(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]));
 },
 // Step 10: Pop В-Е (6) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро В-Е (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет вершину В.',
 type: 'warning'
 }, ...prev]));
 },
 // Step 11: Reject В-Е
 () => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['Е-Ф (7)', 'D-G (8)', 'C-F (9)', 'E-G (10)']);
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро В-Е отброшено. Плесень не р
 type: 'error'
 }, ...prev]));
 },
 // Step 12: Pop Е-Ф (7)

```

```

 type: 'success'
 }, ...prev]);
},
// Step 16: Pop C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью',
 type: 'warning'
 }, ...prev]);
},
// Step 17: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]);
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]);
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)'])
 setLogs(prev => [{

```



```

 setLogs(prev => [{
 title: ' Успех: Плесень захватила весь граф
 description: 'Все 9 вершин поглощены. Итоговый
 иной – мы разрастались из одной точки с помо
 type: 'success'
 }, ...prev]));
 },
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)
 // Node 2(C) -> B-C(4)
 // Node 3(D) -> D-E(3)
 // Node 4(E) -> D-E(3)
 // Node 5(F) -> E-F(7)
 // Node 6(G) -> D-G(8)
 // Node 7(H) -> G-H(10)
 // Node 8(I) -> H-I(12)
 setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8
]));
 setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбира
 description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Concurrently merge into Kolkhozes (Compon
 () => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)

```

```

 e.status === 'included' || e.id === '0-3' ? { .
 });
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский
 description: 'Оба колхоза объединились по мосту
 type: 'success'
 }, ...prev]);
 },
 // Step 5: Check remaining edges & mark as rejected
 () => {
 setEdges(prev => prev.map(e => e.status === 'pend
 setLogs(prev => [{
 title: ' Успех: MST построен Борувкой за 2 шаг
 description: 'Все лишние ребра (В-Е, Е-Н, С-Е,
 вес: 51. Благодаря параллельности, мы потрати
 type: 'success'
 }, ...prev]);
 }
];

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);

```

```

 }
 return 'bg-slate-700 border-slate-500 text-slate-400';
 }
};

```

```

const getEdgeStroke = (status: string, color: string) => {
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};

```

```

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800 p-4">
 <div className="flex flex-col lg:flex-row justify-between items-center">
 <div>
 <div className="flex items-center gap-2 mb-2">

 Сунер-Интерактив 3 в 1

 <h3 className="text-2xl font-bold text-white">
 Алгоритмы Краскала
 </h3>
 </div>
 <p className="text-slate-400 text-sm">
 Наблюдай за работой Краскала (Раскраска),
 </p>
 </div>

 {/* Algorithm Tabs */}

```

```

 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
 </div>

 <button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-center gap-2
 -200 font-semibold text-xs rounded-xl border
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
 initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
 : activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text
)
 }
 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepIndex + 1}
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3
 {/* Graph Canvas */}
 <div className="lg:col-span-2 bg-slate-950 ro

```

```

 <div className="w-3 h-3 rounded-full" style={style}>
 </div>
 </>
)}
 </div>

 <div className="absolute top-4 right-4 bg-slate-500 p-2 text-white font-mono text-sm" style={style}>
 {currentSteps[stepIndex] / currentSteps.length}
 </div>

 { /* SVG Edges */ }
 <svg className="absolute inset-0 w-full h-full" style={style} viewBox="0 0 1000 1000" preserveAspectRatio="none">
 {edges.map(edge => {
 const u = vertices[edge.u];
 const v = vertices[edge.v];
 const strokeColor = getEdgeStroke(edge);
 const isInspecting = edge.status === 'inspect';
 const isRejected = edge.status === 'rejected';
 const isInHeap = edge.status === 'in_heap';

 return (
 <g key={edge.id}>
 <line
 x1={u.x + "%"}
 y1={u.y + "%"}
 x2={v.x + "%"}
 y2={v.y + "%"}
 stroke={strokeColor}
 strokeWidth={isInspecting ? 4.5 : 2}
 strokeDasharray={isRejected ? '4 4' : ''}
 className={"transition-all duration-300"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' : '#000000'}
 style={isInspecting ? {fontWeight: 'bold', color: '#f59e0b'} : {fontWeight: 'normal', color: '#000000'}}
 />
 </g>
);
 })}
 </svg>

```

```

ld-300' : 'text-rose-300'}}>
 MST построено с помощью {activeAlgo
 </p>
 <p className="text-slate-300 text-xs mt
 Общий вес минимального остова: 51. Вс
 </p>
</div>
})
</div>

{/* Logs / Heap */}
<div className="bg-slate-950 rounded-2xl border
 <div className="p-4 bg-slate-900 border-b b
 <h4 className="font-bold text-slate-200 t
 Ход выполнения
 </h4>
 </div>

 {activeAlgo === 'prim' && {
 <div className="p-3 bg-slate-900/60 borde
 <p className="text-[11px] font-bold tex
 ✂ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5"
 {heapQueue.length === 0 ? {
 <span className="text-xs text-slate
 } : {
 heapQueue.map(({item, idx}) => {
 <span key={idx} className={"px-2 p
0 shadow-md animate-pulse" : "bg-slate-800
 {item}

 })
 })
 </div>
 </div>
 })
}
```

```

<div className="flex bg-slate-950 p-1 rounded
 <button
 onClick={() => setActiveCheatTab('kruskal')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 == 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-950 text-slate-400"
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 == 'prim' ? "bg-emerald-600 text-white" : "bg-slate-950 text-slate-400"
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 == 'boruvka' ? "bg-rose-600 text-white" : "bg-slate-950 text-slate-400"
 >
 Борувка (Билет 20)
 </button>
</div>
</div>

```

```

{activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols-2"
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-indigo-500" />
 </p>
 <p className="text-2xl font-black text-white">Краскал</p>
 <p className="text-slate-400 text-xs font-medium">Билет 18</p>
 </div>
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-indigo-500" />
 </p>
 <p className="text-2xl font-black text-white">Прима</p>
 <p className="text-slate-400 text-xs font-medium">Билет 19</p>
 </div>
 </div>
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-indigo-500" />
 </p>
 <p className="text-2xl font-black text-white">Борувка</p>
 <p className="text-slate-400 text-xs font-medium">Билет 20</p>
 </div>
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-indigo-500" />
 </p>
 <p className="text-2xl font-black text-white">Борувка</p>
 <p className="text-slate-400 text-xs font-medium">Билет 20</p>
 </div>
 </div>
 </div>
)

```

```

 </div>
 </div>
 </div>
 })

 {activeCheatTab === 'prim' && (
 <div className="grid grid-cols-1 lg:grid-cols-2" >
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Clock className="w-3.5 h-3.5 text-emerald-400" />
 </p>
 <p className="text-2xl font-black text-white">Clock</p>
 <p className="text-slate-400 text-xs mt-2">
 Clock is a simple clock that shows the current time.
 </p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Award className="w-3.5 h-3.5 text-emerald-400" />
 </p>
 <p className="text-2xl font-black text-white">Award</p>
 <p className="text-slate-400 text-xs mt-2">
 Award is a simple award that shows the current time.
 </p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-emerald-400" />
 </p>
 <p className="text-2xl font-black text-white">Code</p>
 <p className="text-slate-400 text-xs mt-2">
 Code is a simple code editor that shows the current time.
 </p>
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-emerald-400" />
 </p>
 <p className="text-2xl font-black text-white">Code</p>
 <p className="text-slate-400 text-xs mt-2">
 Code is a simple code editor that shows the current time.
 </p>
 </div>
 </div>
 </div>
)
}

import heapq

```







```
}, []));
```

```
return (
```

```
 <>
```

```
 <div className="lg:hidden sticky top-[57px] z-40"
```

```
 <button
```

```
 onClick={() => setOpen(true)}
```

```
 className="w-full flex items-center gap-2.5 p-2"
```

```
 aria-label="Открыть содержание"
```

```
 >
```

```
 <span className="p-1.5 rounded-lg bg-indigo-600"
```

```
 <List className="w-4 h-4" />
```

```

```

```

```

```
 <span className="block text-[10px] uppercase"
```

```
 Содержание • {index + 1} из {total}
```

```

```

```
 <span className="block text-[13px] font-bol"
```

```

```

```
 </button>
```

```
 </div>
```

```
 {open && (
```

```
 <div className="lg:hidden fixed inset-0 z-[90]"
```

```
 <div className="absolute inset-0 bg-slate-950"
```

```
 <div className="absolute inset-y-0 left-0 w-[100%]"
```

```
 te-[tocIn_.18s_ease-out]">
```

```
 <div className="flex items-center justify-b"
```

```
 <span className="font-bold text-white tex"
```

```
 <button
```

```
 onClick={() => setOpen(false)}
```

```
 className="p-2 -mr-2 text-slate-400 hov"
```

```
 aria-label="Закрыть содержание"
```

```
 >
```

```
 <X className="w-5 h-5" />
```

```
 </button>
```

```
 </div>
```

```
 <div className="flex-1 min-h-0">
```

```

 }
 };

 return (
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center gap-3 w-full">
 <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-white font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"

```

```
 className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
</div>
</header>
);
};
```

---

ФАЙЛ 54/83: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотри
 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0">
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 })}
```

```

 }
 return <>{lines}{circles}</>;
 }}())}
</svg>
<div className="absolute bottom-2 left-2 right-2
 z-20">
 V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 border
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0"
 {{{() => {
 const pts = [
 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
];
 const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=0;j<edges.length;j++)

```

ФАЙЛ 55/83: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
```

```
interface Customer {
 id: string;
 name: string;
 emoji: string;
 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```
const PRESETS = [
 { name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue-500 to-blue-600', bubbleText: 'Матана!' },
 { name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet-500 to-violet-600', bubbleText: 'Вкусную порцию борща!' },
 { name: 'Кот Борис', emoji: '🐈', color: 'from-amber-500 to-amber-600', bubbleText: 'Плиз.' },
 { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose-500 to-rose-600', bubbleText: 'Вкусно!' },
 { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald-500 to-emerald-600', bubbleText: 'Пиво для нашего НЛО!' },
 { name: 'Бизнесмен', emoji: '👔', color: 'from-slate-500 to-slate-600', bubbleText: 'Тартаны.' },
];
```

```
let counter = 3;
```

```
export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue-500 to-blue-600', bubbleText: 'Матана!', animState: 'idle' },
])
```

```

 }, [queue, isServing]));

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog('⚠ DEQUEUE: В очереди никого нет! Повар ск...
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsServing(true);
 const headGuest = queue[0];

 addLog(` DEQUEUE: Повар наливает суп первому – ${h...

 // Step 1: Head guest gets eating animation
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {

 setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue sh...
 setQueue(prev => prev.map((c, idx) => idx === 0 ?

 setServedHistory(prev => [`${headGuest.emoji} ${h...

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(` ${headGuest.name} поел и ушел сытым.
 }, 400);
 }, 900);
}, [queue, isServing]));

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: ' ', s...

```



```

 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 opacity-40 disabled:cursor-not-allowed text
 :scale-95 transition-all cursor-pointer flex
 >
 Налить суп первому (DEQUEUE)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800
 cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
 <div className="grid grid-cols-1 md:grid-cols-12" >

 { /* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */ }
 <div className={`md:col-span-9 bg-slate-900 rounded
 -[300px] overflow-hidden ${shakeEmpty ? 'anim
 <div className="absolute top-4 left-4 text-[18px]
 Очередь голодных гостей (Буфер FIFO / BFS)
 </div>

 <div className="flex-1 flex flex-col justify-between
 <div className="flex items-end justify-between
 <div className="flex-1 flex flex-col justify-between
 <div className="flex items-end justify-between
 { /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
 <div className="flex flex-col items-center
 <div className="relative">

```

```

 ${customer.animState === 'in'
 ${customer.animState === 'eat'
 ${customer.animState === 'out
 `}
 >
 {/* Облачко мыслей */}
 <div className={`
 absolute -top-16 left-1/2 -tr
8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 borde
 `}>
 <div className="font-bold tex
 {customer.bubbleText}
 <div className="absolute -bot
it rotate-45"></div>
 </div>

 {/* Персонаж */}
 <div className={`
 w-14 h-14 rounded-2xl border-
relative
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber
 `}>
 {c

 {/* Индексы HEAD/TAIL */}
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-[
 ${isHead ? 'bg-amber-500
 `}>
 {isHead && isTail ? 'h+t'

 })
 </div>
</div>
);

```

```

 {item}
 </div>
)))
 </div>
})
</div>

 { /* Пол */
 <div className="w-full h-3 bg-gradient-to-r from-blue-500 to-purple-500" />
</div>

 { /* ЛОГ ДЕЙСТВИЙ */
 <div className="bg-slate-900 border border-slate-800 p-4" />
 <div className="text-[10px] font-mono text-slate-400" />
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between p-2`}
 >
 {idx === 0 ? История действий : null}
 {entry}
 </div>
 })}
 </div>
 </div>
);
};

```

---

ФАЙЛ 56/83: src/components/SalmonAutomatonWidget.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```

import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';

```

```

useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false),
 return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {} }
 };
 let count = 0;

 wordList.forEach((word) => {
 let curr = 0;
 for (let i = 0; i < word.length; i++) {
 const c = word[i].toUpperCase();
 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 curr = newNodes[curr].children[c];
 }
 if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
 }
 });
}

```

```

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в слове`);
 return;
 }
 const updated = [...words, cleanWord];
 setWords(updated);
 setNewWord('');
 buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Занульск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
};

```

```

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTa
 }

 logs += `Для #${u} ('${char}') fail = #${failTarg
 }

 if (childrenIds.length === 0) {
 logs += 'У этой вершины нет детей, просто переход
 }

 setNodes(updatedNodes);
 setBfsQueue(newQueue);
 setCurrentNode(r);
 setSpeechText(logs);
};

return (
 <div className="bg-slate-800/90 rounded-2xl p-6 bor
 {/* Шапка виджета */}
 <div className="flex flex-wrap items-center justifi
 <div className="flex items-center space-x-3">

 <div>
 <h4 className="text-2xl font-bold text-white
 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400">
 Построение дерева и расчет суффиксных ссы
 </p>
 </div>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}

```

```

 fill="#f87171"
 className="origin-[120px_115px] animate
 />

 {/* Глаз (с морганием) */}
 <circle cx="145" cy="85" r="12" fill="#ff
 {isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85"
) : (
 <>
 <circle cx="148" cy="85" r="6" fill="
 <circle cx="151" cy="82" r="2" fill="
 </>
)}

 {/* Рот (анимация разговора) */}
 {isTalking ? (
 <path d="M 165 105 Q 155 120 165 125 Q
) : (
 <path d="M 165 110 Q 155 115 168 118" s
)}

 {/* Улыбка / жабры */}
 <path d="M 130 80 Q 125 100 130 120" stro
 <path d="M 120 85 Q 115 100 120 115" stro
</svg>

 {/* Декоративные пузыри */}
 <div className="absolute -top-1 right-4 tex
 <div className="absolute top-8 right-1 text
</div>
 <span className="mt-3 bg-rose-600/30 border b
 Эхо-Лосось Сеня

</div>

 {/* Пузырь с текстом (Баббл) */}
 <div className="md:col-span-2 bg-slate-800 bord

```

```


 </button>
 })
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-500"
 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезапустить
 </button>
)}
</div>
</div>
</div>
</div>

```

```

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800" >
 <h5 className="text-white font-bold mb-3 flex items-center">
 Дерево и Суффиксные (fail) ссылки
 </h5>

```

```

 (() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`} >
 <defs>
 <marker id="arrowhead" markerWidth="6" markerHeight="6"
 <polygone points="0 0, 6 3, 0 6" fill="white" />
 </defs>
 </svg>
)
 })

```



```

 { /* Draw Fail Links */
 {simulationStep > 0 && Object.values(nodes)
 if (node.id === 0 || node.fail === 0) return
 }
 // Wait, all nodes get fail link. Let's draw it
 // To avoid too many lines, let's draw it only when step >= 1 and computed it
 const target = nodes[node.fail];
 if (!target || !node.x || !node.y || !target.x || !target.y) return

 // Simple bezier curve for fail link
 const isCurrent = currentNode === node.id

 return (
 <path
 key={`fail-${node.id}`}
 d={`M ${node.x} ${node.y} Q ${node.x + 10} ${node.y - 10} ${node.x + 16} ${node.y}`}
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
 })}

 { /* Draw Nodes */
 Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node.id;
 const isInQueue = bfsQueue.includes(node.id);

 let fill = '#1e293b';
 let stroke = '#475569';

 if (isCurrent) {

```

```

 </text>
 })
 </g>
);
 }}}
</svg>
);
}}({})
</div>

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
 <div className="bg-slate-900/50 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-3">
 Словарь (Паттерны)
 </h5>
 <div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => {
 <span
 key={w}
 className="bg-slate-800 border border-slate-700
 px-2 py-1 gap-1 group"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:text-white"
 title="Удалить"
 >
 ✕
 </button>

)}}
 </div>
 </div>

 <form onSubmit={handleAddWord} className="flex

```

```

 <th className="py-2 px-3">term_link</th>
 <th className="py-2 px-3">Дети</th>
 <th className="py-2 px-3">Слова (dict)
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800">
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node
 const isInQueue = bfsQueue.includes(node)

 return (
 <tr
 key={node.id}
 className={`transition-colors ${
 isCurrent
 ? 'bg-blue-900/40 font-bold text-white'
 : isInQueue
 ? 'bg-slate-800/80'
 : 'hover:bg-slate-800/40'
 }`}
 >
 <td className="py-2.5 px-3 flex items-center">
 <span
 className={`w-2 h-2 rounded-full ${
 node.id === 0
 ? 'bg-purple-500'
 : isCurrent
 ? 'bg-blue-400 animate-pulse'
 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'
 }`}
 >
 #{node.id}
 {node.id === 0 && root}
 </td>
 <td className="py-2.5 px-3">

 {node.children.map((child) => {
 return (
 <div key={child.id} style="display: inline-block; margin-right: 10px; vertical-align: top; text-align: left;">

 {child.term}
 </div>
)
 })}
 </td>
 </tr>
)
 })}
 </tbody>
</table>

```

```


 }}
 </td>
 </tr>
);
 }}}
</tbody>
</table>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 57/83: src/components/SegmentTreeVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallbck, useMe
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRi

```

```

// ===== TYPES =====

```

```

interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node index
 highlightNodes: number[]; // current
 mergeChildren?: [number, number]; // pair be
 arrayHighlight?: [number, number]; // [L, R]
 result?: number; // partial
}

```

```

// ===== PURE LOGIC: build tree =====

```

```

function buildTreeFull(arr: number[]): Record<number, number> {

```

```

 , highlightNodes: [v], arrayHighlight: [l, r] }
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[
 l, r] }`});
 return;
}
const m = (l + r) >> 1;
steps.push({ id: id++, desc: `build[${v}], ${l}, ${r}
 { ...tree }, highlightNodes: [v], arrayHighlight:
go(2 * v, l, m);
steps.push({ id: id++, desc: `Вернулись в build[${v}
 lightNodes: [v], arrayHighlight: [l, r] }`});
go(2 * v + 1, m + 1, r);
tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
steps.push({ id: id++, desc: `Объединяем: tree[${v}]
 tate: { ...tree }, highlightNodes: [v], mergeChild
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: ` Дерево построено! Корень:
 Highlight: [0, n - 1] }`});
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

 function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query[${v}], [${l}...${r}]
 odeLine: 2, treeState: tree, highlightNodes: [v]
 return 0;
 }
 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query[${v}], [${l}...${r}]

```

```

 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l
 0}. Сумма = ${res}. l++.` , codeLine: 5, treeSta
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный
 , treeState: tree, highlightNodes: [r], arrayHi
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${
 , arrayHighlight: [qL, qR], result: res }));
 }
 }
 steps.push({ id: id++, desc: ` Запрос sum([${qL}..${
 ighlightNodes: [1], arrayHighlight: [qL, qR], result
 return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val:
```

```

function buildVisualTree(treeState: Record<number, number>) {
 if (l === r) return { v, l, r, val: treeState[v] ?? null };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
 v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
```

```

const [speed, setSpeed] = useState(700);

useEffect(() => { setIdx(0); setPlaying(false); }, [s

useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx :
 const t = setTimeout(() => setIdx(i => i + 1), speed
 return () => clearTimeout(t);
}, [playing, idx, steps, speed]);

const step = steps[idx] || steps[0];
if (!step) return null;

const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
border-indigo-500 bg-indigo-600', childRing: 'ring-
-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
30 border-emerald-500 bg-emerald-600', childRing: '
-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
der-amber-500 bg-amber-600', childRing: 'ring-rose-!
[color];

return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
 | 'emerald' | 'amber'; arr: number[] }) {
 const p = Player({ steps, color });
 if (!p) return null;
 const { step, idx, setIdx, playing, setPlaying, speed
 const n = arr.length;

 const vTree = useMemo(() => buildVisualTree(step.tree

 const renderNode = useCallback((nd: VNode): React.Rea
 const isHigh = step.highlightNodes.includes(nd.v);
 const isChild = step.mergeChildren?.includes(nd.v);
 const has = nd.val !== null;

```

```

<div className="px-3 py-2 bg-slate-950 border-b b
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= s
 return (
 <div key={i} className={`flex flex-col item
 go' ? 'bg-indigo-950 border-indigo-500' : c
 er-500'} -translate-y-0.5` : 'bg-slate-900 l
 <span className="text-[7px] text-slate-500
 <span className="text-[11px] font-bold for
 </div>
);
 })}
</div>

```

```

{/* Tree */}
<div className="flex-1 p-2 overflow-x-auto bg-sla
 <div className="scale-[0.85] origin-top min-w-m
</div>

```

```

{/* Code */}
<pre className="px-3 py-2 text-[9px] md:text-[10p
 x">
 {code.map((line, i) => {
 const ln = i + 1;
 const active = step.codeLine === ln;
 return (
 <div key={ln} className={`px-2 py-0.5 round
 2 border-indigo-500 text-indigo-200' : colo
 ' : 'bg-amber-600/20 border-l-2 border-ambe
 <span className="inline-block w-4 text-sl
 </div>
);
 })}
</pre>

```

```

{/* Description + result */}
<div className="px-3 py-2.5 bg-slate-900/60 borde
 px]">

```



```

 });
 }

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
 const [customInput, setCustomInput] = useState('5, 8,');
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'>('build');

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);

 const handleApply = (e: React.FormEvent) => {
 e.preventDefault();
 const parsed = customInput.split(',').map(s => parseInt(s));
 if (parsed.length >= 2 && parsed.length <= 16) {
 setArr(parsed);
 }
 };

 const randomize = () => {
 const size = arr.length;
 const a = Array.from({ length: size }, () => Math.floor(Math.random() * 100));
 setArr(a);
 setCustomInput(a.join(', '));
 };

 const buildSteps = useMemo(() => genBuildSteps(arr), [arr]);
 const queryTDSteps = useMemo(() => genQueryTopDownSteps(arr, qL, qR), [arr, qL, qR]);
 const queryBUSteps = useMemo(() => genQueryBottomUpSteps(arr, qL, qR), [arr, qL, qR]);

```

```

<label className="text-[10px] font-bold uppercase" style={{width: 100px;}}>
<div className="flex items-center gap-2">
 <div className="flex items-center gap-1">
 L
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1">
 R
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />
 </div>
</div>
<div className="text-[10px] text-slate-500" style={{width: 100%;}}>
 Запрос: $\text{sum}(A[\{qL\}..\{qR\}]) = \{arr.slice(qL, qR+1).reduce((a, b) => a + b, 0)}\}$
</div>
</div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1">
 <button onClick={() => setView('build')} className="text-sm font-medium text-slate-500" style={{width: 50%;}}>
 Build
 <div style={{width: 100%; height: 10px; background-color: ${view === 'build' ? 'border-indigo-500' : 'border-slate-500'}; border-radius: 5px; margin-top: 5px;}}>

```



```

 {/* Verdict */}
 <div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="font-bold text-blue-400 mb-2
 <p className="text-sm text-slate-300 leadin
 <strong className="text-white">Итеративны
 кальности. Но если нужны <strong className=
 ame="text-white">рекурсивный (сверху-вниз)<
 </p>
 </div>
 </div>
 </div>
 })
 </div>
);
 }

```

ФАЙЛ 58/83: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";
import { chapters } from "../data/content";
import { VIZ_REGISTRY } from "../vizRegistry";

interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Мобильный режим: список живёт в выдвижной панели */
 onNavigate?: () => void;
}

export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId, onNavigate }) => {
 const [query, setQuery] = useState("");

 const groups = useMemo(() => {
 const q = query.trim().toLowerCase();

```

```

{groups.length === 0 && <p className="text-sm t

{groups.map([category, items]) => (
 <div key={category}>
 <div className="text-[10px] font-bold upperc
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selec
 const hasViz = Boolean(VIZ_REGISTRY[cha
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 window.scrollTo({ top: 0, behavio
 }}
 className={`w-full text-left px-3 p
 isSelected
 ? "bg-indigo-600/15 border-indi
 : "bg-slate-800/40 border-transp
 `}
 >
 {hasViz && (
 <span
 className="w-1.5 h-1.5 rounded-
 title="Есть интерактивная визуа
 />
)}
 <span className="font-semibold text
 <ChevronRight
 className={`w-4 h-4 shrink-0 mt-0
 isSelected ? "text-indigo-400"
 `}
 />
 </button>
);
 })}

```

```

 icon: string;
 }

interface Hypothesis {
 id: string;
 text: string;
 probability: number;
}

export const Simulator: React.FC = () => {
 const [activeTab, setActiveTab] = useState<'stack' |

 // Stack state
 const [stack, setStack] = useState<Plate[]>([
 { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
 { id: '3', name: 'Блюдец от торта', icon: ' ' }],
));
 const [popMessage, setPopMessage] = useState<string |

 // Queue state
 const [queue, setQueue] = useState<Person[]>([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
));
 const [dequeueMessage, setDequeueMessage] = useState<

 // Heap state
 const [heap, setHeap] = useState<Hypothesis[]>([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"', probability: 0.5 },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"', probability: 0.3 },
 { id: '3', text: 'Кандидат: "Привет, я испанский интеллект"', probability: 0.2 },
));
 const [newCustomText, setNewCustomText] = useState('');
 const [newCustomProb, setNewCustomProb] = useState(0.5);
 const [heapMessage, setHeapMessage] = useState<string |

```

```
// Queue handlers
const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: ' ' },
 { name: 'Курьер доставки', icon: ' ' },
 { name: 'Усталый админ', icon: ' ' },
 { name: 'Турист с рюкзаком', icon: ' ' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setDequeueMessage(` Встал в конец очереди: ${newPerson.name} `);
};

const servePerson = () => {
 if (queue.length === 0) {
 setDequeueMessage("⚠ Очередь пуста! Суп ждет нового посетителя.");
 return;
 }
 const firstPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setDequeueMessage(` Выдан суп первому в очереди (Фамилия: ${firstPerson.name}) `);
};

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
 setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
```

```
// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability)

return (
 <div className="bg-slate-900/90 border border-slate-800 pb-6 mb-6">
 <div className="border-b border-slate-800 pb-6 mb-6">
 <h2 className="text-2xl font-bold text-transparent bg-clip-text bg-gradient-to-r from-blue-500 to-blue-600">
 Интерактивный симулятор структур данных
 </h2>
 <p className="text-slate-400 text-sm mt-2">
 Проверь работу принципов LIFO, FIFO и Приоритетной очереди
 </p>
 </div>

 { /* Tabs */ }
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 <button
 onClick={() => setActiveTab('stack')}
 className={`flex items-center justify-between gap-2 px-4 py-2 rounded-md
 ${activeTab === 'stack'
 ? 'bg-blue-600/20 border-blue-500 text-blue-400'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
 `}
 >
 <div className="flex items-center gap-3">
 <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'text-blue-400' : 'text-slate-400'}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">Стек
 <div className="text-xs opacity-80">LIFO
 </div>
 </div>
 </div>

 DFS

 </button>

```



```

 </button>
 </div>

 {/* STACK TAB */}
 {activeTab === 'stack' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция стопки тарелок
 <span className="text-xs font-mono bg-b
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Новые тарелки ставятся наверх. Мыть можн
 </p>
 </div>
 <div className="flex flex-wrap items-center
 <button
 onClick={addPlate}
 className="flex-1 md:flex-none flex ite
 rounded-xl text-sm font-bold shadow-lg tra
 >
 <Plus className="w-4 h-4" /> Поставить
 </button>
 <button
 onClick={popPlate}
 className="flex-1 md:flex-none flex ite
 border-blue-500/40 px-4 py-2.5 rounded-xl
 >
 Помыть верхнюю
 </button>
 <button
 onClick={resetStack}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
 tion-colors cursor-pointer"
 >

```

```

 ? 'bg-gradient-to-r from-blue
le-102'
 : 'bg-slate-900/90 border-sla
 }` }
 >
 <div className="flex items-center
 {pla
 <span className={`font-bold tex
 {plate.name}

 </div>
 <span className={`text-xs font-mo
 isTop ? 'bg-blue-500 text-white
 }`}>
 {isTop ? 'Готово к помывке' : `

 </div>
);
 })))
 </div>
 })
 <div className="mt-6 text-xs text-slate-500
 ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
 </div>
 </div>
</div>
})

{/* QUEUE TAB */}
{activeTab === 'queue' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция очереди за супом
 <span className="text-xs font-mono bg-i
 /span>

```

```

 { /* Visualization */ }
 <div className="bg-slate-950/60 p-6 rounded-2xl
 overflow-x-auto">
 {queue.length === 0 ? (
 <div className="text-center py-12 text-slate-400">
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Очередь пуста</p>
 <p className="text-xs mt-1">Добавьте героя</p>
 </div>
) : (
 <div className="w-full flex items-center justify-between">
 <div className="flex flex-col items-center justify-center gap-3">
 <div>

 {queue.length}

 </div>
 <div>
 <div className="text-indigo-500 flex items-center gap-2">
 <ArrowRight className="w-6 h-6 animate-pulse">
 </div>
 <div>
 {queue.map((person, index) => {
 const isFirst = index === 0;
 return (
 <div
 key={person.id}
 className={`flex flex-col items-center justify-center gap-2
 ${isFirst
 ? 'bg-gradient-to-b from-indigo-500 to-slate-900/90
 relative'
 : 'bg-slate-900/90 border-slate-800'}
 `}
 >
 {isFirst && (
 <span className="absolute -top-10 left-1/2
 -translate-x-1/2 text-indigo-500 font-mono font-weight-normal">
 Первый (Head)

)}
 </div>
)
)}
 </div>
 </div>
 </div>
 </div>
)}
 </div>
 </div>
)

```

```

x-Heap
</h3>
<p className="text-slate-400 text-xs mt-1">
 Здесь не важно, кто пришел первым. Важн
</p>
</div>
<div className="flex items-center gap-3 w-f
 <button
 onClick={popHeap}
 className="flex-1 md:flex-none flex ite
py-2.5 rounded-xl text-sm font-bold shadow-
 >
 Взять кандидата с макс. вероятностью
 </button>
 <button
 onClick={resetHeap}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
</div>
</div>

{/* Add Form */}
<form onSubmit={addHypothesis} className="bg-
s-12 gap-4 items-end">
 <div className="md:col-span-6">
 <label className="block text-xs font-bold">
 <input
 type="text"
 value={newCustomText}
 onChange={e => setNewCustomText(e.target
 placeholder='например: "Привет, я лучший"'
 className="w-full bg-slate-900 border b
rder-emerald-500 transition-colors"
 />

```

```

>
{sortedHeap.length === 0 ? (
 <div className="text-center py-12 text-slate-500">
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Куча</p>
 <p className="text-xs mt-1">Добавьте гири</p>
 </div>
) : (
 <div className="w-full max-w-2xl space-y-4">
 <div className="flex items-center justify-between">
 КАНДИДАТ / ГИПОТЕЗА
 ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)
 </div>

 {sortedHeap.map(({item, index}) => {
 const isTop = index === 0;
 const percent = (item.probability * 100).toFixed(
00/10 scale-[1.01])'
 return (
 <div
 key={item.id}
 className={`flex items-center justify-between ${
 isTop
 ? 'bg-gradient-to-r from-emerald-500 to-slate-900/10 scale-[1.01]'
 : 'bg-slate-900/80 border-slate-700'
 }`}
 >
 <div className="flex items-center justify-between">
 <span
 className={`flex items-center justify-between ${
 isTop ? 'bg-emerald-500 text-white' : ''
 }`}
 >#{index + 1}

 <div>
 <div className={`font-bold text-white ${
 isTop && (

```

ФАЙЛ 60/83: src/components/SimulatorHub.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useMemo, useState } from "react";
import { BookOpen, Play, Search, X } from "lucide-react";
import { VIZ_REGISTRY } from "../vizRegistry";
import { chapters } from "../data/content";
```

```
interface Props {
 /** Открыть тренажёр в модальном окне. */
 onOpen: (vizId: string) => void;
 /** Перейти к теме, к которой привязан тренажёр. */
 onGoChapter: (chapterId: string) => void;
}
```

```
/**
 * Каталог всех интерактивных тренажёров.
 *
 * Раньше вкладка «Симулятор» показывала один случайный
 * теперь это витрина: видно всё, что вообще можно потыкать
 */
```

```
export const SimulatorHub: React.FC<Props> = ({ onOpen,
 const [query, setQuery] = useState("");
```

```
 const items = useMemo(() => {
 const titleById = new Map(chapters.map((c) => [c.id, c.title]));
 const all = Object.entries(VIZ_REGISTRY).map(([id, entry], index) => ({
 id,
 entry,
 chapterTitle: titleById.get(id),
 }));
 const q = query.trim().toLowerCase();
 if (!q) return all;
 return all.filter(
 (i) =>
 i.entry.title.toLowerCase().includes(q) ||
 i.chapterTitle.toLowerCase().includes(q)
);
 }, [query]);
```

```

 {items.map(({ id, entry, chapterTitle }) => (
 <div
 key={id}
 className="flex flex-col bg-slate-900 border
 >
 <h3 className="font-bold text-white text-sm
 {entry.hint && <p className="text-[12px] te
 {!entry.hint && <div className="flex-1" />}

 <div className="flex items-center gap-2 mt-
 <button
 onClick={() => onOpen(id)}
 className="flex items-center gap-1.5 te
 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Запустить
 </button>
 {chapterTitle && (
 <button
 onClick={() => onGoChapter(id)}
 title={chapterTitle}
 className="flex items-center gap-1.5
 0 hover:text-white transition-colors min-w-
 >
 <BookOpen className="w-3.5 h-3.5 shrink
 <span className="truncate max-w-[9rem
 </button>
)}
 </div>
 </div>
)})}
 </div>
</div>
);
};

```

---

```

 { id: "cases" as Tab, label: "1 · Механика: Z" },
 { id: "big" as Tab, label: "2 · Большое дерево" },
 { id: "play" as Tab, label: "3 · Собери сам" }
)).map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium
 ${tab === t.id ? "bg-indigo-600 text-white" : ""}
 `}
 >
 {t.label}
 </button>
))}
 </div>

 {tab === "cases" && <SplayRotationsViz />}
 {tab === "big" && <SplayWalkViz />}
 {tab === "play" && <SplayRotationSandbox />}

 {tab === "play" && {
 <p className="text-xs text-slate-500 mt-3">
 Порядок поворотов в песочнице сверяется с прав
 </p>
 }}
</div>
);
};

export default SplayPlayground;

```

ФАЙЛ 62/83: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useCallback, useMemo, useState } from "react";
```



```

 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(
 },
 {
 name: "Смешанный",
 hintCase: "Zig-Zag, затем Zig-Zig",
 target: 5,
 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf
 const n4: TNode = { key: 4, left: leaf(3), right:
 return { key: 8, left: n4, right: leaf(9) };
 },
 },
];

```

/\* ----- операции над деревом -----

```

export const findPath = (root: TNode | null, key: number) => {
 const path: TNode[] = [];
 let cur = root;
 while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};

```

```

 return { node: x.key, kind: "Zig-Zag (сначала нижняя
}
```

```
/* ----- раскладка -----
```

```
interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}
```

```
function layout(root: TNode | null): { nodes: Placed[];
 const nodes: Placed[] = [];
 const edges: [number, number][] = [];
 let order = 0;
```

```
 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
 };
 walk(root, 0);
```

```
 const cols = Math.max(1, order);
 const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
 const colW = 46;
 const rowH = 62;
 return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2 })),
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
 };
}
```

```

 setTree((t) => rotateUp(clone(t) as TNode, key));
 };

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build()))

return (
 <div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800">
 <div className="flex items-start justify-between p-4">
 <div>
 <h4 className="text-sm sm:text-base font-bold text-slate-400">
 <p className="text-[12px] text-slate-400 leading-4">
 Клик по узлу = один поворот с его родителем

 поворотов выбираешь сам, разбор покажем в к

 </p>
 </div>
 <button
 onClick={() => reset((presetIdx + 1) % PRESETS.length)}
 className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-medium text-slate-400 transition-colors shrink-0"
 >
 <Dices className="w-3.5 h-3.5" /> Другое дерево
 </button>
 </div>

 <div className="flex gap-2 mb-3 overflow-x-auto no-scrollbar">
 {PRESETS.map((p, i) => (
 <button
 key={p.name}
 onClick={() => reset(i)}

```

```

 return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && (
 <circle r={22} fill="none" stroke="#6366f1" strokeWidth={2} />
)}
 <circle
 r={16}
 fill={isTarget ? "#6366f1" : path.hasChildren ? "#a5b4fc" : "#475568"}
 stroke={isTarget ? "#a5b4fc" : "#475568"}
 strokeWidth={2}
 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
 }
 }
 </svg>
</div>

<div className="flex flex-wrap items-center gap-2">
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-medium text-white disabled:opacity-40 text-xs font-bold"
 >
 <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
 </button>
 <button
 onClick={redo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-medium text-white disabled:opacity-40 text-xs font-bold"
 >
 <Redo2 className="w-3.5 h-3.5" /> ВОССТАНОВИТЬ
 </button>
</div>

```

```

 <div className="flex items-center gap-2 font-1
 {wrong === 0 ? <CheckCircle2 className="w-4
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length}
 : `Узел ${target} в корне за ${moves.length}
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && (
 <li key={i}>
 ход {i + 1}: крутили <span className=
 {m.expected}

)
)
)}

)}
 {wrong === 0 && moves.length > minimal && (
 <div className="opacity-80">Порядок верный,
)}
 </div>
)}
</div>
);
};

```

ФАЙЛ 63/83: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";

```

```
/**
```

```

inorder: string[];
/** Что означают поддеревья A..D. */
ranges: string;
frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
 pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
 edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
 key: "zig",
 label: "Zig",
 when: "родитель x — уже корень, деда нет",
 rotations: "1 поворот",
 keys: { x: 20, p: 40 },
 inorder: ["A", "20", "B", "40", "C"],
 ranges: "A < 20 · B между 20 и 40 · C > 40",
 frames: [
 {
 ...ZIG_BEFORE,
 kind: "start",
 title: "Было: 20 висит под корнем 40",
 text: "Нам нужен ключ 20, а он на глубине 1. Деду
 ms: RESULT_MS,
 },
 {
 ...ZIG_BEFORE,
 kind: "aim",
 title: "Прицел: крутим ребро 40 — 20",
 text: "Ничего пока не двигается. Смотри на подсве

```

```

 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый сл
 зглаживает.",
 ms: RESULT_MS,
},
{
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
 text: "Главная хитрость этого случая: первым крути
 focus: ["g", "p"],
 ms: AIM_MS,
},
{
 ...ZZ_S1,
 kind: "result",
 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему пр
 убине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
},
{
 ...ZZ_S1,
 kind: "aim",
 title: "Прицел 2: ребро 40 – 20",
 text: "А теперь обычный одиночный поворот – тот ж
 focus: ["p", "x"],
 ms: AIM_MS,
},
{
 pos: { x: [100, 45], A: [45, 118], p: [188, 118],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 в корне, бамбук стал кустом",
 text: "Сравни с первым кадром: А и В были на глуб
 дерево, а не только достаёт ключ.",

```

```

 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе левее",
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZG_S1,
 kind: "aim",
 title: "Прицел 2: ребро 60 – 40",
 text: "Остался одиночный поворот вокруг деда – и все",
 focus: ["g", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p", "g"],
 kind: "result",
 title: "Стало: 20 и 60 – два сына сорока",
 text: "Дерево стало идеально симметричным: все четное",
 moved: ["x", "p", "g", "C"],
 ms: RESULT_MS,
 },
],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```

const COLOR: Record<string, { fill: string; stroke: string }> = {
 x: { fill: "#6366f1", stroke: "#a5b4fc" },
 p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
};

```



```

 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
 })
</g>
);
}}

{edges.map(([a, b], i) => {
 const pa = pos[a];
 const pb = pos[b];
 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a));
 return (
 <line
 key={` ${a} - ${b} - ${i} `}
 x1={pa[0]}
 y1={pa[1]}
 x2={pb[0]}
 y2={pb[1]}
 stroke={hot ? "#fbbf24" : "#475569"}
 strokeWidth={hot ? 5 : 2}
 opacity={focus && !hot ? 0.25 : 1}
 style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
}}

{(Object.keys(pos) as NodeId[]).map((id) => {
 const p = pos[id];
 if (!p) return null;

```

```

const [frame, setFrame] = useState(0);
const [playing, setPlaying] = useState(false);
const [slow, setSlow] = useState(false);

const active = CASES[caseIdx];
const total = active.frames.length;
const idx = Math.min(frame, total - 1);
const current = active.frames[idx];
const prevPos = active.frames[Math.max(idx - 1, 0)].pos;
const duration = useMemo(() => Math.round(current.ms - prevPos));
const last = idx === total - 1;

useEffect(() => {
 setFrame(0);
}, [caseIdx]);

useEffect(() => {
 if (!playing) return;
 const t = setTimeout(() => setFrame((f) => (f + 1) % total), duration);
 return () => clearTimeout(t);
}, [playing, frame, duration, total]);

return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto">
 {CASES.map((c, i) => (
 <button
 key={c.key}
 onClick={() => setCaseIdx(i)}
 className={`px-3 sm:px-4 py-2 rounded-lg text-white
 ${i === caseIdx ? "bg-indigo-600 text-white"}
 `}
 >
 {c.label}
 </button>
))}
 </div>
 </div>

```

```

 }` }
 >
 {t}

)))

 </div>
 <p className="text-[11px] text-slate-500 mt-1"
</div>
</div>

{/* индикатор автопрокрутки */}
<div className="h-1 w-full bg-slate-800 rounded-f
 <div
 key={` ${caseIdx} - ${frame} - ${playing} - ${slow}`
 className="h-full bg-indigo-500"
 style={
 playing
 ? { animation: `demoProgress ${duration}m
 : { width: "100%", background: "#334155"
 }
 }
 />
</div>

<div className="flex flex-wrap items-center gap-2"
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f - 1 + total) % total);
 }}
 disabled={idx === 0}
 className="px-3 py-2 rounded-lg bg-slate-800
 d:hover:text-slate-300 text-xs font-bold tr
 >
 ← Назад
 </button>
 <button
 onClick={() => {

```

```

 <div className="flex items-center gap-3 ml-auto"
 {[["x", "p", "g"] as const].map((id) =>
 active.keys[id] === undefined ? null : (
 <span key={id} className="flex items-center"
 <span className="w-3 h-3 rounded-full"

)
)}
 </div>
 </div>

 <p className="mt-2 text-[11px] text-slate-500">
 Листай кнопкой «Дальше» в своём темпе – кадры не
 </p>
</div>
);
};

```

ФАЙЛ 64/83: src/components/SplayWalkViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useEffect, useMemo, useRef, useState }

/**
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный поворот
 * aim – прицел на тройку g–p–x, назван случай;
 * cut – отстёгиваем среднее поддереву β (пунктир, поворот);
 * rot – поворот: зелёным подсвечены рёбра, которых не было
 * Рядом с логом всегда висит врезка «Механика одного поворота»
 * На каждом кадре – живая автопроверка in-order (сортировка)
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, collect)
 */

```

```

 return r;
 }
 cur = cur.right;
}
}
}

```

/\*\* Поднять child над его родителем (один атомарный пов

```

export function rotateUp(child: SN): void {
 const p = child.parent;
 if (!p) return;
 const g = p.parent;
 if (p.left === child) {
 p.left = child.right;
 if (child.right) child.right.parent = p;
 child.right = p;
 p.parent = child;
 } else {
 p.right = child.left;
 if (child.left) child.left.parent = p;
 child.left = p;
 p.parent = child;
 }
 child.parent = g;
 if (!g) return;
 if (g.left === p) g.left = child;
 else g.right = child;
}

```

/\*\* Рёбра в виде "p→c" – для diff между кадрами. \*/

```

export function collectEdges(t: SN | null): string[] {
 const es: string[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
 };
}

```

```

): { steps: SplayStep[]; found: boolean } {
 let root: SN | null = cloneWithParents(rootIn);
 const steps: SplayStep[] = [];
 if (!root)
 return { steps: [{ kind: "done", root: null, note:

// --- спуск ---
let cur: SN | null = root;
const pathKeys: number[] = [];
let found = false;
let last: SN = root;
while (cur) {
 last = cur;
 pathKeys.push(cur.key);
 if (key === cur.key) {
 found = true;
 break;
 }
 cur = key < cur.key ? cur.left : cur.right;
}
const x = found ? cur : last;
steps.push({
 kind: "search",
 root: cloneWithParents(root)!,
 pathKeys: [...pathKeys],
 note: isInsert
 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск уперся в ${last.key}
 rotCount: 0,
});

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
 while (root && root.parent) root = root.parent;

```

```

 });

 while (target.parent) {
 const p = target.parent;
 const g = p.parent;
 if (!g) {
 emitRotation(
 target,
 "Zig",
 [p.key, target.key],
 `Zig: у ${target.key} нет деда — хватит одного
);
 continue;
 }
 const pIsLeft = g.left === p;
 const xIsLeft = p.left === target;
 if (pIsLeft === xIsLeft) {
 const side = pIsLeft ? "оба слева" : "оба справа"
 emitRotation(
 p,
 "Zig-Zig",
 [g.key, p.key],
 `Zig-Zig (${side}: линия ${g.key}—${p.key}—${ta
 ${g.key}, а ${target.key} пока не трогаем.`
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zig",
 note: `Теперь обычный зиг: ребро ${target.parent
 rotCount,
 });
 emitRotation(
 target,
 "Zig-Zig",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key

```

```

 let d = 0;
 let cur = t;
 while (cur) {
 if (key === cur.key) return d;
 cur = key < cur.key ? cur.left : cur.right;
 d += 1;
 }
 return -1;
 };
const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [...
export const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1];

const buildInitial = (): SN => {
 let t: SN | null = null;
 for (const k of INITIAL_KEYS) t = bstInsert(t, k);
 return t!;
};

const CODE_LINES = [
 { line: 1, text: "while x has parent:" },
 { line: 2, text: " p = parent(x); g = parent(p)" },
 { line: 3, text: " if g is None: # Z" },
 { line: 4, text: " rotate(x over p)" },
 { line: 5, text: " elif p,g,x на одной линии: # I" },
 { line: 6, text: " rotate(p over g) # B" },
 { line: 7, text: " rotate(x over p)" },
 { line: 8, text: " else: # Z" },
 { line: 9, text: " rotate(x over p) # H" },
 { line: 10, text: " rotate(x over g)" },
];

/* ----- врезка «механика одного по

const MechanismInset: React.FC<{ step: SplayStep | null
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";

```



```

</g>
<text x="163" y="96" fill="#10b981" fontSize="20">
<g transform="translate(186,6)">
 <text y="10" fill="#94a3b8" fontSize="11" fontStyle="italic">
 <line x1="70" y1="46" x2="45" y2="86" stroke="#10b981" stroke-width="2">
 <line x1="70" y1="46" x2="95" y2="86" stroke="#10b981" stroke-width="2">
 <line x1="45" y1="86" x2="27" y2="126" stroke="#10b981" stroke-width="2">
 <line
 x1="45" y1="86" x2="63" y2="126"
 stroke={isRot ? "#10b981" : "#64748b"}
 stroke-width={isRot ? 3 : 2}
 />
 <circle cx="70" cy="46" r="15" fill="#059669" stroke="#10b981" stroke-width="2">
 <text x="70" y="50" textAnchor="middle" fill="#10b981" fontStyle="italic">
 <circle cx="45" cy="86" r="15" fill="#1e293b" stroke="#10b981" stroke-width="2">
 <text x="45" y="90" textAnchor="middle" fill="#10b981" fontStyle="italic">
 <circle cx="27" cy="126" r="13" fill="#0f172a" stroke="#10b981" stroke-width="2">
 <text x="27" y="130" textAnchor="middle" fill="#10b981" fontStyle="italic">
 <circle
 cx="63" cy="126" r="13"
 fill={isRot ? "#78350f" : "#0f172a"}
 stroke={isRot ? "#10b981" : "#64748b"}
 stroke-width={isRot ? 2.5 : 1.5}
 />
 <text x="63" y="130" textAnchor="middle" fill="#10b981" fontStyle="italic">
 <circle cx="95" cy="126" r="13" fill="#0f172a" stroke="#10b981" stroke-width="2">
 <text x="95" y="130" textAnchor="middle" fill="#10b981" fontStyle="italic">
 <text x="112" y="92" fill={isRot ? "#10b981" : "#64748b"}
 {isRot ? "↙ β теперь при p" : "β при p"}
 </text>
 <text x="30" y="160" fill="#64748b" fontStyle="italic">
</g>
</svg>
<p className="text-[10px] text-slate-500 leading-5">
 Обход слева-направо α x β p γ не меняется никог-
 числа поворотов. Единственный, кто меняет хозяина
</p>
</div>

```

```

 };

 useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 baseRef.current = steps[steps.length - 1].root ??
 setTreeVers((v) => v + 1);
 return;
 }
 timer.current = setTimeout(() => setIdx((v) => v +
 return () => {
 if (timer.current) clearTimeout(timer.current);
 });
 }, [playing, idx, steps]);

 const step = steps ? steps[Math.min(idx, steps.length
 const viewRoot = step ? step.root : baseRef.current;
 const pos = useMemo(() => layout(viewRoot), [viewRoot
 const nodes = useMemo(() => [...pos.entries()].map(({[
 const edges = useMemo(() => {
 const es: { a: number; b: number }[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push({ a: n.key, b: n.left.key });
 if (n.right) es.push({ a: n.key, b: n.right.key });
 walk(n.left);
 walk(n.right);
 };
 walk(viewRoot);
 return es;
 }, [viewRoot, treeVers, idx]);

 const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.p
 const autoH = Math.max(300, ...nodes.map((n) => n.py

 const dBefore = useMemo(() => (steps ? depthOf(steps[
 const dNow = steps ? depthOf(viewRoot, steps[steps.le

```

```

const strokeColor = (k: number): string => {
 if (!step) return "#64748b";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#f59e0b";
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e.includes(k)) return "#f59e0b";
 }
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 {/* Шанка */}
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Клик
 </p>
 </div>
 </div>
 <div className="flex flex-wrap items-center gap-3" >
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {
 const v = parseInt(input, 10);
 if (!Number.isNaN(v)) {
 const has = inorderKeys(baseRef.current);
 if (has) runKey(v);
 else doInsert();
 setInput("");
 }
 }
 }}
 </input>
 </div>
 </div>
)

```

```

 <button onClick={() => { setIdx(0); setPlaying(
 ld bg-slate-700 text-white disabled:opacity-0.5
 В начало
 </button>
 <button onClick={() => { setPlaying(false); setPlaying(
 -lg text-xs font-bold bg-slate-700 text-white
 ← Шаг
 </button>
 <button onClick={() => setPlaying(!playing)} className="px-3 py-1.5 rounded-lg text-xs font-medium
 er-600" : "bg-emerald-600"}`}>
 {playing ? " Пауза" : "▶ Пуск"}
 </button>
 <button onClick={() => { setPlaying(false); setPlaying(
 className="px-3 py-1.5 rounded-lg text-xs font-medium
 Шаг →
 </button>

 Шаг {idx + 1} из {steps.length} • поворотов

 {step?.caseLabel && (

 00/60">
 {step.caseLabel}

)}
 </div>
)}

 {/* Дерево */}
 <div className="bg-indigo-950/20 rounded-xl p-4 border border-indigo-500">
 {!steps && (
 <div className="absolute top-3 left-3 bg-indigo-950/20 rounded-lg text-white font-bold">
 Кликни по любому узлу – splay покадрово
 </div>
)}
 <svg viewBox={`0 0 ${autoW} ${autoH}`} className="w-full h-full">
 {edges.map((e) => {

```

```

 className="cursor-pointer"
 >
 <circle
 r={isAimNode ? 24 : 20}
 fill={stepColor(n.key)}
 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="#ffffff" fontSize="13"
 >
 {n.key}
 </text>
 </g>
);
}
}
</svg>

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-row"
 <div className="flex-1 bg-slate-950/80 p-4 rounded"
 <p className="font-semibold text-amber-400"
 {step?.kind === "aim"
 ? " Прицел:"
 : step?.kind === "cut"
 ? "∞ Отстёгиваем среднее поддерево:"
 : step?.kind === "rot"
 ? " Поворот:"
 : step?.kind === "search"
 ? " Спуск:"
 : "Статус:"}
 </p>
 <p className="min-h-[40px] flex items-center"
 {step ? step.note : "Клики по узлу или в
ке справа – механика одного поворота."}
 </p>
 {steps && step && step.kind !== "search" &&
 <p className={`text-xs mt-2 ${checkOk ? "

```

```

</div>
<div className="w-full lg:w-1/2 bg-slate-900/60" >
 <h4 className="text-sm font-bold text-slate-200">
 <div className="bg-slate-950/80 rounded-lg p-2">
 {CODE_LINES.map((l) => {
 const zig = step?.caseLabel === "Zig" &&
 const zz1 = step?.caseLabel === "Zig-Zig" &&
 const zz2 = step?.caseLabel === "Zig-Zig" &&
 const za1 = step?.caseLabel === "Zig-Zag" &&
 const za2 = step?.caseLabel === "Zig-Zag" &&
 const active = zig || zz1 || zz2 || za1 || za2
 return (
 <div key={l.line} className={`py-1 px-2 border-400" : "text-slate-400"}`>

 {l.text}

 </div>
)
 })}
 </div>
 </div>
</div>
</div>
);
};

export default SplayWalkViz;

```

ФАЙЛ 65/83: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';

```

```

interface Plate {

```

```

const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESETS
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id === newP
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsWashing(true);

```

```

 });

 const topIndex = dirtyStack.length - 1;

 return (
 <div className="flex flex-col gap-6">
 { /* МНЕМОНИКА / ПРАВИЛО */ }
 <div className="bg-blue-950/40 border border-blue-950">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именную тарелку.
 Попробуй вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>

 { /* УПРАВЛЕНИЕ */ }
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/95 transition-all cursor-pointer flex items-center justify-center"
 >
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/95 transition-all cursor-pointer flex items-center justify-center"
 >
 Вытащить чистую (POP)
 </button>
 </div>
 </div>
);
 }
}

```



```

 <div className="absolute text-4xl anim-cr
 </div>
 })

{dirtyStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-
 <p className="text-slate-500 text-xs mt-1
 </div>
) : (
 <div className={`flex-1 flex flex-col justifi

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}}
 >
 <span className="text-xs font-mono text
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-p
 </div>

 {/* Тарелки (отрисовываем в обратном поряд
 {dirtyStack.slice().reverse().map((plate,
 const idx = dirtyStack.length - 1 - rev
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded
shadow-md select-none transition-all duration
 ${plate.color}
 ${plate.animState === 'in' ? 'anim

```

```

 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-2 w-full"
 {cleanRack.map(plate => {
 <div
 key={plate.id}
 className="w-full h-8 rounded-full flex
items-center justify-center gap-2 opacity-1
 >
 {plate.n}

 {plate.ingredients}

 </div>
 })}
 </div>
)}
 </div>

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r from-blue-500 to-purple-500"
 </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-700 p-4"
 <div className="text-[10px] font-mono text-slate-200"
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between`}
 >
 {idx === 0 ?
 Действие:
 {entry}
 </div>
)}
 </div>
 </div>
</div>
);

```

```

 овпадения нет.` }));
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
}
return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` }));
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` }));
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])
 steps.push({ i, l, r, zTarget: z[i], zSource
 падают! Окно растёт.` }));
 }
}

```

```

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp
 className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}

```

```

 bgColor = step.match ===
-900/40");
 }
 }

// Z Logic
if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900";
 borderColor = "border-emerald-900";
 }
 if (step.zTarget === index) {
 borderColor = "border-amber-500";
 bgColor = step.match === "z" ? "bg-amber-500" :
-900/40");
 }
}

return {
 <div key={index} className=
 {/* L/R markers for Z */}
 {mode === "z" && step.l <= index && index <= step.r ?
 <div className="absolute left-0 right-0 top-0 bottom-0
 }}
 {mode === "z" && step.r <= index && index <= step.l ?
 <div className="absolute left-0 right-0 top-0 bottom-0
 }}

 {/* Index */}
 <span className={`text-emerald-900 font-mono text-lg transition-colors
 {char}
 </div>

 {/* Value array box */}

```

```


 </div>
 })
 {
 /* Found flash effect */
 {step.found && mode === 'flash' ?
 <div className="absolute
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_rgba(34,197,94,0.2)]
 }
 {step.found && mode === 'normal' ?
 <div className="absolute
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_rgba(34,197,94,0.2)]
 }
 }
 </div>
)
 }
 }
 </div>
 </div>

<div className="flex flex-col sm:flex-row gap-4">
 {
 /* Controls */
 <div className="flex bg-slate-900 border border-slate-800 rounded-lg
 <button onClick={prevStep} disabled={step === 0} className="p-2
rounded-lg disabled:opacity-30 disabled:hover:opacity-50">
 <Undo strokeWidth={3} size={16} />
 </button>
 <button onClick={playPause} className="p-2 text-white">
x items-center justify-center min-w-[80px]">
 {autoplay ? <Play /> : <Pause size={16} />}
 </button>
 <button onClick={nextStep} disabled={step === steps - 1} className="p-2
r:bg-slate-800 rounded-lg disabled:opacity-30 disabled:hover:opacity-50">
 <SkipForward strokeWidth={3} size={16} />
 </button>
 <button onClick={reset} className="p-2 text-white border border-slate-800">
 <RefreshCw strokeWidth={3} size={16} />
 </button>
 }
 }
</div>

```

```

 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
 }` : `int l = 0, r = 0;
 for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
 }` }

```

</pre>

</div>

</div>

);

}

---

ФАЙЛ 67/83: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";

```

```

interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

```

```

useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];

 const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
 };

 recordStep(
 "Начало топологической сортировки. Используем кла",
 null,
);

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, () => []);
 dagEdges.forEach((e) => adj[e.u].push(e.v));

 const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} ("${dagNodes[u].name}`,
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 }
 }
 };

```



```

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-

```

```

 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
</marker>
<marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
</marker>
</defs>

{/* Edges */}
{dagEdges.map((edge, idx) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v));

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#3341
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-ar

```

```

 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-100"
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" ")}
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join(" ")}
 </text>
 </g>
);
}
}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 text-slate-400 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

```

```


 Черные (готово)

 </div>
 </div>
 </div>

 <div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 text-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 text-white"
 >
 Вперед
 </button>
 </div>
 </div>
</div>
</div>
</div>
);
};

```

---

```

 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
 { key: 1, grade: -4 },
];

 /** Порядок ввода (как в черновике) – намеренно НЕ отсортирован
 export const INPUT_ORDER: Pair[] = [
 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 1, grade: -4 },
 { key: 7, grade: 2 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 12, grade: 0 },
];

 // (3,5) дубль ключа/приоритета – уберём: ввод без повт
 export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p) => !p.key || !p.grade);

 const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => a.grade - b.grade);

 /* ----- чистые алгоритмы (экспортируются для тестов) -----

 export function insert(root: TNode | null, p: Pair): TNode | null {
 if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade, left: null, right: null };
 if (p.key <= root.key) root.left = insert(root.left, p);
 else root.right = insert(root.right, p);
 return root;
 }

 export function buildTree(ps: Pair[]): TNode | null {
 let root: TNode | null = null;
 for (const p of ps) {
 root = insert(root, p);
 }
 return root;
 }

```

```

 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, note: `Соединение целиком.` });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: `Выборим его правое поддеревом с (${b.key}, ${b.grade})` });
 a.right = go(a.right, b);
 return a;
 }
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: `Выборим (${a.key}, ${a.grade}) с его левым поддеревом.` });
 b.left = go(a, b.left);
 return b;
};
return { root: go(a, b), trace };
}

export function eraseNode(root: TNode | null, key: number): TNode | null {
 const trace: { atId: NodeId | null; note: string }[] = [];
 const go = (t: TNode | null): TNode | null => {
 if (!t) return null;
 if (key < t.key) {
 trace.push({ atId: t.id, note: `${key} < ${t.key}` });
 t.left = go(t.left);
 return t;
 }
 if (key > t.key) {
 trace.push({ atId: t.id, note: `${key} > ${t.key}` });
 t.right = go(t.right);
 return t;
 }
 trace.push({ atId: t.id, note: `Нашли (${t.key}, ${t.grade})` });
 const m = mergeTree(t.left, t.right);
 trace.push(...m.trace.map((s) => ({ atId: s.chosenId, note: s.note })));
 return m.root;
 };
 return { root: go(root), trace };
}

```

```

 return pos;
 };

const treeEdges = (root: TNode | null): { from: NodeId;
 const es: { from: NodeId; to: NodeId }[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ from: n.id, to: n.left.id });
 if (n.right) { es.push({ from: n.id, to: n.right.id });
 };
 walk(root);
 return es;
};

interface TreeViewProps {
 root: TNode | null;
 width?: number;
 height?: number;
 nodeColor?: (id: NodeId) => string;
 nodeStroke?: (id: NodeId) => string;
 edgeColor?: (from: NodeId, to: NodeId) => string;
 edgeDash?: (from: NodeId, to: NodeId) => boolean;
 pulseId?: NodeId | null;
 subtitle?: string;
}

const TreeView: React.FC<TreeViewProps> = ({ root, width,
 const pos = useMemo(() => layoutTree(root), [root]);
 const nodes = useMemo(() => [...pos.entries()].map(([
 const edges = useMemo(() => treeEdges(root), [root]);
 const autoW = Math.max(420, ...nodes.map((n) => n.px +
 const autoH = Math.max(260, ...nodes.map((n) => n.py +
 const w = width ?? autoW;
 const h = height ?? autoH;
 const byId = (id: NodeId) => nodes.find((n) => n.id ===
 return (
 <div>
 <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full

```

/\* ----- плоскость (клетчатая бумага) -----

```
const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H
```

```
interface PlaneProps {
 points: Pair[];
 drawnEdges: { a: Pair; b: Pair }[];
 pending: Pair | null;
 interactive?: boolean;
 selected?: Pair | null;
 cursor?: { x: number; y: number } | null;
 badEdge?: { a: Pair; b: Pair } | null;
 onPointDown?: (p: Pair) => void;
 onPointUp?: (p: Pair) => void;
 onMove?: (e: React.PointerEvent<SVGSVGElement>) => vo
}
```



```

 stroke={isSel ? "#a5b4fc" : done ? "#38bdf8" : "#ccc"}
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} font-size=12>
 ">{p.key}</text>
 </g>
 });
}}}

{pending && (
 <g className="transition-all duration-500">
 <circle cx={PX(pending.key)} cy={PY(pending.grade)} r=10 />
 <text x={PX(pending.key)} y={PY(pending.grade) + 3.5} font-size=12>
 {pending.key}</text>
 </g>
)}
</svg>
);
};

/* ===== ИГРА: соединение мышкой ===== */

export type GEdge = { a: Pair; b: Pair }; // a – родители, b – дети

export const CANON_GAME: GEdge[] = (() => {
 const es: GEdge[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ a: { key: n.key, grade: n.grade }, b: n.left }); }
 if (n.right) { es.push({ a: { key: n.key, grade: n.grade }, b: n.right }); }
 };
 walk(buildTree(POINTS));
 return es;
})();

const sameP = (p: Pair, q: Pair) => p.key === q.key && p.grade === q.grade;
const gpid = (p: Pair) => `${p.key}:${p.grade}`;

```

```

const bst = forestIsBst(edges, all);
const roots = all.filter((p) => !edges.some((e) => gp
const complete = edges.length === all.length - 1 && r
return { heap, oneParent, acyclic, bst, complete, win
}

```

```

/** Попытка хода: почему нельзя – или ОК. */
export function tryConnect(edges: GEdge[], a: Pair, b: P
 if (sameP(a, b)) return { ok: false, reason: "Это одн
 if (a.grade === b.grade)
 return { ok: false, reason: "«!» равны – «кто выше?
 const parent = a.grade > b.grade ? a : b;
 const child = a.grade > b.grade ? b : a;
 if (parentOf(edges, gpId(child)))
 return { ok: false, reason: `У точки (${child.key};
 линию». ` };
 if (reaches(edges, parent, child))
 return { ok: false, reason: "Цикл: линия замкнёт са
 const next = [...edges, { a: parent, b: child }];
 if (!forestIsBst(next, POINTS))
 return { ok: false, reason: "Линия вниз по у допуст
 return { ok: true };
}

```

```

/* ===== РЕЖИМ BUILD ===== */

```

```

type BuildPhase = "sort" | "connect" | "game";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";

```

```

const parsePairs = (raw: string): { pairs: Pair[]; erro
 const pairs: Pair[] = [];
 const errors: string[] = [];
 const seen = new Set<string>();
 const chunks = raw.split(/[,;\n]+/).map((c) => c.trim
 for (const chunk of chunks) {
 const nums = chunk.split(/[\sxx*]+/).filter(Boolea
 if (nums.length !== 2 || nums.some((v) => !Number.i

```

```

 " });
 const [parent, child] = x.grade > y.grade ? [x, y] :
 if (edges.some((e) => pid(e.b) === pid(child)))
 return { ok: false, reason: `у точки (${child.key});
 // цикл: родитель уже сидит в компоненте ребёнка (реб
 const childComp = new Set<string>([pid(child)]);
 let grew = true;
 while (grew) {
 grew = false;
 for (const e of edges) {
 if (childComp.has(pid(e.b)) && !childComp.has(pid
 childComp.add(pid(e.a));
 grew = true;
 }
 }
 if (childComp.has(pid(parent))) return { ok: false, r
 return {
 ok: true,
 edge: { a: parent, b: child },
 reason:
 x.grade > y.grade ? undefined : "линия в treap вс
 };
}

```

```

export function checkGameState(
 points: Pair[],
 edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected:
 const n = points.length;
 if (n < 2) return { heapOk: true, singleParent: true,
 const heapOk = edges.every((e) => e.a.grade > e.b.gra
 const parentCount = new Map<string, number>();
 for (const e of edges) parentCount.set(pid(e.b), (par
 const singleParent = [...parentCount.values()].every(
 // связность: компоненты через union-find
 const comp = new Map<string, string>(points.map((p) =>
 const find = (x: string): string => {

```

```

 bstOk = false; // двое детей с равным x или с
 return;
 }
 left = sorted[0];
 right = sorted[1];
} else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяет
 if (sorted[0].key <= p.key) left = sorted[0];
 else right = sorted[0];
}
if (left) walk(left);
inorder.push(p.key);
if (right) walk(right);
};
walk(roots[0]);
if (seen.size !== n) bstOk = false;
for (let i = 1; i < inorder.length; i++)
 if (inorder[i] < inorder[i - 1]) bstOk = false;
}
return { heapOk, singleParent, connected, countOk, bstOk };
}

```

/\*\* Дерево из проведённых линий – для показа результата

```

export function edgesToTree(edges: GameEdge[]): TNode | null {
 const parentOf = new Map<string, GameEdge>();
 for (const e of edges) parentOf.set(pid(e.b), e);
 const roots = edges.filter((e) => !parentOf.has(pid(e.a)));
 if (!roots.length && edges.length) return null;
 const nodes = new Map<string, TNode>();
 const make = (p: Pair): TNode => ({
 id: pid(p),
 key: p.key,
 grade: p.grade,
 left: null,
 right: null,
 });
 const rootPair = roots.length ? roots[0].a : edges.length > 1 ? edges[0].a : null;
 if (!rootPair) return null;
 const root = make([rootPair, rootPair]);
 const queue = [root];
 while (queue.length) {
 const node = queue.shift();
 const e = parentOf.get(pid(node.b));
 if (e) {
 const child = make([pid(e.a), pid(e.b)]);
 if (pid(e.a) < pid(e.b)) node.left = child;
 else node.right = child;
 queue.push(child);
 }
 }
 return root;
}

```

```

 if (node.right) { es.push({ a: { key: node.key, g
 de.right); }
 };
 walk(userTree);
 return es;
 }, [userTree]);
 const leftRightEdges = useMemo(() => [...canonEdges].

 useEffect(() => {
 if (!playing || phase === "game") return;
 const t = setTimeout(() => {
 if (phase === "sort") {
 if (algo === "none") { setPlaying(false); return;
 if (sortIdx < 3) setSortIdx(sortIdx + 1);
 else setPlaying(false);
 } else {
 const total = order === "y-desc" ? canonEdges.l
 if (connIdx < total) setConnIdx(connIdx + 1);
 else setPlaying(false);
 }
 }, 900);
 return () => clearTimeout(t);
 }, [playing, phase, sortIdx, connIdx, algo, order, can

 const startConnect = (ord: ConnectOrder) => {
 setOrder(ord);
 setPhase("connect");
 setConnIdx(0);
 setPlaying(true);
 };

 const comparisons = algo === "quicksort" ? (n > 1 ? M
 const connEdgesAll = order === "y-desc" ? canonEdges
 const connEdges = connEdgesAll.slice(0, connIdx);
 const connDone = phase === "connect" && n >= 1 && conn

 // --- игра «соедини сам» ---
 const [gameEdges, setGameEdges] = useState<GameEdge[]>

```

```

 if (!sel) {
 setSel(p);
 setGameMsg(`Выбрана (${p.key}; ${p.grade}) – теперь
 } else if (sel.key === p.key && sel.grade === p.grade) {
 setSel(null);
 setGameMsg(null);
 } else {
 attemptConnect(sel, p);
 setSel(null);
 }
 };

const pointUp = (p: Pair) => {
 if (phase !== "game" || !sel) return;
 if (sel.key === p.key && sel.grade === p.grade) return;
 attemptConnect(sel, p);
 setSel(null);
 setCursor(null);
};

const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
 if (!sel) return;
 const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
 setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100 });
};

const undoEdge = () => {
 setGameEdges((prev) => prev.slice(0, -1));
 setGameMsg(null);
};

const showSolution = () => {
 setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
 setHintUsed(true);
 setGameMsg("Решение показано – собери сам в следующем уровне");
};

return (
 <div className="space-y-5">
 {/* Произвольный список */}

```

```

 <span className="text-xs text-slate-400"
 {[["quicksort", "counting"] as const].map((a) => {
 <button
 key={a}
 onClick={() => { setAlgo(a); setSort(a);
Math.round(n * Math.log2(n)) : 0} сравнений
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg text-sm
bg-emerald-600 text-white" : "bg-slate-700 text-slate-400"}
 >
 {a === "quicksort" ? "quicksort (сравнения)" : "counting"}
 </button>
)]}
 </>
) : (
 <>

 <button
 onClick={() => startConnect("y-desc")}
 className={`px-3 py-1.5 rounded-lg text-sm
bg-emerald-600 text-white" : "bg-slate-700 text-slate-400"}
 >
 по y ↓ (алгоритм)
 </button>
 <button
 onClick={() => startConnect("left-right")}
 className={`px-3 py-1.5 rounded-lg text-sm
"bg-emerald-600 text-white" : "bg-slate-700 text-slate-400"}
 >
 слева направо
 </button>
 <button
 onClick={startGame}
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg text-sm
go-600 text-white" : "bg-indigo-500/80 text-white"}
 >
 соединить сам

```

```

) : gameMsg ? (
 <>△ {gameMsg}</>
) : (
 <>Кликни точку-родителя (выше по «!»), за
)
) : connDone ? (
 <> Проведено {connEdges.length} линий в по
 : точки прибиты, дерево одно. А теперь
) : (
 <>Линий проведено: {connIdx} из {connEdgesA
 })
 </div>
</div>

```

```

{/* Чек-лист инвариантов */}
<div className={`rounded-xl p-4 border transition
er-slate-700 bg-slate-900/40`} `>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {phase === "game" ? (
 <>
 {verdict.heapOk ? " " : " "} каждое р
 {verdict.singleParent ? " " : " "} у
 {verdict.connected ? " " : " "} все т
 {verdict.countOk ? " " : " "} линий р
 {verdict.bstOk ? " " : " "} обход сле
 </>
) : (
 <>
 {connDone ? " " : " "} каждое ребро в
 {connDone ? " " : " "} обход слева-на
 {connDone ? " " : " "} линий ровно <b
 </>
)}

 {phase === "game" && (
 <div className="flex flex-wrap gap-2 mt-3">
 <button onClick={undoEdge} disabled={!gameE

```





```

 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 шаг {i
 </div>

```

```

<div className="grid grid-cols-1 md:grid-cols-2 g
 <div className="bg-sky-950/20 rounded-xl p-3 bo
 <h5 className="text-sm font-bold text-sky-300
 <TreeView root={l} width={480} height={220} n
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 b
 <h5 className="text-sm font-bold text-rose-300
 <TreeView root={r} width={480} height={220} n
 </div>
</div>

```

```

<div className="text-xs text-slate-300 bg-slate-9
 {cur?.chosenId ? (
 <>⊗ {cur.note}</>
) : (
 <>Готово: {done ? "деревья склеены обратно в
 />
 </>
)}
</div>

```

```

{done && (
 <div className="bg-emerald-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-emerald
 <TreeView root={merged.root} subtitle="merge(
 </div>
)}

```

```

<div className="text-xs text-slate-400 bg-slate-9
 Грамотно про «слияние»: у корней сравни
 в (то, что сохраняет порядок x) отправляется
</div>
</div>

```

```

);

```

```

 root={fullTree}
 nodeColor={({id} => (pathSet.has(id) ? "#783"
 nodeStroke={({id} => (pathSet.has(id) ? "#f5
 subtitle="Куча сама умеет удалять только ве
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={res.root} subtitle="Валидность
 <ul className="mt-2 space-y-1 text-sm">
 обход слева-направо по-прежнему сорти
 каждое ребро вниз по у (куча целая) –

 </div>
)}

 <div className="text-xs text-slate-300 bg-slate-9
 {steps[idx]?.note}
 </div>

 <div className="text-xs text-slate-400 bg-slate-9
 Ответ на «убрать можно только самый верхний
 ягивает merge двух детей. Удаляется любая<
 </div>
</div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("h
 // Полное дерево из 7 узлов (куча): индексы 1..7
 const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
 const full = useМемо(() => buildTree(POINTS), []);
 // Честная слотовая разметка treap: индексы как в куче
 const slots = useМемо(() => {

```

```

<div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-rose-300" >
 >
 <TreeView root={full} subtitle="Попробуй «посчитать»" />
 <div className="flex gap-2 overflow-x-auto mt-2" >
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13].map((i) => {
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 rounded border-slate-700` : "bg-rose-950/40 border-rose-950/40"} >
 <div className="text-slate-500 text-sm" >{i}</div>
 {id ? <div className="text-white font-weight-bold" >{id}</div> : null}
 </div>
);
 })}
 </div>
 <p className="text-xs text-rose-300 mt-2">Смотри, что левая ветка ушла на 4 уровня и заняла два уровня указателями (left/right), а не арифметикой.</p>
 </div>
)}

<div className="text-xs text-slate-400 bg-slate-900/40 p-2" >
 Грабля «дети в массиве по 2k/2k+1»: формируем: слои дырявые, и «адрес» ребёнка посчитаем.
</div>
</div>
);
};

/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) ===== */

const SearchMode: React.FC = () => {
 // Подобрано так, что бинарный поиск ГАРАНТИРОВАННО пройдет
 // mid (индекс 3) = 9 > 7 → уходит влево, а цель лежит вправо
 const shuffled: Pair[] = [
 { key: 12, grade: 0 },
];
};

```

```

 {stage === 0 ? "Найти у для x=7 бинарным по
 </button>
 }}
 {stage === 2 && (
 <button onClick={() => setBidx(Math.min(bsSte
 -1.5 rounded-lg text-xs font-bold bg-indigo
 Шаг поиска →
 </button>
)}
 <button onClick={() => { setStage(0); setBidx(0
 Сброс</button>
</div>

<div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p,
 const highlight = stage === 1 && i === wrongM
 const inBs = stage === 2 && (() => { const s
 const isMid = stage === 2 && (() => { const s
 const isFound = stage === 2 && bidx >= bsStep
 return (
 <div key={` ${p.key}:${p.grade}-${i}`} class
 nsition-all ${highlight ? "border-rose-500
 r-amber-500 bg-amber-950/30" : inBs ? "bord
 <div className="text-white font-bold">{p.
 <div className="text-slate-400 text-[10px
 </div>
);
)}}
</div>

<div className="text-xs text-slate-300 bg-slate-9
 {stage === 0 && <>Массив пар не отсортирован
 а беспорядок.</>}
 {stage === 1 && <> Бинарный поиск посмотрел на
 ю половину... в которой лежала цель. На
 реставил: массив как был в беспорядке, та
 {stage === 2 && bidx < bsSteps.length - 1 && <>
 {stage === 2 && bidx >= bsSteps.length - 1 && <>

```

```

 const child = sel.grade > p.grade ? p : sel;
 setEdges([...edges, { a: parent, b: child }]);
 setMsg(`  Линия (${parent.key}; ${parent.grade})`);
 setSel(null);
 } else {
 setMsg("x " + res.reason);
 setSel(null);
 }
};

```

```
const win = v.win;
```

```

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setEdges(edges.slice(0, 3)); }}>
 rounded-lg text-xs font-bold bg-slate-700 tex
 <button onClick={() => { setEdges([]); setSel(null); }}>
 3 py-1.5 rounded-lg text-xs font-bold bg-slate-700
 <button onClick={() => { setEdges(CANON_GAME.map((e, i) => {
 }); }} className="px-3 py-1.5 rounded-lg text-xs font-b
 линий:
 </div>

 <div className="bg-emerald-950/20 rounded-xl p-4">
 <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} className="w-full h-full">
 <defs>
 <pattern id="gm-grid" width="14" height="14">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1">
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16}>
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)}>
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PY(1)}>

 {edges.map((e, i) => (
 <line
 key={`ge-${i}`}

```

```


 {win && {
 <p className="mt-3 text-sm font-bold text-emerald-500">
 ения просто не существует.</p>
 }}
 </div>

 <div className="text-xs text-slate-300 bg-slate-900">
 </div>
);
 };

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "layers";

const TABS: { id: Tab; label: string }[] = [
 { id: "build", label: "Собрать" },
 { id: "split", label: "Split $\times$ " },
 { id: "merge", label: "Merge " },
 { id: "erase", label: "Erase " },
 { id: "layers", label: "Миф: $2k/2k+1$ " },
 { id: "search", label: "Поиск $\neq$ сортировка" },
 { id: "game", label: "Игра " },
];

export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 <div className="flex flex-wrap items-center justify-center">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">

```

```

import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimula";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomaton";
import { SegmentTreeVisualizer } from "./SegmentTreeVis";
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithms";
import { TopologicalSortViz } from "./TopologicalSortVi";
import { WaterfallAnimationWidget } from "./WaterfallAn";
import ChapterImage from "./ChapterImage";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumVi";
import { SparseTableViz } from "./visualizers/SparseTab";
import { TreapBuildViz } from "./TreapBuildViz";

```

```

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

```

```

/**
 * Единый реестр интерактивных демонстраций.

```



```

 Component: TreapBuildViz,
 },
 "prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число — увидите, из чего оно сложилось",
 Component: PrefixSumViz,
 },
 "dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
 },
 "splay-tree": {
 title: "Splay: механика → большое дерево → собери сегмент",
 hint: "Три вкладки: покадровая механика поворотов в узле,
 песочница «подними узел сам» с проверкой порядка.",
 Component: SplayPlayground,
 },
 "splay-rotations": {
 title: "Splay: механика → большое дерево → собери сегмент",
 hint: "Три вкладки: покадровая механика поворотов в узле,
 песочница «подними узел сам» с проверкой порядка.",
 Component: SplayPlayground,
 },
 "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: GraphDfsBfs },
 "top-sort": { title: "Топологическая сортировка", Component: TopSort },
 "scc-kosaraju": { title: "Компоненты сильной связности", Component: SccKosaraju },
 "graph-articulation": { title: "Точки сочленения", Component: GraphArticulation },
 "bridges-code": { title: "Мосты: DFS + tin/low", Component: BridgesCode },
 "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: EulerPathVsCycle },
 "planarity-euler-formula": { title: "Планарность: K5 и K3,3", Component: PlanarityEulerFormula },
 dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем соседей",
 Component: () => (
 <>
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
)
 }
}

```



```

const [jumpPath, setJumpPath] = useState<{ from: number, to: number }>(null);
const [foundMatches, setFoundMatches] = useState<{ index: number, match: string }[]>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0);

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/ /g, '');
 setTextToScan(clean);
 resetSearchSimulation();
};

const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся на базу');
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;

```

```

 });
 }
 temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWords}
 setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMat
}
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди -
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 },

```

```

 targetNodeId: 8,
 title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТ
 desc: ' ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
). Рыба плавает в ROOT и ищет ветку "H". У корня
 суффикс "H"!',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель
 е подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'R',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК СТ
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плавает в #1 (H) и ищет ветку "E". У вершины

```

```

 {/* Переключатель режимов */}
 <div className="bg-slate-900 p-1.5 rounded-xl b
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg sl
 : 'text-slate-400 hover:text-white'
 `}
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корн
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg sha
 : 'text-slate-400 hover:text-white'
 `}
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
 </div>
</div>

```

```

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x
 <div>
 <h5 className="text-indigo-300 font-bold
 Полный обход в ширину (В
 </h5>

```

```

 {trainStep === TRAINING_STEPS_INFO.length
 </button>
 </div>
</div>

/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
 >
 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
 ainInfo.inspectedParentFail}.label}}
 {currentTrainInfo.codeLine === 2 && <
 к родителю}
 </div>
 <div className={`p-1.5 rounded flex ite
 l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
 т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex ite

```

```

 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border border-blue-500 focus:outline-none focus:border-blue-500"
 />
 </div>

 <div>
 <div className="mb-4 flex flex-wrap gap-1">

 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center justify-center
 ${idx === currentIndex ? 'bg-blue-600 text-white shadow' :
 idx < currentIndex ? 'bg-emerald-950/60 text-emerald-500' :
 'bg-slate-800 text-slate-500'}
 >
 {char}

)}
 </div>

 ГOTOB0 ✓

 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-medium text-white

```



```

 4. if (is_terminal[curr]) report
 {activeSearchCodeLine === 4 && <span <
овпадение!}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-300 font-bold m
 <AlertCircle className="w-3.5 h-3.5 tex
 </h5>
 <div className="bg-slate-800 p-3.5 rounded
tems-center">
 {searchLog}
 </div>
</div>
</div>
</div>
))

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 vi
 -hidden shadow-2xl">
 {/* Водопадные фоновые эффекты */}
 <div className="absolute inset-0 opacity-10 bg-
 o-900 to-transparent pointer-events-none"></d
 <div className="absolute top-0 left-1/2 -transl
 <div className="w-8 bg-blue-400 h-full animat
 <div className="w-12 bg-blue-300 h-full anima
 <div className="w-6 bg-blue-500 h-full animat
 </div>

 <div className="flex items-center justify-betwe
 <h5 className="text-white font-bold text-base
 Ветвящийся синий водопад (Бо
 </h5>
 <div className="flex flex-wrap gap-4 text-xs"
 <span className="flex items-center gap-1 te

```

```

 /* Рендер прямых переходов (потоков воды)
 {Object.entries(TRIE_TRANSITIONS).map(([fromNode, toNode]) => {
 const fromNode = WATERFALL_NODES[Number(fromNode)];
 return Object.entries(children).map(([childId, childNode]) => {
 const toNode = WATERFALL_NODES[toId];

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' ||
 уpe === 'normal';

 // Подсветка в режиме обучения (когда и
 let isTrainingExamined = activeMode === 'training' ||
 let isMatchBranch = isTrainingExamined & isMatchBranch;

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`}>
 /* Линия течения воды */
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
 className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
 />
 /* Символ перехода */
 <circle cx={{fromNode.x + toNode.x} / 2} cy={{fromNode.y + toNode.y} / 2}
 r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5b') : 'white'}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 />
 <text
 x={{(fromNode.x + toNode.x) / 2}}
 y={{(fromNode.y + toNode.y) / 2} + 15}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5b') : 'white'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 />
 </g>
);
 });
 });
);
}

```

```

 const cy = node.y + dy / 2 - 30;

 return (
 <g key={`fail-${node.id}`}>
 <path
 d={`M ${node.x} ${node.y} Q ${cx} $
 fill="none"
 stroke={isHighlighted ? '#f43f5e' :
 strokeWidth={isHighlighted || isJust
 strokeDasharray="6,6"
 className={isHighlighted || isJustE
 opacity={isHighlighted || isJustEst
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b
 >
 fail[{node.label}] = {targetNode.
 </text>
)}
 </g>
);
 }
 }
}

```

```

 { /* Рендер вершин (бассейнов водопада) */ }
 {Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'target' && node.id === targetNode.id;
 const hasWords = node.words.length > 0;

```

```

 return (
 <g key={`node-${node.id}`} className={isCurrent ? 'active' : ''}>
 { /* Внешнее свечение для активной вершины */ }
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r={node.radius} fill="none" stroke="black" stroke-width={2} />
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r={node.radius} fill="none" stroke="red" stroke-width={2} />
)}

```

```

 <text x="0" y="3" fontSize="10" f
 *
 </text>
 </g>
 })
</g>
);
}}
}}}

{/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */
<g>
 {/* Круг-подложка под рыбу */}
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 {/* Сама рыба */}
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :
 </text>
</g>

</svg>
</div>
</div>

```

ФАЙЛ 71/83: `src/components/visualizers/PrefixSumViz.tsx`

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМ:

```
import React, { useMemo, useState } from "react";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка — наведение на число:
 * • в 1D наводим на P[i] — подсвечивается кусок массива
 * • в 1D наводим на a[i] — видно, в какие префиксы он входит
 * • в 2D наводим на ячейку — подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];

const cellBase = `flex items-center justify-center rounded-md font-mono
w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-sm`;

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 return (
 <div className="w-full bg-slate-950 rounded-2xl border border-white/10">
 <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
 {(
 [
 <button key="1d" type="button" className={mode === "1d" ? "bg-white text-black" : "text-white"}>1D</button>,
 <button key="2d" type="button" className={mode === "2d" ? "bg-white text-black" : "text-white"}>2D</button>
]
)}
 </div>
 <div></div>
 </div>
);
};
```

```

const sum = pref[range.r + 1] - pref[range.l];

return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
 <div className="inline-block min-w-full">
 {/* индексы */}
 <div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map((_, i) => (
 <div key={i} className={` ${cellBase} !h-5
 {i}
 </div>
))}
 </div>

 {/* массив a */}
 <div className="flex gap-1.5 items-center mb-1">
 <div className="w-[46px] shrink-0 text-right">
 {A1.map((v, i) => {
 const inCover = covered && i >= covered.f
 const inRange = i >= range.l && i <= range.r
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: 'cover' })}
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r ? r : i))}
 className={` ${cellBase} cursor-pointer
 {inCover
 ? "bg-indigo-500 text-white ring-1 ring-indigo-500"
 : inRange
 ? "bg-indigo-900/70 text-indigo-300"
 : "bg-slate-800 text-slate-300"}
 `}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
)
)}
 </div>
 </div>
 </div>
 </div>
)
)

```

```

 hover.i === 0 ? {
 <p className="text-slate-300">
 <b className="text-emerald-400">P[0] = 0
 отрезка, начинающегося с нуля.
 </p>
 } : {
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 – это сумма всего, что набежало от начала

 a[0..{hover.i - 1}] = {A1.slice(0, hove

 </p>
 }
) : hover?.kind === "a" ? {
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[hover.i]}
 {" "}
 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.

 </p>
) : {
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку
 </p>
 }
 })
 </div>

 {/* Запрос */}
 <div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 </div>

```

```

const cornerOf = (i: number, j: number) => {
 if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
 if (i === rect.r1 && j === rect.c2 + 1) return "B";
 if (i === rect.r2 + 1 && j === rect.c1) return "C";
 if (i === rect.r1 && j === rect.c1) return "D";
 return null;
};

return (
 <div className="space-y-5">
 <div className="grid gap-5 lg:grid-cols-2">
 {/* Исходная матрица */}
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {A2.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
 return (
 <div
 key={j}
 onClick={() => setRect((r) => {
 i < r.r1 || j < r.c1 ? { r1: i, r2: i, c1: j, c2: j } : r;
 })}
 className={` ${cellBase} cursor-pointer ${
 inRect ? "bg-indigo-500 text-white" : ""
 }`}
 />
)
 })}
 </div>
 {v}
 </div>
)};
 </div>
 </div>
 </div>
 </div>
);

```



```

 </div>
)}}
 </div>
 </div>
</div>

<div className="min-h-[54px] bg-slate-900 border border-indigo-500"
 {hover ? {
 <p className="text-slate-300">
 <b className="text-amber-400">
 $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hover.j] +$
 {" "}
 – сумма всего прямоугольника от левого верхнего угла до
 </p>
 } : {
 <p className="text-slate-500">Наведите курсор на ячейку
 }}
</div>

<div className="bg-slate-900 border border-indigo-500"
 <p className="font-mono text-sm sm:text-base text-slate-500">
 сумма = A + <span
 C + <span
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
</div>
</div>
);
};

```

```

 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);

 for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
 }

 for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],
 ST2D[r + (1 << (kx-1))][c][kx-1][ky]
);
 } else {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx][ky-1],
 ST2D[r][c + (1 << (ky-1))][kx][ky-1]
);
 }
 }
 }
 }
 }

 export const SparseTableViz: React.FC = () => {
 const [tab, setTab] = useState<"1d" | "2d_build" | "2d">("1d");

 return (

```

```

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

 // Total steps: we animate computing each element for LOG steps
 const computeSteps: { i: number; j: number }[] = [];
 for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
 }

 const maxStep = computeSteps.length;
 const covered = hover ? { from: hover.i, to: hover.i + (1 << hover.j) } : null;

 return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-600">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))])$
 </p>
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-700 disabled:opacity-50"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 </div>
 </div>
 <div>
 <table>
 <caption>Sparse Table (Min)</caption>
 <thead>
 <tr>
 <th>i</th>
 <th>j</th>
 <th>ST[i][j]</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <td>0</td>
 <td>1</td>
 <td>min(0, 1)</td>
 </tr>
 <tr>
 <td>1</td>
 <td>2</td>
 <td>min(1, 2)</td>
 </tr>
 <tr>
 <td>2</td>
 <td>3</td>
 <td>min(2, 3)</td>
 </tr>
 <tr>
 <td>3</td>
 <td>4</td>
 <td>min(3, 4)</td>
 </tr>
 <tr>
 <td>4</td>
 <td>5</td>
 <td>min(4, 5)</td>
 </tr>
 <tr>
 <td>5</td>
 <td>6</td>
 <td>min(5, 6)</td>
 </tr>
 <tr>
 <td>6</td>
 <td>7</td>
 <td>min(6, 7)</td>
 </tr>
 <tr>
 <td>7</td>
 <td>8</td>
 <td>min(7, 8)</td>
 </tr>
 <tr>
 <td>8</td>
 <td>9</td>
 <td>min(8, 9)</td>
 </tr>
 <tr>
 <td>9</td>
 <td>10</td>
 <td>min(9, 10)</td>
 </tr>
 <tr>
 <td>10</td>
 <td>11</td>
 <td>min(10, 11)</td>
 </tr>
 <tr>
 <td>11</td>
 <td>12</td>
 <td>min(11, 12)</td>
 </tr>
 <tr>
 <td>12</td>
 <td>13</td>
 <td>min(12, 13)</td>
 </tr>
 <tr>
 <td>13</td>
 <td>14</td>
 <td>min(13, 14)</td>
 </tr>
 <tr>
 <td>14</td>
 <td>15</td>
 <td>min(14, 15)</td>
 </tr>
 <tr>
 <td>15</td>
 <td>16</td>
 <td>min(15, 16)</td>
 </tr>
 <tr>
 <td>16</td>
 <td>17</td>
 <td>min(16, 17)</td>
 </tr>
 <tr>
 <td>17</td>
 <td>18</td>
 <td>min(17, 18)</td>
 </tr>
 <tr>
 <td>18</td>
 <td>19</td>
 <td>min(18, 19)</td>
 </tr>
 <tr>
 <td>19</td>
 <td>20</td>
 <td>min(19, 20)</td>
 </tr>
 <tr>
 <td>20</td>
 <td>21</td>
 <td>min(20, 21)</td>
 </tr>
 <tr>
 <td>21</td>
 <td>22</td>
 <td>min(21, 22)</td>
 </tr>
 <tr>
 <td>22</td>
 <td>23</td>
 <td>min(22, 23)</td>
 </tr>
 <tr>
 <td>23</td>
 <td>24</td>
 <td>min(23, 24)</td>
 </tr>
 <tr>
 <td>24</td>
 <td>25</td>
 <td>min(24, 25)</td>
 </tr>
 <tr>
 <td>25</td>
 <td>26</td>
 <td>min(25, 26)</td>
 </tr>
 <tr>
 <td>26</td>
 <td>27</td>
 <td>min(26, 27)</td>
 </tr>
 <tr>
 <td>27</td>
 <td>28</td>
 <td>min(27, 28)</td>
 </tr>
 <tr>
 <td>28</td>
 <td>29</td>
 <td>min(28, 29)</td>
 </tr>
 <tr>
 <td>29</td>
 <td>30</td>
 <td>min(29, 30)</td>
 </tr>
 <tr>
 <td>30</td>
 <td>31</td>
 <td>min(30, 31)</td>
 </tr>
 <tr>
 <td>31</td>
 <td>32</td>
 <td>min(31, 32)</td>
 </tr>
 <tr>
 <td>32</td>
 <td>33</td>
 <td>min(32, 33)</td>
 </tr>
 <tr>
 <td>33</td>
 <td>34</td>
 <td>min(33, 34)</td>
 </tr>
 <tr>
 <td>34</td>
 <td>35</td>
 <td>min(34, 35)</td>
 </tr>
 <tr>
 <td>35</td>
 <td>36</td>
 <td>min(35, 36)</td>
 </tr>
 <tr>
 <td>36</td>
 <td>37</td>
 <td>min(36, 37)</td>
 </tr>
 <tr>
 <td>37</td>
 <td>38</td>
 <td>min(37, 38)</td>
 </tr>
 <tr>
 <td>38</td>
 <td>39</td>
 <td>min(38, 39)</td>
 </tr>
 <tr>
 <td>39</td>
 <td>40</td>
 <td>min(39, 40)</td>
 </tr>
 <tr>
 <td>40</td>
 <td>41</td>
 <td>min(40, 41)</td>
 </tr>
 <tr>
 <td>41</td>
 <td>42</td>
 <td>min(41, 42)</td>
 </tr>
 <tr>
 <td>42</td>
 <td>43</td>
 <td>min(42, 43)</td>
 </tr>
 <tr>
 <td>43</td>
 <td>44</td>
 <td>min(43, 44)</td>
 </tr>
 <tr>
 <td>44</td>
 <td>45</td>
 <td>min(44, 45)</td>
 </tr>
 <tr>
 <td>45</td>
 <td>46</td>
 <td>min(45, 46)</td>
 </tr>
 <tr>
 <td>46</td>
 <td>47</td>
 <td>min(46, 47)</td>
 </tr>
 <tr>
 <td>47</td>
 <td>48</td>
 <td>min(47, 48)</td>
 </tr>
 <tr>
 <td>48</td>
 <td>49</td>
 <td>min(48, 49)</td>
 </tr>
 <tr>
 <td>49</td>
 <td>50</td>
 <td>min(49, 50)</td>
 </tr>
 <tr>
 <td>50</td>
 <td>51</td>
 <td>min(50, 51)</td>
 </tr>
 <tr>
 <td>51</td>
 <td>52</td>
 <td>min(51, 52)</td>
 </tr>
 <tr>
 <td>52</td>
 <td>53</td>
 <td>min(52, 53)</td>
 </tr>
 <tr>
 <td>53</td>
 <td>54</td>
 <td>min(53, 54)</td>
 </tr>
 <tr>
 <td>54</td>
 <td>55</td>
 <td>min(54, 55)</td>
 </tr>
 <tr>
 <td>55</td>
 <td>56</td>
 <td>min(55, 56)</td>
 </tr>
 <tr>
 <td>56</td>
 <td>57</td>
 <td>min(56, 57)</td>
 </tr>
 <tr>
 <td>57</td>
 <td>58</td>
 <td>min(57, 58)</td>
 </tr>
 <tr>
 <td>58</td>
 <td>59</td>
 <td>min(58, 59)</td>
 </tr>
 <tr>
 <td>59</td>
 <td>60</td>
 <td>min(59, 60)</td>
 </tr>
 <tr>
 <td>60</td>
 <td>61</td>
 <td>min(60, 61)</td>
 </tr>
 <tr>
 <td>61</td>
 <td>62</td>
 <td>min(61, 62)</td>
 </tr>
 <tr>
 <td>62</td>
 <td>63</td>
 <td>min(62, 63)</td>
 </tr>
 <tr>
 <td>63</td>
 <td>64</td>
 <td>min(63, 64)</td>
 </tr>
 <tr>
 <td>64</td>
 <td>65</td>
 <td>min(64, 65)</td>
 </tr>
 <tr>
 <td>65</td>
 <td>66</td>
 <td>min(65, 66)</td>
 </tr>
 <tr>
 <td>66</td>
 <td>67</td>
 <td>min(66, 67)</td>
 </tr>
 <tr>
 <td>67</td>
 <td>68</td>
 <td>min(67, 68)</td>
 </tr>
 <tr>
 <td>68</td>
 <td>69</td>
 <td>min(68, 69)</td>
 </tr>
 <tr>
 <td>69</td>
 <td>70</td>
 <td>min(69, 70)</td>
 </tr>
 <tr>
 <td>70</td>
 <td>71</td>
 <td>min(70, 71)</td>
 </tr>
 <tr>
 <td>71</td>
 <td>72</td>
 <td>min(71, 72)</td>
 </tr>
 <tr>
 <td>72</td>
 <td>73</td>
 <td>min(72, 73)</td>
 </tr>
 <tr>
 <td>73</td>
 <td>74</td>
 <td>min(73, 74)</td>
 </tr>
 <tr>
 <td>74</td>
 <td>75</td>
 <td>min(74, 75)</td>
 </tr>
 <tr>
 <td>75</td>
 <td>76</td>
 <td>min(75, 76)</td>
 </tr>
 <tr>
 <td>76</td>
 <td>77</td>
 <td>min(76, 77)</td>
 </tr>
 <tr>
 <td>77</td>
 <td>78</td>
 <td>min(77, 78)</td>
 </tr>
 <tr>
 <td>78</td>
 <td>79</td>
 <td>min(78, 79)</td>
 </tr>
 <tr>
 <td>79</td>
 <td>80</td>
 <td>min(79, 80)</td>
 </tr>
 <tr>
 <td>80</td>
 <td>81</td>
 <td>min(80, 81)</td>
 </tr>
 <tr>
 <td>81</td>
 <td>82</td>
 <td>min(81, 82)</td>
 </tr>
 <tr>
 <td>82</td>
 <td>83</td>
 <td>min(82, 83)</td>
 </tr>
 <tr>
 <td>83</td>
 <td>84</td>
 <td>min(83, 84)</td>
 </tr>
 <tr>
 <td>84</td>
 <td>85</td>
 <td>min(84, 85)</td>
 </tr>
 <tr>
 <td>85</td>
 <td>86</td>
 <td>min(85, 86)</td>
 </tr>
 <tr>
 <td>86</td>
 <td>87</td>
 <td>min(86, 87)</td>
 </tr>
 <tr>
 <td>87</td>
 <td>88</td>
 <td>min(87, 88)</td>
 </tr>
 <tr>
 <td>88</td>
 <td>89</td>
 <td>min(88, 89)</td>
 </tr>
 <tr>
 <td>89</td>
 <td>90</td>
 <td>min(89, 90)</td>
 </tr>
 <tr>
 <td>90</td>
 <td>91</td>
 <td>min(90, 91)</td>
 </tr>
 <tr>
 <td>91</td>
 <td>92</td>
 <td>min(91, 92)</td>
 </tr>
 <tr>
 <td>92</td>
 <td>93</td>
 <td>min(92, 93)</td>
 </tr>
 <tr>
 <td>93</td>
 <td>94</td>
 <td>min(93, 94)</td>
 </tr>
 <tr>
 <td>94</td>
 <td>95</td>
 <td>min(94, 95)</td>
 </tr>
 <tr>
 <td>95</td>
 <td>96</td>
 <td>min(95, 96)</td>
 </tr>
 <tr>
 <td>96</td>
 <td>97</td>
 <td>min(96, 97)</td>
 </tr>
 <tr>
 <td>97</td>
 <td>98</td>
 <td>min(97, 98)</td>
 </tr>
 <tr>
 <td>98</td>
 <td>99</td>
 <td>min(98, 99)</td>
 </tr>
 <tr>
 <td>99</td>
 <td>100</td>
 <td>min(99, 100)</td>
 </tr>
 </tbody>
 </table>
 </div>
 </div>
);
};

```

```

 — это минимум на отрезке{" "}

 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\text{hover.j}} = \{1 \ll \text{hover.j}\} \cdot \{ " \}$

 min({arr1D.slice(covered.from, covered.to)})

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое число
 </p>
)}
</div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-50">
 <div className="grid grid-cols-[auto_repeat(4,1fr)]">
 { /* Headers */ }
 <div className="text-slate-500 font-mono text-sm">
 i \ j
 </div>
 <div className="text-center font-bold text-slate-500">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^3 (L=8)
 </div>
 </div>
 </div>
</div>

```

```

 isSrc2 = true;
 }
 }

 return (
 <div key={j} className="relative flex items-center justify-center"
 {!isValid ? (
 <div className="w-[clamp(30px,9vw,17px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors duration-200"
 > {st1D[i][j]}
 </div>
) : (
 <motion.div
 onMouseEnter={() => setIsVisible(true)}
 onMouseLeave={() => setIsVisible(false)}
 onClick={() => setIsVisible(!isSrc1 && !isSrc2)}
 initial={{ opacity: 0, scale: 0.8 }}
 animate={{
 opacity: isVisible ? 1 : 0,
 scale: isVisible ? 1 : 0.8,
 }}
 className={`w-[clamp(30px,9vw,17px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors duration-200
 ${isSrc1 ? "bg-sky-500 text-white" : isSrc2 ? "bg-indigo-500 text-white" : isSrc1 && isSrc2 ? "bg-emerald-500 text-white" : isVisible ? "bg-amber-500 text-white" : "bg-slate-800 text-white"}
 `}
 > {st1D[i][j]}
 </motion.div>
)}
)
 }
}

/* Arrows visualization */

```

```


 ST[{c.i}][{c.j}]
 { " " }
 = min(
 <span className="text-emerald-400 font-b
 ST[{c.i}][{c.j - 1}]

 ,
 <span className="text-amber-400 font-bo
 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1

 {st1D[c.i + half][c.j - 1]}

) ={ " " }
 <span className="text-indigo-400 font-b
 {st1D[c.i][c.j]}

 </p>
);
))()
) : {
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
 })
</div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: num
 const [locked, setLocked] = useState<{ r: number; c: r

```

```

<h4 className={`font-bold mb-3 flex
 }>
 Матрица блоков {stepL}x{step
 <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
 </h4>
 <div className="max-w-full overflow-
 <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

 let highlightClass = isCurr ? '
/50 cursor-pointer' : 'bg-slate-800/50 text
 let zIndex = 'z-0';

 const isActive = !!activeCell &&
ll.c + L) && ((r - activeCell.r) % (1 << st

 if (isActive) {
 if (isCurr) {
 highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (k - 1);
 const isTop = r < activeCel
 const isLeft = c < activeCe

 if (isTop && isLeft) {
 highlightClass = 'bg-ros
md:scale-[1.08]';
 } else if (isTop && !isLeft
 highlightClass = 'bg-blur
] md:scale-[1.08]';
 } else if (!isTop && isLeft

```





```

 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
)
 }}()}}
 </div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L
 </div>
)}
 </div>
)}
 </div>
 </div>
 </div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);

 const H = r2 - r1 + 1;

```

```

 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
 />

 /* Показываем 4 угла в самой матрице
 <div className="absolute pointer-events-none shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(
 <div className="absolute pointer-events-none shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(
 width: `calc(
 <div className="absolute pointer-events-none shadow-[0_0_10px_#10b981]" style={{ top: `calc(
 8px)` , width: `calc(
 <div className="absolute pointer-events-none shadow-[0_0_10px_#f59e0b]" style={{ top: `calc(
 00}% + 8px)` , width: `calc(
 </div>

 <div className="bg-slate-950 p-4 rounded-2xl shadow-[inset_0_2px_10px_rgba(0,0,0,0.5)]" style={{
 <label className="flex items-center justify-between" style={{ width: 100%; }}>

 <div className="flex gap-2">
 <input type="number" min=0 max=100 value=0 />
 </div>

 <div className="flex gap-2">
 <input type="number" min=0 max=100 value=0 />
 </div>

 </label>
 <label className="flex items-center justify-between" style={{ width: 100%; }}>

 <div className="flex gap-2">
 <input type="number" min=0 max=100 value=0 />
 </div>

 </label>

```

```

В запроса)

 <span className="text-rose
r1][<span className="text-white bg-r

 <span className="text-slate
во, чтобы правый край уперся в c2)<b
 <span className="text-blue
r1][<span className="text-white bg-b

 <span className="text-slate
рх, чтобы нижний край уперся в r2)<b
 <span className="text-emerg
nded">r2 - Lx + 1][<span className="

 <span className="text-slate
и влево от c2)

 <span className="text-amber
">r2 - Lx + 1][<span className="text
{'}}')});
</div>

<div className="w-full flex justify-cent
<div className="flex flex-col items-cent
<h4 className="text-slate-400 font-bo
 Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][{{kx}}][{{ky}}]
</h4>
 <div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}}>
 {Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - Lx + 1 && c === c1;
 let isBR = r === r2 - Lx + 1 && c === c2;

 let color = "bg-slate-800 text-

```

```

 + 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
 </div>
 </div>

 </div>
</div>
)
}
```

=====

## РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

=====

-----

ФАЙЛ 73/83: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОВОК: 137 | РАЗМЕР

-----

```
 </div>
 </main>
</div>

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').cla
 ? 'px-3 py-1 rounded text-sm font-bold l
 : 'px-3 py-1 rounded text-sm font-bold

 document.getElementById('btn-mode-wtf').cla
 ? 'px-3 py-1 rounded text-sm font-bold l
 : 'px-3 py-1 rounded text-sm font-bold

 const aside = document.querySelector('aside
 const headerMain = document.querySelector('
 const questionsContainer = document.getElem
 const wtfContainer = document.getElementById
```

```

 document.getElementById('btn-theme-light').addEventListener('click', () => {
 document.body.classList.remove('ext-white transition-colors') ; 'px-3 py-1 rounded transition-colors';
 document.getElementById('btn-theme-notebook').addEventListener('click', () => {
 document.body.classList.add('ext-white transition-colors') ; 'px-3 py-1 rounded transition-colors';
 })
 })

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu-btn');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer-btn');
 const drawerOverlay = document.getElementById('drawer-overlay');

 function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('closed');
 if (isClosed) {
 mobileDrawer.classList.remove('closed');
 if (drawerOverlay) {
 drawerOverlay.classList.remove('hidden');
 // Timeout allows display:block to take effect
 setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
 }
 document.body.style.overflow = "hidden";
 } else {
 mobileDrawer.classList.add('closed');
 if (drawerOverlay) {
 drawerOverlay.classList.add('hidden');
 setTimeout(() => drawerOverlay.classList.add('hidden'), 10);
 }
 document.body.style.overflow = "";
 }
 }

 mobileMenuBtn.addEventListener('click', toggleDrawer);
 closeDrawerBtn.addEventListener('click', toggleDrawer);
 drawerOverlay.addEventListener('click', toggleDrawer);
});

```

```
 });
 }, observerOptions);

 document.querySelectorAll('section[id^="que
 observer.observe(section);
 });
});
</script>
</body>
</html>
```

---

ФАЙЛ 74/83: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОВОК: 1169 | РАЗМЕР: 10.5 КБ

---

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
 <title>Пособие по алгебре: От пиццы к предикатам</title>
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js"></script>
 <script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3.2.1/es5/tex-mml-chtml.js"></script>
 <script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']],
 displayMath: [['$$', '$$'], ['\\[', '\\]']],
 },
 };
 </script>
 <style>
 @import url('https://fonts.googleapis.com/css2?family=Fira+Code&display=block');
 @reference './src/index.css';
 .math-font { font-family: 'Fira Code', monospace; }
 </style>
```

```

 -webkit-overflow-scrolling: touch;
 }
 mjax-container[display="true"]::-webkit-scrollbar {
 display: none;
 }

 @media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font-size: 12px; }
 .uno-card::before, .uno-card::after { font-size: 12px; }
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px; }

 .uno-card.mini { width: 28px !important; height: 28px !important; font-size: 4px; }
 .uno-card.mini::before, .uno-card.mini::after { font-size: 4px; }

 .uno-equation .text-2xl { font-size: 1.2rem; }
 .uno-equation.gap-4 { gap: 0.5rem !important; }

 /* Allow horizontal scroll for equations in formula scroll */
 .formula-scroll { flex-wrap: nowrap !important; }
 }

 .uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }

 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-blue { background-color: #004dcf; } .card-green { background-color: #339e35; } .card-yellow { background-color: #ffc400; } .card-black { background-color: #222; }
 .card-black::before, .card-black::after { text-align: center; }
 .info-block { @apply bg-slate-800/80 border-l-4

```

```

 }
 body.theme-notebook .text-slate-200, body.theme
 ant; }
 body.theme-notebook .text-slate-400 { color: #5
 body.theme-notebook .border-slate-600, body.the
 : #94a3b8 !important; }
 body.theme-notebook .font-mono { font-family: "
 body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1);
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
}
.creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
}
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead
 <div class="sticky top-0 z-50 w-full bg-slate-950/9
 center items-center gap-4 sm:gap-8 transition-col
 <div class="flex items-center gap-3">

```



```

</marker>
<marker id="arrow-lime" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d" stroke="black" stroke-width="1" />
</marker>
<marker id="arrow-green" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" stroke="black" stroke-width="1" />
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20" fill="#fde725" stroke="black" stroke-width="1" />
 <rect x="6" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <rect x="26" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
</g>
<g id="lego-red">
 <rect x="0" y="0" width="40" height="20" fill="#e34a33" stroke="black" stroke-width="1" />
 <rect x="6" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <rect x="26" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
</g>
<g id="lego-rem">
 <rect x="0" y="0" width="40" height="10" fill="white" stroke="black" stroke-width="1" />
 <rect x="6" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <rect x="26" y="-5" width="8" height="6" fill="black" stroke="black" stroke-width="1" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">
 <path d="M 10 40 L 15 20 Q 25 15 35 20" fill="white" stroke="black" stroke-width="1" />
 <rect x="5" y="40" width="40" height="20" fill="white" stroke="black" stroke-width="1" />
 <!-- Ручка -->
 <path d="M 45 50 Q 60 50 60 30 L 75 30" fill="white" stroke="black" stroke-width="1" />
 <circle cx="75" cy="30" r="4" fill="#333" stroke="black" stroke-width="1" />
 <!-- Выход -->
 <path d="M 5 45 L -5 45 L -5 55 L 5 55" fill="white" stroke="black" stroke-width="1" />
 <circle cx="25" cy="50" r="15" fill="white" stroke="black" stroke-width="1" />
</g>

```

```
 <button id="close-drawer-btn" class="
 <svg xmlns="http://www.w3.org/20
 <path stroke-linecap="round
 </svg>
 </button>
 </div>
 <nav class="space-y-3 text-sm font-medium
 <details class="group" open>
 <summary class="list-none cursor

 1. Алгебраические структуры

 </summary>
 <div class="pl-6 space-y-2 text-sm">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor

 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text-sm">

 </div>
 </details>

 <details class="group">
```

```


 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-lime-500 pl-3 font-bold text-lime-300">

 6. Делимость, НОД и Евклид

 </summary>
 <div class="pl-6 space-y-2 text-lime-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-pink-500 pl-3 font-bold text-pink-300">

 7. Взаимно простые числа

 </summary>
 <div class="pl-6 space-y-2 text-pink-300">


```

```


 10. Делимость и Схема Г

 </summary>
 <div class="pl-6 space-y-2 text">
 <a href="#q10-1" class="blo
 <a href="#q10-2" class="blo
 <a href="#q10-3" class="blo
a>
 <a href="#q10-4" class="blo
 <a href="#q10-5" class="blo
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-11" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
 11. НОД и НОК многочлен

 </summary>
 <div class="pl-6 space-y-2 text">
 <a href="#q11-1" class="blo
 <a href="#q11-2" class="blo
нственности НОД
 <a href="#q11-3" class="blo
ПИРАЛЬ)
 <a href="#q11-4" class="blo
вращается!)
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-12" clas
der-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые мно
```

```
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критерий

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многоч.

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <div class="mt-4 mb-2 text-xs font-l
 <a href="#question-17" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-tea

 <a href="#question-18" class="block
```

```

 // Scroll spy logic
 document.addEventListener('DOMContentLoaded', () => {
 const sections = document.querySelectorAll(
 "h2[id^='q5-'], div[id^='q6-'], div[id^='q7-'], div[id^='q8-'], div[id^='q9-'], div[id^='q10-'], div[id^='q11-'], div[id^='q12-'], div[id^='q13-'], div[id^='q14-'], div[id^='q15-']"
);
 let isScrolling = false;

 document.querySelectorAll('.group-item-link').forEach(link => {
 link.addEventListener('click', () => {
 isScrolling = true;
 setTimeout(() => isScrolling = false, 300);
 });
 });

 const observer = new IntersectionObserver(entries => {
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;
 }
 });

 // Highlight active link
 document.querySelectorAll('a').forEach(a => {
 a.style.fontWeight = 'normal';
 if (a.href.endsWith(visibleId)) {
 a.style.fontWeight = 'bold';
 }
 });

 // Open the parent details
 const correspondingLi = document.querySelector(`li[id^='${visibleId}']`);
 if (correspondingLi) {
 const details = correspondingLi.querySelector('details');
 if (details && !details.open) {
 details.open();
 }
 }

 document.querySelector(`div[id^='${visibleId}']`)?.scrollIntoView({
 behavior: 'smooth',
 block: 'center'
 });
 });
 });

```

```
box.style.borderRadius = '10px'
box.style.display = 'flex';
box.style.flexDirection = 'column'
box.style.color = '#fff';

const header = document.createElement('div')
header.innerText = 'Экспорт в '
header.style.marginBottom = '10px'

const subHeader = document.createElement('div')
subHeader.innerText = 'Из-за не
ние веб-студии в новой вкладке (через меню)'
subHeader.style.marginBottom = '10px'
subHeader.style.color = '#f87171'
subHeader.style.fontSize = '0.9em'

const textarea = document.createElement('div')
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px'
textarea.style.fontFamily = 'monospace'

const btnContainer = document.createElement('div')
btnContainer.style.display = 'flex'
btnContainer.style.gap = '10px'
btnContainer.style.marginTop = '10px'

const copyBtn = document.createElement('div')
copyBtn.innerText = 'Копировать'
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = 'white';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer'
copyBtn.onclick = () => {
```

```

 // Smart markdown parsing based
 clone.querySelectorAll('h2').fo
 clone.querySelectorAll('h3').fo
 clone.querySelectorAll('.analog
 clone.querySelectorAll('li').fo
 clone.querySelectorAll('svg').fo
 let caption = '';
 if (el.nextElementSibling &
 caption = el.nextElement
 }
 el.textContent = '\n\n[Илл
 });
 clone.querySelectorAll('.img-cap

let text = clone.innerText;
text = text.replace(/\\n{3,}/g,

if (window !== window.top) {
 fallbackModal(text, 'Markdow
 return;
}
const blob = new Blob([text], {
const url = URL.createObjectURL
const link = document.createEle
link.href = url;
link.download = 'vyisshaya_algel
document.body.appendChild(link)
link.click();
document.body.removeChild(link)
}

function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate

);

document.getElementById('theme-

```



```

body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuch
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

`
} else if (theme === 'terminal'
body.classList.add('bg-blac
document.getElementById('th
style.innerHTML = `
body.bg-black .bg-slate
.bg-slate-950 {

```

```

 <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-500">
 <div class="flex items-center gap-2">
 <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
 <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
 <button id="theme-terminal" onclick="toggleTheme('terminal')"
-center justify-center text-green-400 hover:text-green-500
терминала"> </button>
 </div>
 </div>
 </div>
</div>

</div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
-base min-w-0">
 <header class="text-center mb-12 border-b border-slate-700">
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">
 Welcome to the world of Higher Algebra
 </p>

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-slate-400">
 <table border="0">
 <tr>
 <td>Chapter 1: Introduction</td>
 <td>Chapter 1</td>
 </tr>
 <tr>
 <td>Chapter 2: Linear Algebra</td>
 <td>Chapter 2</td>
 </tr>
 <tr>
 <td>Chapter 3: Matrix Theory</td>
 <td>Chapter 3</td>
 </tr>
 <tr>
 <td>Chapter 4: Vector Spaces</td>
 <td>Chapter 4</td>
 </tr>
 <tr>
 <td>Chapter 5: Linear Transformations</td>
 <td>Chapter 5</td>
 </tr>
 <tr>
 <td>Chapter 6: Eigenvalues and Eigenvectors</td>
 <td>Chapter 6</td>
 </tr>
 <tr>
 <td>Chapter 7: Quadratic Forms</td>
 <td>Chapter 7</td>
 </tr>
 <tr>
 <td>Chapter 8: Bilinear Forms</td>
 <td>Chapter 8</td>
 </tr>
 <tr>
 <td>Chapter 9: Symmetric Matrices</td>
 <td>Chapter 9</td>
 </tr>
 <tr>
 <td>Chapter 10: Diagonalization</td>
 <td>Chapter 10</td>
 </tr>
 <tr>
 <td>Chapter 11: Canonical Forms</td>
 <td>Chapter 11</td>
 </tr>
 <tr>
 <td>Chapter 12: Applications</td>
 <td>Chapter 12</td>
 </tr>
 </table>
 </div>
 </div>

```

```

 олько полиномов</div>
 <div class="bg-cyan-900/40 border-4 border-yellow-500 hover:bg-slate-700 transition" style="border: 4px solid yellow; background-color: cyan; color: yellow; padding: 10px; text-align: center;">
 сные числа</div>

 <!-- Row 1: База -->

 -4 border-yellow-500 hover:bg-slate-700 transition" style="border: 4px solid yellow; background-color: cyan; color: yellow; padding: 10px; text-align: center;">
 <div class="text-xs text-yellow" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">

 -4 border-orange-500 hover:bg-slate-700 transition" style="border: 4px solid orange; background-color: cyan; color: orange; padding: 10px; text-align: center;">
 <div class="text-xs text-orange" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">

 -4 border-cyan-500 hover:bg-slate-700 transition" style="border: 4px solid cyan; background-color: cyan; color: cyan; padding: 10px; text-align: center;">
 <div class="text-xs text-cyan-500" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">

 <!-- Row 2: Базовые операции / Евклид -->

 -4 border-yellow-500 hover:bg-slate-700 transition" style="border: 4px solid yellow; background-color: cyan; color: yellow; padding: 10px; text-align: center;">
 <div class="text-xs text-yellow" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">

 -1-4 border-orange-500 hover:bg-slate-700 transition" style="border: 4px solid orange; background-color: cyan; color: orange; padding: 10px; text-align: center;">
 <div class="text-xs text-orange" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">

 -4 border-cyan-500 hover:bg-slate-700 transition" style="border: 4px solid cyan; background-color: cyan; color: cyan; padding: 10px; text-align: center;">
 <div class="text-xs text-cyan-500" style="margin-bottom: 5px;">
 <div class="font-bold text-slate-900" style="margin-bottom: 5px;">


```

```

<a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-500"
 <div class="font-bold text-slate-700"

<!-- Row 6: Кратность -->
<div class="hidden md:block"></div>
<a href="#question-14" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange-500"
 <div class="font-bold text-slate-700"

<div class="hidden md:block"></div>

<!-- Row 7: Производная -->
<div class="hidden md:block"></div>
<a href="#question-15" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange-500"
 <div class="font-bold text-slate-700"

<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-3"
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setM
-l-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-500"
 <div class="font-bold text-slate-700"

 <a href="#question-18" onclick="setM
-l-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-500"
 <div class="font-bold text-slate-700"


```

```

 <svg class="w-8 h-8 text-red-500" data-bbox="110 110 190 190">
 <path stroke-linecap="round" stroke-width="2" d="M110 40 190 190" />
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border-l-2 border-l-blue-500" data-bbox="210 210 290 290">
 </div>
 </div>

```

```

 <div class="prose prose-invert max-w-4xl" data-bbox="310 310 390 390">
 <p class="text-lg" data-bbox="330 330 410 410">
 Хватит тупить. Доказательств
 ь для того, чтобы жать на кнопки. Вот жес
 </p>
 </div>

```

```

 <h2 class="text-2xl font-bold border-l-2 border-l-blue-500" data-bbox="450 450 530 530">
 (Меметика)</h2>
 </div>

```

```

 <!-- Прямое -->
 <div class="bg-slate-800/80 p-6" data-bbox="550 550 630 630">
 <h3 class="text-xl text-blue-400" data-bbox="570 570 650 650">
 <p class="text-sm text-slate-400" data-bbox="590 590 670 670">
 <ul class="list-disc pl-5" data-bbox="610 610 690 690">

 k$).

 ово.

 </p>
 </h3>
 </div>

```

```

 <!-- От противного -->
 <div class="bg-slate-800/80 p-6" data-bbox="750 750 830 830">
 <h3 class="text-xl text-fuchsia-400" data-bbox="770 770 850 850">
 <p class="text-sm text-slate-400" data-bbox="790 790 870 870">
 <ul class="list-disc pl-5" data-bbox="810 810 890 890">

 i>

```

```
<ul class="list-disc pl-5 s
 <span class="text-w
вверх по уравнениям).
 <span class="text-w
рху вниз, показывая, что $\Delta \vdots r_i$
 <span class="text-w
х. Твой d больше.

</div>
```

```
<h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
со следующим. Если в конце 0 – пациент мертв
 </div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
 c . Пока получается 0 – корень жив. Как то
корня минус один.</p>
 </div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
ициентов (они делятся на p), кроме старше
зложить.</p>
 </div>
```

```
 <h4 class="text-cyan">
 <p class="text-sm">
есть сложение — есть ноль. Если есть умноже
 <code class="text-x
 </div>

 <li class="bg-slate-800/50">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
авь их драться. Умножение всегда врывается
 <code class="text-x
 </div>

</div>
</section>
</div>

<div id="questions-container">
<!-- ===== ВОПРОС 1 =====
```

---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 75/83: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```

 } catch {
 /* чужой домен — пропускаем */
 }
 }
 }
 return parts.join("\n");
 }

const EXTRA_CSS = `
/* — правки для автономного файла — */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid #
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
}
`;

const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");

```





```

const css = await collectCss();
const { body, termIds } = prepareBody(chapter.content);
const html = wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
});
triggerDownload(html, safeName(chapter.title, chapter
}

```

/\*\* Выгрузить всё пособие одним файлом со сквозным оглавлением \*/

```

export async function downloadBookHtml(chapters: Chapter[]) {

```

```

 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];

 const toc = chapters
 .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}">${c.title}</a>`
).join("\n");

 chapters.forEach((c) => {
 const { body, termIds } = prepareBody(c.content);
 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="border: 1px solid #0f172a; padding: 10px 10px 0 10px; margin-top: 10px;">
 <p style="text-align:right;font-size:.72rem;margin:0 0 10px 0">
 ${c.title}
 </p>
 ${body}
 </div>`);
 });

```

```

 const body = `<nav id="toc" style="background:#0f172a;color:#fff;padding:10px 10px 0 10px; margin-bottom: 10px;">
 <h2 style="color:#fff;font-size:1.05rem;font-weight:800;margin:0 0 10px 0">Оглавление
 <ol style="margin:0;padding-left:1.1rem;font-size:.85rem">
 ${toc}

 </nav>
 ${bodies.join("\n")}`;

```

```

const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\\s{2

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^(\d+)\.\/)?.[1]
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && l
const gapViz = rows.filter((r) => r.hasAnalogy && !has2

```

## Универсальное пособие • полный исходный код

```

* в каждой ли теме находятся термины из глоссария.
*
* node scripts/audit-terms.mjs
*/
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossa",
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".m",
 external: ["react", "react-dom", "motion", "lucide-re",
});

const { chapters } = await import(path.join(ROOT, "tmp/");
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
 `(?!<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
 "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/>/gi, " ")
 .replace(/<[^\>]+\>/g, " ");

 const perTerm = new Map();
```

## Универсальное пособие • полный исходный код

```

* в единый документ public/export/full_code_all_pages.js
*/
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
 }

 const pages = fs
 .readdirSync(PAGES_DIR)
 .filter((f) => f.endsWith(".png"))
 .sort();

 if (pages.length === 0) throw new Error("Не найдено ни одного файла");

 ensureDir(OUT_DIR);

 const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 });

 const stream = fs.createWriteStream(PDF_PATH);
 doc.pipe(stream);
```

```
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
 [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
 [/^src\/data\/$/, "Контент и данные пособия"],
 [/^src\/components\/visualizers\/$/, "Визуализаторы (в
 [/^src\/components\/$/, "Интерактивные компоненты и сим
 [/^src\/layout\/$/, "HTML-разметка (layout)"],
 [/^src\/utils\/$/, "Утилиты"],
 [/^src\/$/, "Ядро приложения"],
 [/^scripts\/$/, "Скрипты сборки и экспорта"],
 [/^\.github\/$/, "CI / автоматизация"],
 [/^[^\/]+\$/ , "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re

/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
```

```

 return ia !== ib ? ia - ib : a.localeCompare(b);
 });
}

```

/\*\* Достаём заголовки глав из данных пособия (без запус

```

function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("s
 const src = fs.readFileSync(path.join(ROOT, rel), "
 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
 }
 return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
 return na - nb;
 }));
}

```

```

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { ti
 const totalBytes = files.reduce((s, f) => s + fs.stat

 push(HR);
}

```

```

 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 push(HR);
 push();
 }
 const content = fs.readFileSync(path.join(ROOT, rel));
 const lines = content.split("\n");
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
});

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`    файлов:  ${files.length}`);
console.log(`    глав:    ${chapters.length}`);
console.log(`    строк:   ${txt.split("\n").length}`);
console.log(`    выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```



```
};
```

```
export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);
```

```
export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}
```

```
export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}
```

```
/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 for (const cmd of ["magick", "convert"]) {
 try {
 execFileSync(cmd, ["-version"], { stdio: "ignore" });
 return cmd;
 } catch {
 /* пробуем следующий */
 }
 }
 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick"
);
}
```

---

ФАЙЛ 82/83: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 1.5 KB

---

```
/**
 * ШАГ 2: TXT -> PNG
```

```
 pages.push(source.slice(i, i + PAGE.rows));
 }
 return pages;
}

async function main() {
 if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}
 }

 const im = imageMagickCmd();
 const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

 fs.rmSync(PAGES_DIR, { recursive: true, force: true });
 ensureDir(PAGES_DIR);

 const total = pages.length;
 const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

 const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "DejaVuSansMono",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
];
```

---

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

---

---

ФАЙЛ 83/83: `.github/workflows/rebuild-pdf.yml`

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 128 | РАЗМЕР: 4.0 КБ

---

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (
#
|
|→ full_code_all_page
```

# Готовые TXT и PDF коммитаются обратно в ветку (их отда

# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

description: "DPI растеризации страниц (по умол

required: false

default: "150"

- ```
done
convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`\${public}/export/full\_code\_all\_pages`"
 echo "| \`\${public}/export/full\_code\_all\_pages`"
 echo "| PNG-страниц | \$(ls tmp/export-pages.
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v4
with:
 name: full-code-export
 path: |